APPUNTI PRESI A LEZIONE

FONDAMENTI DI ANALISI DATI

Santo Lo Bianco

INTRODUZIONE

Siamo circondati da dati tutti i giorni, e ogni nostra operazione online, seppur la più stupida, potrà essere soggetta a stoccaggio e infine utilizzata a seconda dello scopo.

I contenuti che sono proposti dalle applicazioni, le pubblicità invasive e tutte le ulteriori personalizzazioni derivano dall'elaborazione dei dati che sono stati conservati.

I campioni raccolti, oltre ad essere utilizzati a scopo commerciale, sono anche ampiamente utilizzati in campi come:

- Salute -> traccia delle malattie epidemiologiche per capire dove e quando vaccinare, parametri vitali effettuata dagli smartwatch;
- Sondaggi e opinione pubblica ->campioni per stimare l'opinione pubblica, sondaggi che influenzano anche le strategie per campagne aziendali;
- Elezioni -> influenzare le persone che non appartengono agli zoccoli duri dei vari partiti;
- Clima e ambiente -> sulla base delle informazioni e l'elaborazione eseguite potremmo fare delle stime sul clima, si analizza costantemente il fenomeno della deforestazione;
- Finanza -> consulenze senza intervento umano costruito sui dati acquisiti nel tempo, algoritmi di rilevazione frodi.

L'intelligenza artificiale, che in questo momento è in forte sviluppo, è alimentata dai dati che vengono utilizzati per perfezionare e convalidare l'AI.

L'intelligenza artificiale può essere di due tipi:

- **Simbolica** -> si basa su regole logiche esplicite scritte da esseri umani(potrebbe essere inteso come un grande programma già scritto). I pro sono fortemente legati al determinismo, dato che con il tempo non cambia, e possono essere ripercorsi tutti i passaggi logici che hanno indotto l'intelligenza a rispondere in un certo modo. I contro sono: l'impossibilità di scalare e aggiornare, la non comprensione di ambiguità data dai linguaggi naturali e l'impossibilità di imparare.
- **Statistica** -> impara automaticamente dai dati, attraverso l'utilizzo di modelli probabilistici e numerici che riconoscono dei pattern. Si evolve nel tempo ed è molto flessibile a linguaggi naturali ecc.; ma potrebbe non essere trasparente il funzionamento e può anche imparare errori.

L'intelligenza artificiale statistica riceve in pasto enormi quantità di dati, spesso pari a migliaia di terabyte. Essa non memorizza i dati in modo diretto, ma li utilizza per individuare schemi e relazioni. In questo modo impara a predire, secondo leggi probabilistiche, quale parola o elemento verrà dopo. L'obiettivo non è ricordare, ma generare risposte basate sui modelli linguistici che ha interiorizzato durante l'addestramento.

I dati sono una raccolta di valori che sono ottenuti rispetto variabili che descrivono un certo fenomeno. Per studiare un fenomeno è necessario raccogliere dati, analizzarli e trarre delle conclusioni.

Si considerano tre concetti fondamentali per l'analisi dei dati, ovvero:

- **Popolazione**: insieme completo di individui che rispettano una certa condizione;
- **Osservazione**: da una raccolta di dati si si osserva il singolo elemento;
- **Campione**(samples): un sottoinsieme della popolazione per osservare un fenomeno.

Come si decide se il campione è valido per studiare un fenomeno? Come decido il numero di individui da prendere dall'insieme popolazione? Con la **statistica inferenziale** effettuiamo una stima, a partire dai dati di un campione, per trarre conclusioni sulla popolazione.

La grandezza del campione influenza l'affidabilità della stima e il costo impiegato.

- Più grande: affidabilità maggiore ma più costosa;
- Più piccola: affidabilità minore ma più economica;

Formalizzando l'analisi dei dati e rendendo non ambigui alcune parti citate prima, di seguito sono proposte alcune definizioni.

Una **variabile** è una **funzione che mappa ogni osservazione in** un valore **specifico**.

Si considera l'insieme **Ω** che corrisponde alla **popolazione** soggetta ad analisi, e **S** è il **set** dei possibili **valori**.

$$\mathbf{X: \Omega \rightarrow S}$$

$$\boldsymbol{\omega} \rightarrow \mathbf{s}$$

X nome della variabile(funzione applicata alla popolazione) x valore che assume

Esempio:

$$A: \Omega \rightarrow N$$

$$\boldsymbol{\omega} \rightarrow a(\text{anni di una persona})$$

I tipi di variabili possono essere:

- **qualitativi**: descrive le qualità che non possono essere numericamente misurate e ordinate, es. SEX(male,female);
- **quantitativi**: descrive quantità che possono essere numericamente misurate e ordinate, es. Age(25,ecc.);
- **discreti**: descrive una quantità di numeri finita o infinita che contengono dei buchi in mezzo tra un elemento e un altro, es. Age(25,26,27) tra un numero e un altro non ci sono infiniti elementi;
- **continui**: descrive una quantità di numeri infinita anche solo tra un numero di un intervallo e un altro, spesso utilizzato per misure che contengono i decimali;
- **scalari**: singola unità di dato;
- **multidimensionali**: composto da più componenti per ogni valore, che rappresentano spesso aspetti differenti della stessa entità, es RGB(255,255,255);

Oltre i tipi di variabili, si deve anche analizzare la **scala di misura**, ovvero il tipo di informazioni che essa contiene e quale operazione matematica/statistica possiamo applicare su di essa. Di seguito, abbiamo le scale di misura principali:

- **Nominale** -> Valori non ordinabili(valori non numerici), dove i valori sto distinti ma non si possono ordinare dal migliore al peggiore, al massimo si possono contare;
- **Ordinale** -> valori ordinabili con differenze che non hanno un significato numerico preciso. Gli intervalli tra i livelli non hanno necessariamente lo stesso valore, ad esempio tra "abbastanza"- "molto" e "poco"-"abbastanza", le differenze possono non essere le stesse;
- **Intervallo** -> valori ordinabili con differenze significative e lo 0 non assume l'assenza di qualcosa(es. nei gradi lo zero è un valore come tutti gli altri);
- **Ratio** -> valori ordinabili con differenze significative ed esiste lo 0 che indica l'assenza di qualcosa;

Per collezionare dati si possono intraprendere strade diverse, la cui scelta è fortemente influenzata: dalle disponibilità, dagli obiettivi e dai mezzi adoperati.

I metodi chiave sono i seguenti:

Sondaggi(surveys)	Form online, questionari cartacei o interviste telefoniche.	I vantaggi sono dati: dai costi bassi(specialmente online), dalla stardidazione e versatilità.	
Esperimenti	Esperimenti controllati o randomised controlled trials(RCTs).	Si ha un elevato controllo sulle variabili e la misura precisa per la determinazione di eventi causa-effetto.	
Studi osservazionali	Si osservano gli individui senza artefatti.	Dati realistici, non si ha controllo.	
Risorse online	Risorse online prelevati mediante api o web scraping(scraper)	Costi inferiori e fatica ridotta.	

Un insieme di dati relativi ad uno specifico fenomeno è chiamato dataset. I dataset solitamente vengono conservati su delle tabelle(simile a basi), dove:

- le colonne rappresentano le diverse variabili;
- le righe rappresentano le diverse osservazioni

Un dataset potrebbe essere rappresentato dai fogli di calcolo di excel.

Account Number	Total Amount	Outstanding balance
00000001	1500,50	200,00
00000002	5012,24	1400,00
00000003	702,64	

La tabella che ospita il dataset si chiama **design matrix**.

Il dato mancante si chiama **missing data**, e solitamente si ha o a causa di un errore durante il prelievo del dato, oppure l'osservazione non prevedeva quel dato. Possiamo ricostruire il dato mancante attraverso delle regole, infatti in questo caso specifico, se l'utente non ha saldo dovuto allora esso sarà pari a 0.

Possiamo lavorare con un dataset famoso, ma macabro, che al suo interno contiene variabili quantitative e qualitative e inoltre molte delle scale di misura sono presenti. Questo dataset è quello del Titanic.

Per caricare il csv ci serviremo della libreria pandas in Python. I valori NaN specificano valori mancanti, missing data.

Se sarà possibile si sceglierà un index, che sarà estrapolato dal dataset ed usato come key per la ricerca attraverso **.loc[].**
Altrimenti si utilizzerà **.iloc[]** che semplicemente effettuerà la ricerca per numero di riga o colonna, partendo sempre da 0.

```
8    print(titanic['Name'].loc[3])
9    print(titanic['Name'].iloc[3])
PROBLEMI    OUTPUT    TERMINALE    ···

(base) santolobianco@MBP-di-Santo pratica % p
Heikkinen, Miss. Laina
Futrelle, Mrs. Jacques Heath (Lily May Peel)
```

Quando i dati verranno quindi caricati non saranno puliti o già pronti all'uso, ma bensì potranno contenere missing data, formati inconsistenti e altri errori citati prima.

Il **data wrangling** è il processo di trasformazione dei dati grezzi in formati strutturati e utilizzabili per l'analisi e la modellazione. Questo processo può prevedere: conversioni, gestione valori nulli, ridenominazione colonne, applicazione filtri e molto altro.

Applicando il wrangling ci assicuriamo di lavorare con dati ben organizzati e affidabili.

```
#DATA WRANGLING
titanic_clean = titanic.dropna()
age_clean = titanic['Age'].fillna(titanic['Age'].mean())
titanic_clean['Age'].unique()
titanic_clean.rename({'SibSp':'SiblingSpouses'}, inplace= True)
titanic_clean = titanic_clean[titanic_clean['Age']>= 18]
```

I dati possono essere organizzati in modi diversi sulla base degli obiettivi da seguire. Ci sono due formati da seguire:

- **Wide Format**, usato per la modellazione statistica o il machine learning, prevede che:
    - ogni variabile abbia la propria colonna;
    - ogni osservazione è una riga singola;

AccountNumber	CurrentBalance	Annual average statement 2021	Annual average statement 2022	Annual average statement 2023
IT45Q0306909606100000123456	3 480.75	2 950.20	3 120.40	3 370.80
IT22D0501803200000005678901	7 910.20	6 870.55	7 240.90	7 850.30
IT60X0200805201000400123456	1 245.10	980.40	1 120.75	1 210.30
IT89A0103003400000009998887	15 670.55	14 230.10	14 850.90	15 400.25

- **Long Format**, usati per avere dati ordinati tipo nel caso di misurazioni ripetute, prevede che:
    - le variabili sono impilate in meno colonne;
    - un'osservazione può essere presente in più righe;

AccountNumber	CurrentBalance	Year	Annual average statement
IT45Q0306909606100000123456	3 480.75	2022	3 120.40
IT22D0501803200000005678901	7 910.20	2021	6 870.55
IT60X0200805201000400123456	1 245.10	2023	1 210.30
IT89A0103003400000009998887	15 670.55	2021	14 230.10
IT22D0501803200000005678901	7 910.20	2022	7 240.90
IT60X0200805201000400123456	1 245.10	2022	1 120.75

...

Utilizzare il formato wide può rendere i dati più facili da leggere, poiché la loro struttura risulta più intuitiva e il confronto tra le colonne è immediato. Tuttavia, questo formato presenta alcuni limiti: la manipolazione dinamica del dataset è più complessa. Ad esempio, in presenza di osservazioni che variano nel tempo, l'aggiunta di un nuovo anno richiederebbe la creazione di una nuova colonna, rendendo l'aggiornamento dei dati meno agevole.

Per quanto riguarda il formato long possiamo dire che è molto più flessibile e scalabile, ideale per misure ripetute nel tempo ed è perfetto per analisi e grafici. Il problema è che estendere un'osservazione in più righe può comportare ridondanza ergo spreco di memoria.

L'analisi dei dati, quindi, non è una singola operazione, ma una sequenza di operazioni che includono conoscenza del dominio, strumenti computazionali e altre conoscenze/competenze. I passaggi spesso prevedono:

- **Inspecting(ispezione)**: valutazione della struttura e del contenuto del dataset-> numero colonne e righe, tipo variabili, valori e anomalie.
- **Cleaning(pulizia)**: correzione di eventuali anomalie nei dati-> rimozione dei duplicati, gestione dei valori nulli e applicazione filtri alla tabella.
- **Transforming(trasformazione)**: modifica dei dati per renderli adatti alle analisi -> cambiare range, conversione dei formati.
- **Modelling(modellazione)**: applicazione di modelli statistici o di apprendimento automatico per riassumere, spiegare o prevedere aspetti dei dati.

Un **workflow** tipico per l'analisi dei dati prevede diverse fasi che di seguito sono proposte:

- **Definire la domanda di ricerca(analisys question)**: l'obiettivo e le domande a cui vogliamo rispondere.
- **Raccolta dei dati**: si devono ottenere i dati necessari che sono pertinenti e affidabili. La raccolta può avvenire in diversi modi, e solitamente è fortemente condizionata dal primo passaggio.
- **Pulizia e formattazione dei dati**: si correggono errori, si gestiscono dati mancanti e si formattano a seconda dell'uso.
- **Esplorazione e descrizione dei dati**: si analizzano i dati attraverso statistiche descrittive e visualizzazioni. Se l'analisi da esito positivo il dataset segue il workflow in modo naturale, altrimenti si ritorna al primo punto per rivisitare alcuni obiettivi o scelte.
- **Selezione modelli appropriati**: si scelgono il tipo di modelli analitici o statistici più appropriati rispetto gli obiettivi prefissi.
- **Addestramento, ottimizzazione e valutazione dei modelli**: si applicano i vari modelli e si fa una valutazione prestazionale e di solidità.
- **Revisione o aggiornamento dell'analisi**: si migliora il processo all'inserimento di nuovi dati o si rivedono le analisi fatte precedentemente.

Il processo di analisi dei dati non è necessariamente lineare. In alcuni casi, infatti, può emergere la necessità di ridefinire gli obiettivi iniziali qualora quelli stabiliti non risultino raggiungibili o adeguati.

È comune che alcuni problemi non vengano individuati nelle fasi iniziali, ma diventino evidenti solo in quelle successive.

Per questo motivo, il workflow prevede un approccio iterativo, che consente di correggere e migliorare le fasi precedenti sulla base delle nuove informazioni emerse.

Descrizione e visualizzazione dei dati

L'enorme quantità di dati raccolta e raffinata secondo le tecniche del data wrangling, possono non mostrare informazioni significative sulla loro struttura o i pattern che seguono. Per questo motivo, si utilizzano descrizione statistiche e/o tecniche di visualizzazione che ci permettono di farci un'idea del dataset esaminato.

La descrizione dei dati ci permette di:

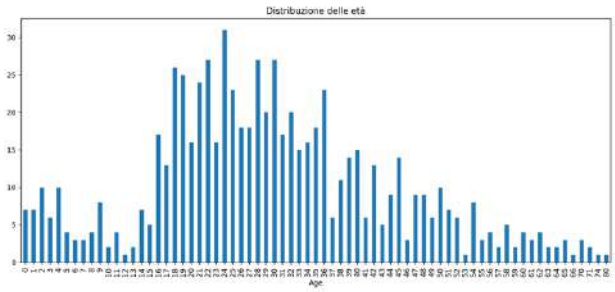
- raccogliere approfondimenti sui dati;
- capire le caratteristiche delle variabili;
- capire il collegamento tra le variabili;
- trovare trends o punti comuni nelle variabili;
- effettuare operazioni pre-modellung;

Un primo passaggio per descrivere i dati è calcolare la "**frequenza assoluta**", ovvero il numero di volte che compare un valore all'interno del nostro data set. Solitamente il calcolo viene effettuato per variabili discrete, ovvero per valori finiti.

Possiamo infine vedere i risultati attraverso un grafico a barre, dove ad ogni valore corrisponde una frequenza.

```
age_count = titanic['Age'].dropna().astype(int).value_counts().sort_index()

from matplotlib import pyplot as plt
age_count.plot.bar(figsize=(18,6))
plt.xlabel('Age')
plt.title('Distribuzione delle età')
plt.show()
```

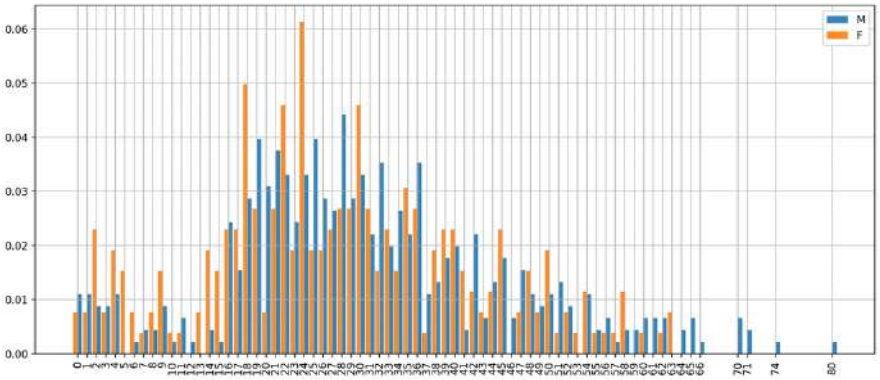
Si occupa di effettuare il calcolo della frequenza assoluta



La frequenza assoluta restituisce in modo preciso il numero di volte che un valore compare nel dataset. Infatti, se volessimo vedere quante persone hanno 24 anni, potremmo usare il grafico e scoprire che sono 28.

Diversa è la **frequenza relativa**, che invece di calcolare il numero preciso di ricorrenze associato ad un valore, calcola la **proporzione** o **percentuale** di osservazioni, che assumono un determinato valore rispetto al totale. *(value_counts(normalize=True))*

Si possono poi utilizzare le frequenze relative e bar chart per mostrare le differenze tra diversi campioni. Ad esempio, possiamo separare le osservazioni per sesso ed effettuare delle comparazioni.



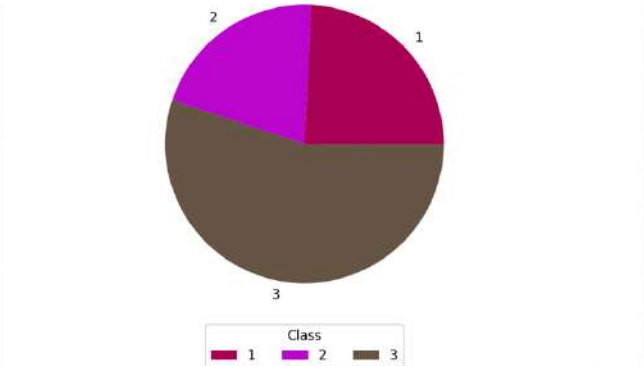
```
sex_counts = titanic.groupby(lambda _: 'All')['Sex'].value_counts(normalize=True).unstack()
sex_counts.plot(kind='bar', stacked=True, color=['yellow','green'])

plt.ylim(0,1)
plt.title("PROPORTION OF MALE AND FEMALE")

plt.legend(title='Sex', loc='upper center', bbox_to_anchor=(0.5,0), ncol=2)
plt.grid(axis='y',linestyle=':',alpha=0.7)
plt.show()
```

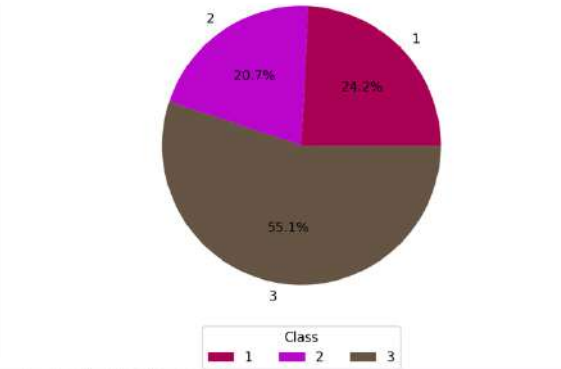
L'operatore **unstack()** viene solitamente usato dopo un **groupby()**, che serve a raggruppare i dati in classi tramite una variabile. Nel codice citato, combinando **unstack()** con **stacked=True**, le frequenze degli elementi di ciascuna classe vengono impilate nel grafico, rendendo più chiaro il contributo di ogni categoria al dataset complessivo.(in questo caso groupby raggruppa tutti gli elementi del database, ma potremmo anche trovarci varianti del tipo groupby('Sex'), ecc.)

Quando i dati non possono essere ordinati lungo un asse o l'ordinamento non ha senso, un'alternativa ai grafici a barre è rappresentarli con un **grafico a torta (pie chart)**. Questo tipo di grafico è particolarmente utile quando ci sono pochi elementi, perché permette di visualizzare le proporzioni in modo semplice e compatto. Si utilizzano soprattutto per dati qualitativi come(contestualizzati all'esempio): sesso o classe dell'imbarco.

Tuttavia, bisogna prestare attenzione: non è sempre immediato capire solo dall'ampiezza degli angoli quanto ogni classe contribuisca al totale.

Per rendere più chiara la lettura, è possibile utilizzare l'attributo **autopct='%1.1f%%'**, che mostra direttamente all'interno del grafico la percentuale di ogni classe.



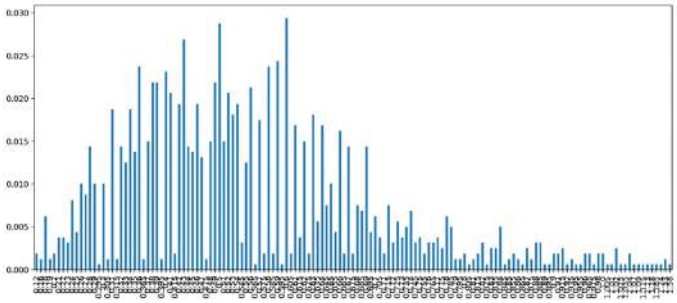


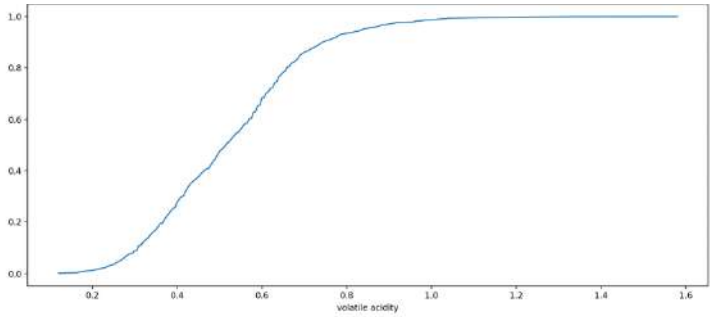
Quando ci sono pochi valori e sono discreti, possiamo calcolare la frequenza assoluta e relativa.

Nel caso in cui i valori siano continui, quindi i dati possono assumere tanti valori diversi, le frequenze per ogni singolo valore potrebbe risultare poco significative. Inoltre, quanto sono più precisi i valori(molte cifre numeriche dopo la virgola) quanto più rumore potrebbe generarsi durante l'acquisizione del campione. Per questo motivo, anche se due valori sono molto vicini, a causa del rumore potrebbero finire in due categorie diverse.

Una soluzione a questo problema è calcolare non le frequenze relative e assolute, bensì la **funzione di distribuzione cumulativa empirica**. Questa funzione, abbreviata in ECDF, consiste nel calcolare per un valore x, la somma delle frequenze relative più piccole e uguali ad x.

Come mostrato nella figura, il calcolo della frequenza relativa di molti valori continui e la disposizione su un grafico a barre può risultare difficile da consultare.
Considerando che potremmo avere anche grafici più complessi, è meglio ricorrere all'utilizzo della ECDF e un grafico di tipo linea.
Nella figura successiva si nota un grafico molto più semplice da consultare.





Utilizzando la distribuzione cumulativa si può notare quando una frequenza può essere presente in un grafico.
Contestualizzato all'esempio possiamo vedere come siano pochi i vini che hanno o poca o molta acidità volatile.

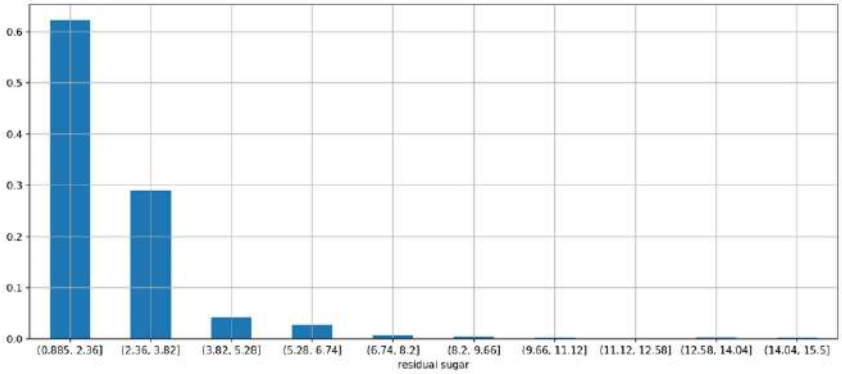
Nel nostro codice, per specificare che dobbiamo applicare una ECDF, si introduce **.cumsum()** e si pone dopo **.value_counts()**.

```
ecdf = wines['volatile acidity'].value_counts(normalize=True).sort_index().cumsum()
ecdf.plot(kind='line', figsize=(18,6))
plt.show()
```

Un altro modo per risolvere il problema che si crea quando abbiamo dei valori continui, è utilizzare l'istogramma. Attraverso l'istogramma, possiamo raggruppare osservazioni simili nella stessa classe. In pratica si definiscono degli intervalli chiamati **bin**, e si assegnerà ciascun valore al proprio bin di corrispondenza. Così non si conteranno le frequenze relative di ciascun valore, ma di interi intervalli.

Attraverso l'utilizzo della funzione .cut() di python, possiamo dividere in n intervalli tutti uguali, tanti quanti sono specificati nell'assegnazione a bins.

rs = wines['residual sugar']
rs = pandas.cut(rs, bins= 10)
#funzioni per mostrare il grafico
Contestualizzato al dataset dei vini, si può osservare come i vini con il residuo di zucchero più basso, siano in maggiore quantità rispetto a tutto il resto.
Si raggruppano vini con quantità simile di zucchero residuo, e si calcola la frequenza relativa.



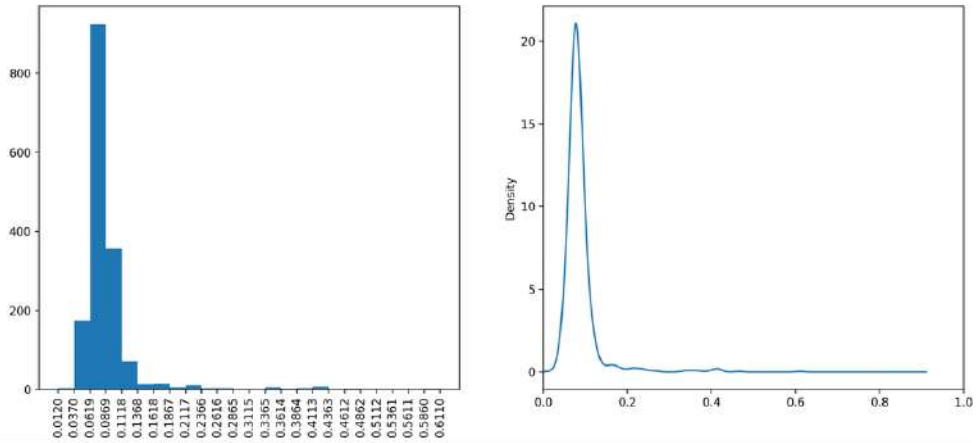
Con la funzione **.cut()**, stiamo solo dividendo gli intervalli in n bins, per questo motivo poi deve essere applicata la funzione **.value_counts()**. Nel caso in cui usiamo la funzione **.hist()**, si divide il grafico in n intervalli e il conteggio avviene automaticamente, quindi, non dobbiamo aver già applicato la funzione **.value_counts()**.

Il numero di intervalli può assumere un numero arbitrario, come nell'esempio precedente, oppure si può decidere attraverso criteri euristici. Questi criteri sono:

- **sturges**: il numero di intervalli si ottiene tramite **int(3.3*numpy.log(len(elementi)))**, che permette di ottenere un numero sufficiente di elementi per un intervallo, se la distribuzione è normale. Tende a sottostimare in presenza di dataset grandi.
- **rice**: il numero di intervalli si ottiene tramite **int(2*len(elementi)**(1/3))**, che permette di gestire dataset di grandi dimensioni. All'aumentare del numero degli elementi, aumentano anche gli intervalli.

Il problema principale degli istogrammi, è che tentano di categorizzare i dati continui utilizzando gli intervalli. La scelta del numero degli intervalli e della loro larghezza influenza altamente l'istogramma, infatti, cambiando i bin l'istogramma cambia.

La **stima della densità** cerca di risolvere questo problema, effettuando una stima di quanto è probabile trovare un valore in ogni punto dell'asse. Il risultato è una funzione continua e liscia, dove il risultato dipende solo dalla distribuzione all'interno del dataset.



La funzione si applica attraverso la seguente riga di codice:

dataset.plot.density()

Si applica come se stessimo applicando ai dati la funzione **.bar()**,**.hist()**, ecc. . Utilizzando il KDE o densità, possiamo vedere come ci sono alcuni artefatti, ovvero che la linea passa anche per valori non presenti nel dataset, questo perché la funzione è liscia e continua(non discreta come per gli istogrammi).

Solitamente i nostri dataset, contengono molte informazioni, ma non tutte sono essenziali. Infatti, solitamente si è interessati non alla singola misurazione prelevata, ma bensì ad una sintesi che intesi che interpreti i dati.

Attraverso le funzioni statistiche, possiamo riassumere in modo significativo ma coinciso ciò che il dataset contiene.

I vantaggi che si ottengono applicando funzioni statistiche sono:

- semplificazione -> riducono grandi volumi di dati a pochi valori significativi;
- descrizione -> forniscono una descrizione numerica della tendenza e della distribuzione;
- confronto -> si può effettuare un confronto semplice con altri dataset, sottolineando similarità e differenze;

Una naturale tendenza umana è fare confronti con la media. Nella quotidianità si usano i valori medi, come: il confronto con l'altezza media delle persone, il confronto con le temperature medie, ecc. .

Lo studio attraverso l'utilizzo dei valori medi è dipeso dalla natura dei dati e la sua scala e inoltre dall'obiettivo dello studio.

Le funzioni che descrivono la media o i valori centrali dei dati vengono dette **misure di tendenza centrale**.

La grandezza di un campione è data dal numero di valori che contiene. La dimensione potrebbe anche essere diversa da quella stimata, dato che sono presenti valori nulli(Not a Number).

La **media**, è possibile calcolarla attraverso il richiamo della funzione **.mean()**. Essa nel conteggio non include i dati nulli, ed è sensibile agli outliers e agli estremi, inoltre, anche i valori dei grafici che sono asimmetrici(non a campana) influenzano il valore della media.
È utile perché prende in considerazione tutti i punti dei grafici ed è una rappresentativa dei dati, affidabile nel caso che non esistano anomalie.

Si può calcolare anche la media per intervalli, attraverso la seguente formula:

$$\frac{1}{n}\sum_{i=0}^{n} m_i * f_i$$

Dove:

- n è il numero di intervalli;
- m è il valore medio dell'intervallo;
- f è la frequenza di elementi dell'intervallo che sono contenuti all'interno del dataset.

La mediana, è il valore che si trova nella posizione centrale, quando i dati sono ordinati in modo ascendente. Si divide il dataset in due metà: il 50% dei valori più piccoli della mediana e il 50% dei valori più grandi della mediana.

Nel caso in cui i dati siano pari si fa la media tra i due mediani. Nel linguaggio python per ottenere la mediana di una feature si fa attraverso la funzione **.median()**.

Il vantaggio nell'usare la mediana è che non è influenzata da outliers o estremi, bensì si mantiene sempre nella parte centrale. Essa utilizza la posizione dei dati e non i dati in sé, rendendola anche adatta per i dati ordinali.

I quartili, percentili, decili, ecc., dividono i dati ordinati in intervalli con uguale probabilità. Ci danno informazioni legate sulle soglie che separano una distribuzione. Ad esempio:

campione = [1,2,3,4,5,80,90,100,120,240]

quale è il valore del terzo quartile(75%)?

$P_{0,75}$= (n+1)*(3/4) = (10+1)*(3/4) = 8,25

$Q_{0,75}$ =valore(int(P))+decimale(P)*(valore(int(P+1))-valore(int (P))) = 100+0,25*(120-100) = 100+5 = 105

Stessa regola vale per i decili e percentili, con gli elementi minori uguali di $Q_{0,75}$ e quelli maggiori. Quindi divide il dataset con l'intervallo contenente valori minori al quantile di α e il restante 1- α maggiori.

Per ottenere il quantile di un campione, si utilizza la funzione **.quantile()**. Percentili e quartili, sono solo casi specifici dei quantili.

Inoltre, ci sono quantili specifici:

- il quantile (0) è riferito al minimo del campione;
- il quantile(0.5) è riferito alla mediana del campione;
- il quantile(1) è riferito al massimo presente nel campione.

Un'ulteriore misura di tendenza centrale è la **moda**. La moda, infatti, rappresenta il valore che compare più frequentemente nei dati. Si calcola la frequenza assoluta di ogni valore, è l'elemento che ha la frequenza più alta è considerato la moda di un dataset.

Per applicare la moda, si utilizza la funzione .mode(), e può essere applicata a qualsiasi tipo di dato. Restituisce valori univoci e non, e non è influenzata da outliers o estremi.

Solitamente, dopo che si è invocata la funzione, essa ritorna più elementi. Per questo motivo prendiamo il primo elemento della lista.

Misure di dispersione

La concentrazione o dispersione che ruota attorno ad un valore, è un'altra proprietà che caratterizza i dati e la loro distribuzione. Misure di tendenza centrale per determinati casi potrebbero non risultare utili, per analizzare i dati, ad esempio:

ipotizzando che si analizzino i ritardi agli incontri

	1 incontro	2 incontro	3 incontro	4 incontro	5 incontro
Santo	0	0	0	0	0
Giulia	0	-10	10	-10	10

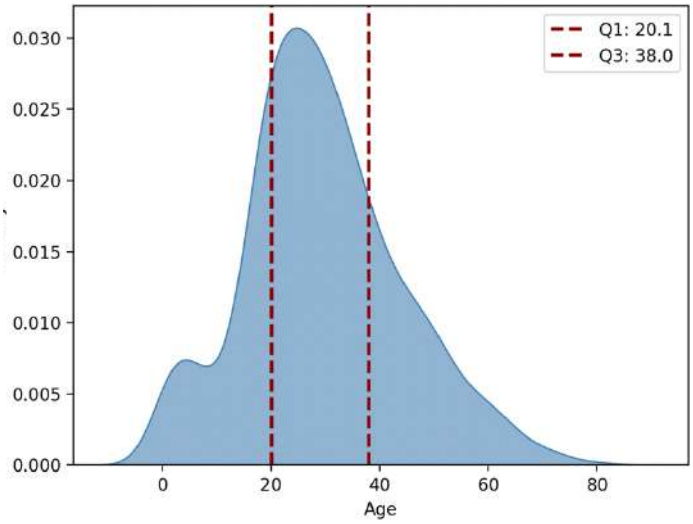
Possiamo notare che la media dei ritardi messa a confronto è la stessa, anche se Giulia ha fatto molto più ritardi di Santo.

La prima misura di dispersione, molto semplice da implementare e capire, è il calcolo del **range**. Il calcolo del range utilizza solo i due estremi e serve a capire la larghezza dei dati.

Una misura di dispersione più robusta è l'interquartile range, che non è sensibile agli outliers o estremi. Attraverso l'interquartile range rappresentiamo l'ampiezza centrale di u insieme di dati, calcolato attraverso la differenza tra il primo quartile e il terzo.

$$\mathbf{IQR = Q_3 - Q_1}$$

È utilizzato per trovare i valori anomali all'interno di un set di dati. Si identificano se non appartengono al range [Q_1-1.5*IQR, Q_3+1.5*IQR]. Si concentra sulla porzione centrale dei dati.



Una delle più importanti misure di dispersione, se non la più importante è la **varianza**. La varianza o **deviazione quadratica media**, fornisce una stima di quanto i valori si discostano dalla media. La formula è la seguente:

$$\sigma^2 = \frac{\sum_{i=1}^{n}(x_i - media)^2}{n} \text{(per una popolazione)}$$

$$s^2 = \frac{\sum_{i=1}^{n}(x_i - media)^2}{n-1} \text{(per un campione)}$$

Per un campione è diverso, poiché ha dimensione minore rispetto alla popolazione. Dato che la varianza campionaria tende a risultare più bassa, si sottrae 1 a n per ottenere un valore più alto e più rappresentativo della popolazione.

Una varianza bassa all'interno di un dataset, ci dice che quasi tutti i valori sono vicini alla media, viceversa se abbiamo una varianza alta.

Per calcolare la varianza in python, possiamo utilizzare la funzione **var()** nel seguente modo:

campione.**var()**

popolazione.**var(ddof=0)**

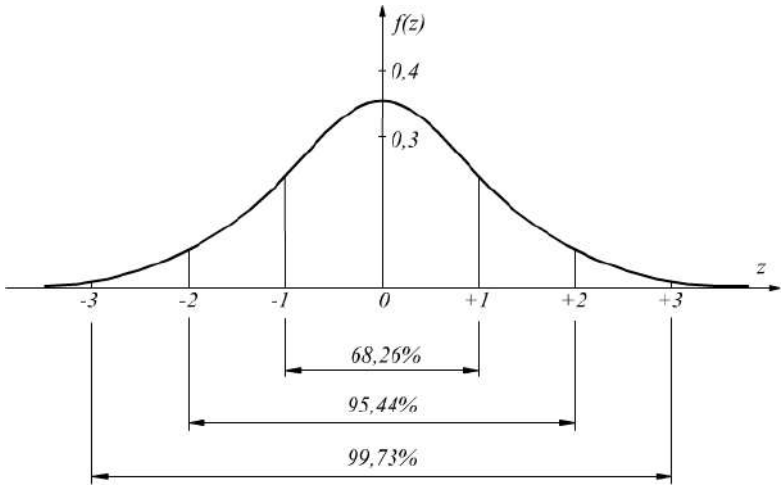
Tutti gli indici di partenza visti hanno la stessa unità di misura dei dati di partenza. Dato che quando calcoliamo la varianza si effettua il quadrato, allora l'unità di misura del risultato sarà uguale al quadrato dell'unità di misura di partenza. Per questo motivo possiamo applicare la radice quadrata, per ripristinare la stessa unità di misura.

L'applicazione della radice quadrata sulla varianza da vita alla **deviazione standard**.

$$\sigma = \sigma^2 = \sqrt{\frac{\sum_{i=1}^{n}(x_i - media)^2}{n}}$$

Per calcolare la deviazione standard si applica la funzione std() nel seguente modo:

campione.**std()**



In una distribuzione normale:

- il 68,3% dei dati ricade sulla prima deviazione standard;
- il 95% dei dati ricade sulla seconda deviazione standard;
- il 99% dei dati ricade sulla terza deviazione standard;

Normalizzazione dei dati

Molte volte, ci potremmo trovare nelle condizioni per cui due feature debbano essere valutati tra loro, e non necessariamente devono avere la stessa unità di misura.

Pertanto, esistono tecniche di normalizzazione dei dati, che permettono di confrontare anche due unità di misura diverse tra loro.

Si hanno diversi tipi di normalizzazione:

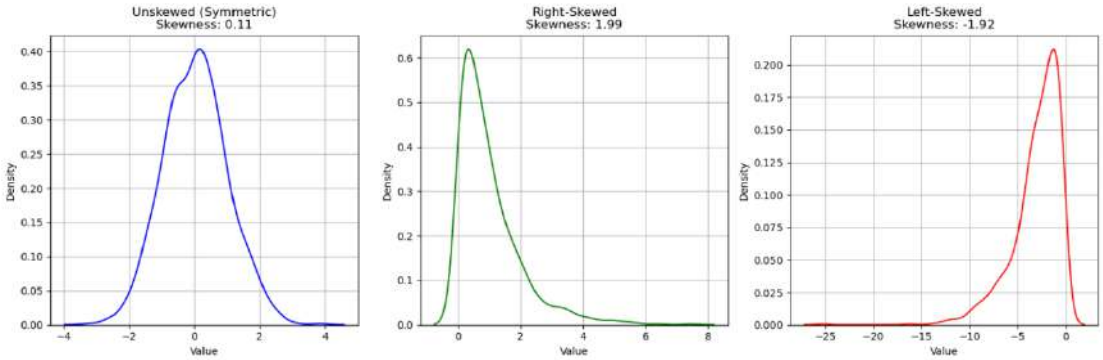
- **normalizzazione tra 0 e 1**:
  - $x_{normalizzato} = \frac{(x - x_{min})}{(x_{max} - x_{min})}$
- **normalizzazione tra -1 e 1**:
  - $x_{normalizzato} = \frac{(x_{max} + x_{min} - 2 * x)}{(x_{max} - x_{min})}$ per valori grandi si va a -1, per valori piccoli si tende a 1;
- **standardizzazione(z-scoring)**:
  - $z_i = \frac{x_i - media}{\sigma}$ in molti casi si vuole confrontare dati che hanno media sottratta e deviazione standard sia pari ad 1, e questo si ottiene dividendola.

Ulteriori funzioni statistiche descrittive

Lo **skewness**, o asimmetria, è una misura che indica quanto una distribuzione dei dati sia sbilanciata rispetto alla media.

- **Valori positivi** indicano che la coda della distribuzione è più lunga a destra, cioè che **pochi valori molto grandi** tendono a "tirare" la media verso destra.
- **Valori negativi** indicano che la coda è più lunga a sinistra, cioè **pochi valori molto piccoli** tirano la media verso sinistra.
- Uno skewness **vicino a zero** indica che la **distribuzione è simmetrica**.

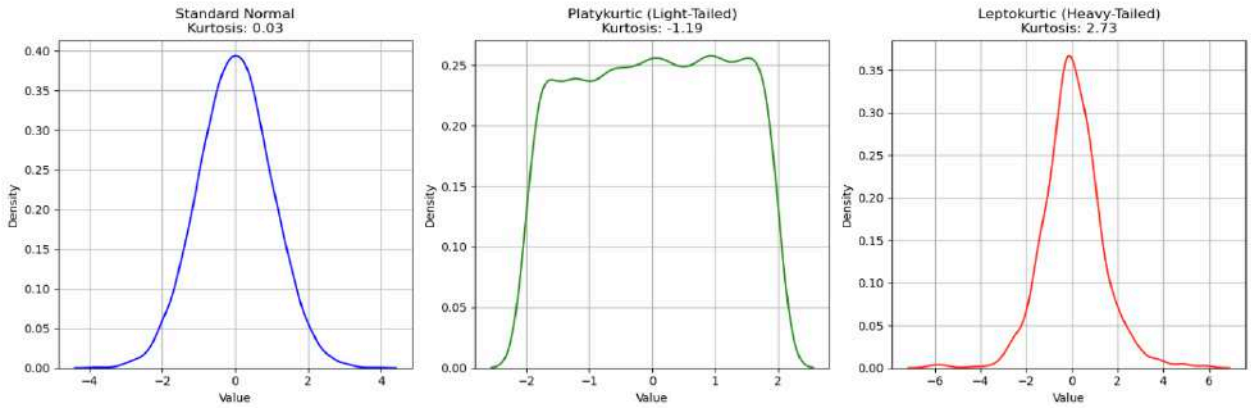
La skewness può essere calcolata con la formula basata sui momenti centrali dei dati, oppure tramite funzioni già pronte in Python come **data.skew()**.



La **kurtosis**, o curtosi, misura invece la "appiattimento" o la "piccolezza" della distribuzione rispetto a una distribuzione normale.

- **Valori positivi** indicano distribuzioni **leptocurtiche**, con grafici più appuntiti e stretti;
- **Valori negativi** indicano distribuzioni **platycurtiche**, con grafici più larghi;
- **Valori vicini a zero** indicano distribuzioni **mesocurtiche**, più simili alle distribuzioni normali.

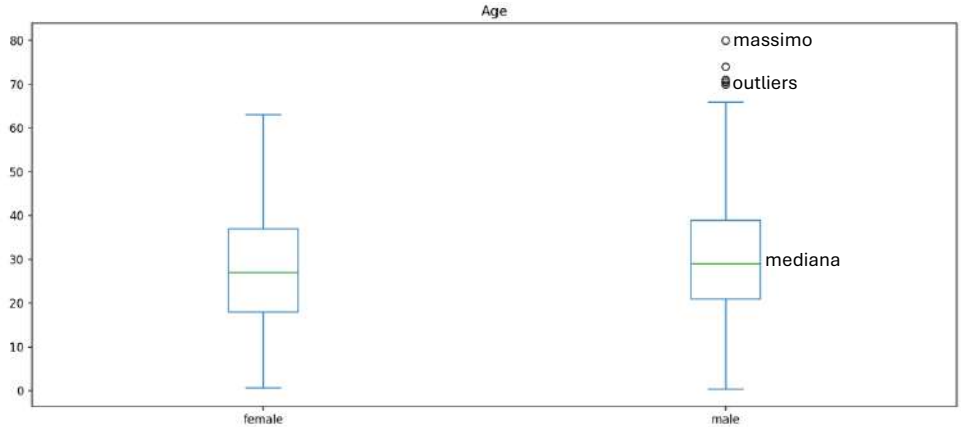
Uno skewness e una kurtosis insieme forniscono informazioni più complete sulla forma della distribuzione, andando oltre le misure di tendenza centrale e dispersione. In Python è possibile calcolare la curtosi con **data.kurtosis()** e ottenere un'indicazione di quanto la distribuzione si discosti dalla forma normale.



I **boxplot** sono uno strumento compatto per visualizzare alcune caratteristiche descrittive dei dati. Permettono di rappresentare in modo chiaro la mediana, il primo e il terzo quartile e, a seconda della versione del boxplot, il valore minimo e massimo.

La **parte centrale** del **boxplot** è il **rettangolo ("box")** che mostra l'intervallo interquartile, cioè la distanza tra il primo e il terzo quartile, mentre la linea all'interno indica la mediana.

I **"baffi"** del boxplot estendono la visualizzazione fino ai valori estremi o fino a un certo limite, evidenziando eventuali outlier. Questa rappresentazione è molto utile perché consente di confrontare rapidamente la distribuzione, la simmetria e la presenza di valori anomali in un dataset.



Come si può vedere nel grafico, solo i maschi hanno un'età che si discosta dal boxplot

In python abbiamo due alternative per realizzare il grafico del boxplot:

- **popolazione(campione).boxplot(column="*colonna osservata*", by="*eventuale group by*")**
- **feature.plot.box()**

Probabilità per l'analisi dei dati

Quando si lavora con i dati, spesso ci si chiede quali valori essi assumeranno. Questo dipende fortemente dal sistema di appartenenza.

In un ambiente deterministico, possiamo calcolare i risultati partendo da un insieme di condizioni iniziali. Ad esempio, possiamo sapere in anticipo quale sarà il risultato di 2^4, perché conosciamo le condizioni iniziali e il risultato sarà sempre prevedibile e identico. Le condizioni iniziali, in questo caso, potrebbero essere le seguenti:

- la potenza è definita come il prodotto della base con sé stessa tante volte quanto indicato dall'esponente;
- $2 \times 2 \times 2 \times 2 = 16$→ il risultato sarà sempre lo stesso.

Ma, in scenari reali, ci troviamo davanti un'incertezza. Quest'incertezza può dipendere da diverse fonti:

- **Casualità intrinseca**: alcuni eventi sono dovuti al caso -> lancio del dato o il pescaggio di una carta;
- **Osservazione incompleta**: non si conoscono tutte le informazioni rilevanti -> risultato appare incerto;
- **Modellazione incompleta**: anche se un sistema ha leggi note e fisse, potrebbe non avere gli strumenti o le informazioni necessarie per dare un risultato deterministico. Ad esempio, se volessimo stimare il traffico stradale, avremmo a disposizione delle leggi che guidano i comportamenti degli autisti(non contando i fuorilegge) e la segnaletica stradale. Quello che manca riguarda le informazioni, dato che non potremmo sapere a priori i movimenti degli autisti e la quantità di traffico.

La teoria della probabilità fornisce un ragionamento sull'incertezza. Ci permette di:

- **quantificare eventi incerti**: la probabilità ci concede di assegnare un valore compreso tra 0 e 1 a ciascun evento, indicando quanto sia probabile che un evento accada. Si passa da giudizi qualitativi a giudizi quantitativi.
- **combinare informazioni note e ignote**: spesso abbiamo informazioni parziali sui fenomeni, dati storici o conoscenze teoriche. Possiamo combinare queste informazioni, con informazioni non conosciamo attraverso le regole matematiche, permettendoci di aggiornare le nostre convinzioni.
- **derivare nuove affermazioni probabilistiche utilizzando regole formali**: attraverso strumenti quali somma, prodotto o regola di Bayes possiamo calcolare probabilità complesse a partire da dati semplici. Così facendo possiamo rispondere a delle domande in modo preciso e riproducibile.
- **interpretare e analizzare i dati**: la probabilità ci permette di riconoscere pattern significativi, distinguendo segnali reali da rumori. Inoltre, consente di fare previsioni affidabili anche in condizioni di incertezza, e sulla base di queste previsioni si possono intraprendere delle decisioni.

Quando l'acquisizione dei dati è influenzata dall'incertezza, si ha un esperimento casuale. Un esperimento casuale è un esperimento che può essere ripetuto molte volte avendo risultati potenzialmente diversi.

La terminologia utilizzata quando si parla di esperimento casuale è:

- **spazio campionario Ω(evento sicuro):** l'insieme di tutti i possibili risultati di un esperimento casuale. Quindi ogni volta che viene ripetuto l'esperimento il risultato sarà uguale a s, con s € Ω.
- **evento semplice ω_i:** un risultato appartenente allo spazio campionario.
- **evento A ⊆ Ω:** un evento può essere anche un sottoinsieme dello spazio campionario, ovvero può avere uno o più risultati. Ad esempio nel lancio dei dati:
    - A = {5}
    - A = {2,4,6} numeri pari
    - A = {1,3,5} numeri dispari
    - A = {2,3,5} numeri primi
    - A = {1,4,6} numeri non primi
    - A = {1,2,3} primi 3 numeri

Una variabile casuale è una variabile il cui valore dipende dal risultato di fenomeni aleatori. Non possiamo sapere il valore in anticipo, ma possiamo calcolare tutti i possibili risultati e la loro probabilità.

- Variabili statistiche -> variabili in statistica che descrivono già dati osservati;
- Variabili casuali -> descrivono fenomeni incerti prima dell'osservazione

Formalmente una variabile casuale è una funzione che mappa ogni risultato dello spazio campionario in un valore in uno spazio misurabile E.

$$X: \Omega \longrightarrow E$$

Con X la funzione che associa ogni valore di Ω ad un valore E = ℝ.

Ad esempio nel lancio della moneta:

- Lancio(testa) ⟶1
- Lancio(croce) ⟶0

Una variabile casuale si indica con le lettere maiuscole.

Similmente alle variabili statistiche, le variabili casuali possono essere:

	Discrete	Continue
Scalari	Lancio Moneta	Altezza della persona
Multidimensionali	Coppia di dai	Peso e altezza

Per esprimere formalmente le regole sui valori che la variabile casuale assumerà, utilizzeremo le seguenti notazioni e assiomi.

Per assegnare un valore di probabilità a un dato esito, possiamo definirla mediante la P:

P(classe = 1) = x, con x € [0,1]

P(1) = P(classe = 1)

Gli assiomi di Kolmogrov e i corollari sono:

ASSIOMI	
Non Negatività	0 <= P(A) <= 1 $\forall A \subseteq \Omega$
Certezza	P(Ω) = 1
Additività per eventi disgiunti	se $A \cap B = \emptyset$ allora P(A U B) = P(A)+P(B)

COROLLARI	
Regola della complementazione	P($\bar{A}$) = 1-P(A)
Evento impossibile	P(∅) = 0
Formula generale additività	P(A U B) = P(A)+P(B) – P($A \cap B$)
Monotonicità	A ⊆ B allora P(A) <= P(B)

Quando tutti gli eventi sono equi-probabili, si può parlare di probabilità di Laplace.

La probabilità si calcola:

$$P(A) = \frac{|A|}{|\Omega|}$$

Dove: $|A|$è il numero di casi favorevoli e $|\Omega|$ è il numero di casi totali.

Se non si ha la certezza che degli eventi sono equiprobabili, possiamo stimare la probabilità dalle osservazioni. Si possono seguire due approcci:

- **Approccio frequentista**: la probabilità di un evento non è qualcosa di astratto, ma si stima osservando quante volte accade quell'evento quando l'esperimento viene ripetuto molte volte. La formula è la seguente: P(X = x) = $\frac{n\ volte\ X=x}{prove\ totali}$
  Per la legge dei grandi numeri più è ripetuto un esperimento, più la probabilità si avvicina alla probabilità teorica.
- **Approccio bayesiano**: diverso dall'approccio frequentista, misuriamo la probabilità come un'incertezza o di convinzione. Quindi, si hanno delle conoscenze pregresse e a seguito dell'acquisizione di nuove informazioni, si aggiornano le conoscenze. Le conoscenze si aggiornano attraverso la formula di Bayes descritta a seguire.

Probabilità congiunta

La probabilità congiunta è la probabilità che due eventi accadano contemporaneamente. È definita così:

P(X,Y) oppure **P($X \cap Y$)**

P(X,Y) = P(Y,X) è simmetrica

La probabilità congiunta è estesa ad un numero arbitrario di variabili P(X_1, X_2, … ,X_{n-1},X_n).

Senza l'utilizzo di una formula per la probabilità congiunta possiamo utilizzare un approccio frequentista, ovvero ripetiamo l'esperimento più volte e vediamo quando si verificano insieme le condizioni.

Si può creare una tabella che tenga traccia di quante volte si è osservata una data combinazione. La tabella in questione si chiama **tabella di contingenza**.

Prendendo in questione un dataset simile a quello del titanic. Qual è la probabilità di trovare una persona di sesso maschile all'interno della prima classe?

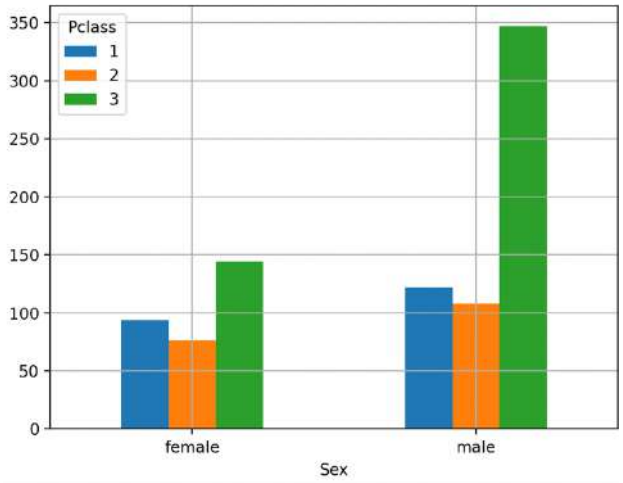
	Prima classe	Seconda Classe	Terza Classe	All
Male	3	5	7	15
Female	3	3	4	10
All	6	8	11	25

Dalla tabella di contingenza possiamo passare alla **tabella della probabilità congiunta**, dividendo tutti gli elementi della tabella per il numero di osservazioni totali.

	Prima classe	Seconda Classe	Terza Classe	All
Male	3/25 = 0,12	5/25 = 0,20	7/25 = 0,28	15/25 = 0,6
Female	3/25 = 0,12	3/25 = 0,12	4/25 = 0,16	10/25 = 0,40
All	6/25 = 0,24	8/25 = 0,32	11/25 = 0,44	25/25 = 1

In python possiamo creare la tabella di contingenza attraverso la funzione **.crosstab(column,row)**, inoltre, possiamo creare il grafico relativa ai valori della tabella di contingenza.

```
from matplotlib import pyplot as plt
pd.crosstab(titanic['Sex'],titanic['Pclass']).plot.bar()
plt.grid()
plt.xticks(rotation=0)
plt.show()
```



Possiamo avere la probabilità di contingenza e non solo la tabella con i valori di frequenza attraverso normalize=True. In questo modo dividiamo ogni elemento per il totale delle osservazioni.

La probabilità marginale è la probabilità di un singolo evento ignorando l'altro con cui si è calcolata la probabilità congiunta. Si effettua sommando lungo la dimensione che si vuole ignorare.

Marginalizzazione

Pclass	1	2	3
Sex			
female	0.105499	0.085297	0.161616
male	0.136925	0.121212	0.389450

Se volessimo calcolare la probabilità marginale del sesso quando assume il valore maschile, basta scorrere le colonne che vedono i maschi nelle diversi classi e sommare le probabilità. Se volessi calcolare la probabilità marginale della prima classe basta scorrere le righe.

Molte volte si è interessati alla probabilità di un evento, dato che se n'è verificato un altro:

P(X=x| Y=y) probabilità di X che assuma x, se Y ha assunto y

La probabilità condizionata si calcola attraverso la seguente formula:

$$P(X|Y) = \frac{P(X,Y)}{P(Y)} o \frac{P(X \cap Y)}{P(Y)} (stessa\ cosa)$$

Da questa formula possiamo anche derivare la formula della probabilità congiunta:

$$P(X \cap Y) = P(X|Y) * P(Y) = P(Y|X) * P(X)$$

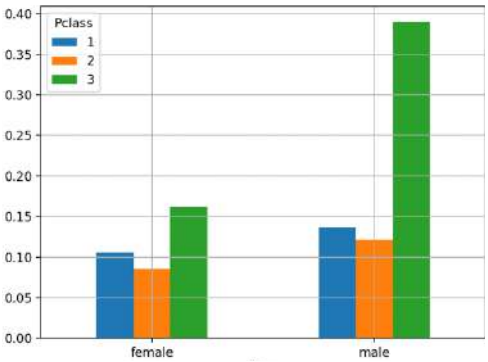
regola del prodotto probabilità congiunta = probabilità condizionata*probabilità marginale(di una solo componente)

In Python possiamo calcolare la probabilità condizionale con la funzione crosstab(). Quando abbiamo la probabilità congiunta, dividiamo i valori per il totale. In questo caso, per ottenere la probabilità condizionata, dividiamo la probabilità congiunta per la probabilità marginale della variabile che funge da condizionante. Lo stesso concetto si applica alla crosstab: se vogliamo le probabilità delle righe condizionate dalle colonne, dividiamo per le colonne, e viceversa.

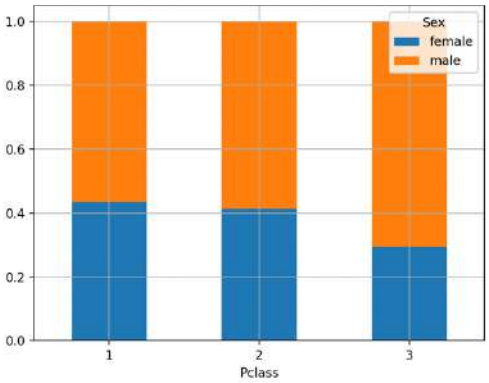
```
contingencytable = pd.crosstab(titanic['Sex'],titanic['Pclass'], normalize='columns')
print(contingencytable)
```

```
contingencytable = pd.crosstab(titanic['Sex'],titanic['Pclass'], normalize='index')
print(contingencytable)
```

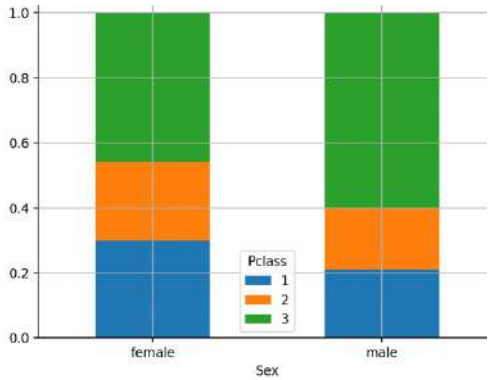
```
Pclass         1         2         3
Sex
female  0.435185  0.413043  0.293279
male    0.564815  0.586957  0.706721
Pclass         1         2         3
Sex
female  0.299363  0.242038  0.458599
male    0.211438  0.187175  0.601386
```



P(Sex⋂Pclass)



P(Pclass|Sex)



P(Sex|PClass)

Nella seconda figura per vedere effettivamente la distribuzione del sesso tra le varie classi, possiamo fare la trasposta della matrice come segue:

```
pd.crosstab(titanic['Sex'],titanic['Pclass'],normalize='columns').T.plot.bar(stacked=True)
```

Attraverso **.T** possiamo effettuare la trasposta.

Ovviamente la probabilità congiuntiva non è limitata a due sole feature, ma può esse estesa a più variabili, e la formula adoperata che utilizza il prodotto tra probabilità condizionata e probabilità marginale si può riscrivere come:

$$\boldsymbol{P(X_1, \dots, X_n) = P(X_1) * \prod_{i=2}^{n} P(X_i | X_1, \dots, X_{n-1})}$$ **(regola della catena(chain rule))**

dove $\prod_{i=2}^{n}$ ruolo analogo alla sommatoria, ma l'operazione è il prodotto

Caso base n=2

$P(X_1,X_i) = P(X_1)*P(X_i|X_1,X_{n-1}) = P(X_1,X_2) = P(X_1)*P(X_2|X_1,X_1) = P(X_1,X_2) = P(X_1)*P(X_2|X_1)$

Teorema di Bayes

Il teorema di Bayes aggiorna la probabilità di un evento sulla base di una nuova evidenza, questo è un concetto chiave in inferenza statistica e intelligenza artificiale.

La formula è la seguente:

$$\boldsymbol{P(A|B) = \frac{P(B|A) * P(A)}{P(B)}}$$

Un esempio molto semplice per capire la formula è il seguente:

P(B)(evidenza) = cielo coperto dalle nuvole = 40/100

P(A)(precedente) = sta per piovere = 30/100

P(B|A)(probabilità) = probabilità del cielo coperto di nuvole se sta per piovere = 50/100

P(A|B)(posteriore) = probabilità che piova se il cielo è coperto di nuvole = ?

P(A|B) = $\frac{\frac{15}{100} * \frac{30}{100}}{\frac{40}{100}} = \frac{50}{100} * \frac{30}{100} * \frac{100}{40} = \frac{50*30}{100*40} = \frac{1500}{4000} (diviso\ 500) = \frac{3}{8} = 0{,}375 = \frac{37{,}5}{100}$

Verifichiamo se è vero calcolando P(B|A) = $\frac{\frac{37{,}5}{100} * \frac{40}{100}}{\frac{30}{100}} = \frac{37{,}5}{100} * \frac{40}{100} * \frac{100}{30} = \frac{37{,}5*40}{100*30} = \frac{1500}{3000} = 0{,}5 = \frac{50}{100}$

verificato

Indipendenza di due variabili

Due variabili sono indipendenti se l'esito di una non influenza minimamente l'esito dell'altra. Formalmente esse si dicono indipendenti se:

$$\boldsymbol{P(A, B) = P(A) * P(B)}$$

altrimenti la probabilità congiunta si calcolerebbe attraverso la formula classica:

$$\boldsymbol{P(A, B) = P(A|B) * P(B)}$$

Da ciò possiamo derivare che la formula della probabilità condizionata per variabili indipendenti è la seguente:

$$\boldsymbol{P(A|B) = \frac{P(A, B)}{P(B)} = \frac{P(A) * P(B)}{P(B)} = P(A)}$$

Associazione tra due variabili

Precedentemente abbiamo analizzato una singola variabile(campione univariato), spesso però accade che le osservazioni in analisi siano composte da più variabili.

Si devono quindi utilizzare metodi per analizzare le osservazioni in modo multivariato, al fine di scoprire i diversi fenomeni che intercorrono tra i dati.

Due variabili sono associate se il valore di una variabile ci dice qualcosa sull'altra. Come nella statistica due variabili sono indipendenti se il valore di una non influenza l'altra.

La correlazione non implica direttamente causalità, infatti, due variabili possono essere staticamente associate senza esisti una causa diretta.

È utile conoscere però il comportamento delle variabili tra loro, in modo che se esiste un pattern: si analizza e infine si utilizza per scopi specifici.

Ci sono diverse misure che ci permettono di capire quante due variabili sono associate.

Una tra queste misure è la distribuzione di frequenza condizionale. Prendendo in considerazione il database del titanic, se volessimo sapere qual è la probabilità di sopravvivenza dei passegeri per ogni classe.

```
survivedforclass = pd.crosstab(titanic['Pclass'],titanic['Survived'],normalize=0)
```

```
Pclass
1          0.370370  0.629630
2          0.527174  0.472826
3          0.757637  0.242363
```

Già dalla tabella di contingenza possiamo verificare se due variabili sono effettivamente indipendenti o meno.
Se sostituiamo i valori presenti nella matrice con quelli calcolati tramite la formula della probabilità congiunta per variabili indipendenti, possiamo confrontarli:
se i valori coincidono con quelli osservati nella tabella originale, allora le variabili sono indipendenti; al contrario, se differiscono, significa che esiste una forma di associazione o correlazione, anche se minima.

Calcoliamo la frequenza attesa per dire con certezza se due eventi sono indipendenti, partendo dalla formula della probabilità:

$$(\boldsymbol{P(S,C)})\frac{\boldsymbol{n_{ij}}}{\boldsymbol{n}} = \frac{\boldsymbol{n_{i+}} * \boldsymbol{n_{+j}}}{\boldsymbol{n^2}}$$

$$\boldsymbol{n_{ij}} = \boldsymbol{n} * \frac{\boldsymbol{n_{i+}} * \boldsymbol{n_{+j}}}{\boldsymbol{n^2}} = \frac{\boldsymbol{n_{i+}} * \boldsymbol{n_{+j}}}{\boldsymbol{n}}$$

In questa formula calcoliamo quindi la frequenza attesa di un evento congiunto attraverso: prodotto della somma degli elementi che sono lungo la riga i per la somma degli elementi che sono lungo la colonna j, il tutto rapportato a n numero di elementi totali.

```
Frequenze osservate
 Survived    0    1
Pclass
1           80  136
2           97   87
3          372  119
```

In questo esempio, possiamo verificare se due eventi sono indipendenti utilizzando la formula proposta:
n_{11}=(80+136)*(80+97+372)/891= 133,09
n_{12}=(80+136)*(136+87+119)/891 = 82,91
n_{21}=(97+87)* (80+97+372)/891 = 113,37
n_{22}=(97+87)*(136+87+119)/891 = 70,62
n_{31} =(372+119)* (80+97+372)/891= 302,53
n_{32} = (372+119)*(136+87+119)/891 = 188,46

Quindi, se gli eventi fossero realmente indipendenti allora la matrice di contingenza sarebbe la seguente:

$$\begin{matrix} 133{,}09 & 82{,}91 \\ 113{,}37 & 70{,}62 \\ 302{,}53 & 188{,}46 \end{matrix}$$

Attraverso questa matrice, possiamo calcolare la **statistica $\mathbf{X^2}$(chi-quadro) di Pearson**, una misura che calcola quanto i valori osservati si discostano da quelli attesi in caso di indipendenza tra variabili.

La formula è la seguente:

$$\mathbf{X^2} = \sum_{i=1}^{k} \sum_{j=1}^{l} \frac{\left(n_{ij} - \widetilde{n_{ij}}\right)^2}{\widetilde{n_{ij}}}$$

Dove:

$\mathbf{n_{ij}}$ è la frequenza osservata della tabella di contingenza;

$\widetilde{n_{ij}}$ è la frequenza attesa in caso di indipendenza;

In base al valore che ci tornerà la statistica, potremmo dire che:

- Se il valore è molto vicino a 0, allora come avevamo previsto le variabili sono indipendenti;
- Se il valore è molto alto, allora c'è una qualche associazione positiva/negativa.

Il valore è quindi compreso nell'intervallo [0,+∞[, ed è positivo perché la differenza è quadratica.

Per calcolare la statistica chi-quadro su python si utilizza il seguente codice:

```
from scipy.stats import chi2_contingency
contingency = pd.crosstab(titanic['Pclass'], titanic['Survived'])
print(chi2_contingency(contingency))
```

chi2_contigengy ritorna la matrice di contingenza con i valori attesi se le variabili fossero indipendenti. Inoltre, ritorna la statistica del chi-quadro, il **p-value** e i **gradi di libertà(dof)**.

Quando lavoriamo con il chi-quadro, dobbiamo tenere a mente che:

- i dati con cui lavoriamo devono essere categoriali, ovvero le variabili assumono un numero finito di categorie;
- ogni cella della tabella di contingenza deve contenere almeno 5 osservazioni attese. Se alcune celle hanno frequenze attese troppo basse, ad esempio minori di 1, la stima diventa non affidabile e il test chi-quadro potrebbe fornire risultati errati.

La statistica X^2 di Pearson, non è normalizzata, per questo motivo si utilizza **la statistica V di Cramer**, che normalizza i valori da 0 a 1.

La formula è la seguente:

$$V = \sqrt{\frac{X^2}{n \cdot (min(k,l) - 1)}}$$

Attraverso il risultato possiamo capire il tipo di associazione:

- se V assume il **valore 0** allora le feature sono **indipendenti**;
- se V assume il **valore 0,33** allora c'è una **debole associazione** tra le variabili;
- se V assume il **valore 1** allora c'è una **perfetta associazione** tra le variabili.

Per calcolare la statistica V in python si fa attraverso scipy.stats.contingency e il codice è il seguente:

```
contingency = pd.crosstab(titanic['Pclass'], titanic['Survived'])
from scipy.stats.contingency import association
print(f"Level of association : {association(contingency):0.2f}")
```

In questo caso la funzione tornerà solo un valore compreso tra 0 e 1, che corrisponde al valore di associazione.

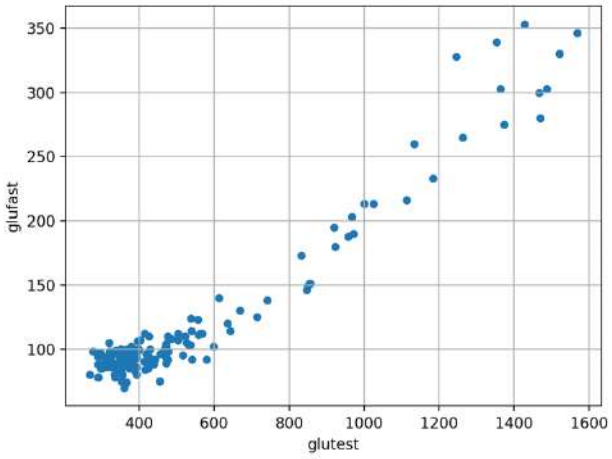
Quanto le variabili osservate sono continue e non discrete, non potremmo più utilizzare la matrice di contingenza. Prendiamo in esempio un dataset standard utilizzato a scopi didattici, che proviene dalla libreria **statsmodels.datasets**.

Il dataset in questione è un dataset esempio che si userebbe per analizzare i casi di diabete.

```
from statsmodels.datasets import get_rdataset
data = get_rdataset('Diabetes','heplots').data
```

Per dati continui possiamo utilizzare lo **scatter plot(grafico a dispersione)**, che non ha bisogno di variabili categoriali, ma che **mostra direttamente la relazione bivariata di ogni osservazione come un punto nello spazio XY**. Permette di visualizzare graficamente:

- correlazioni positive o negative;
- pattern lineari o non lineari;
- outlier;
- cluster ovvero gruppi sparsi nel grafico.



Prendendo in considerazione glutest e glufast, possiamo vedere che:

- seguono un pattern regolare;
- c'è una correlazione positiva tra le due variabili

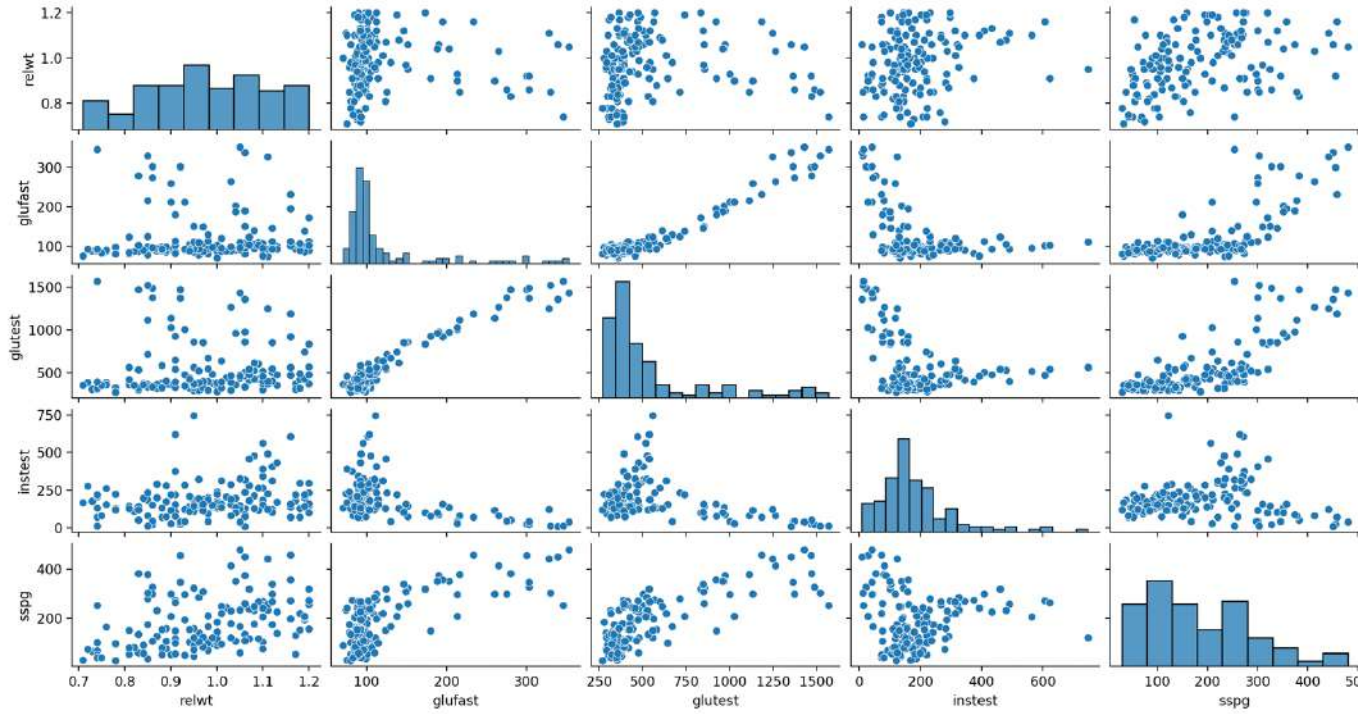
Come per i grafici visti precedentemente, il codice è il seguente:

```
data.plot.scatter(x='glutest',y='glufast')
plt.grid()
plt.show()
```

Per mostrare più scatter plot contemporaneamente possiamo utilizzare la **scatter matrix** o **pair plot**, che permette di visualizzare tutti gli scatter plot delle feature suggerite. L'utilizzo ci permette di vedere nella metà inferiore o superiore(simmetrica) lo scatter plot associato alle feature i e j. Inoltre, lungo la diagonale principale sono presenti gli istogrammi associati alla singola feature, che mostra come sono distribuiti i dati.

Per creare la scatter matrix si utilizza la libreria seaborn, come segue:

```
import seaborn as sbn
sbn.pairplot(data, height=1.5, aspect=2)
plt.show()
```



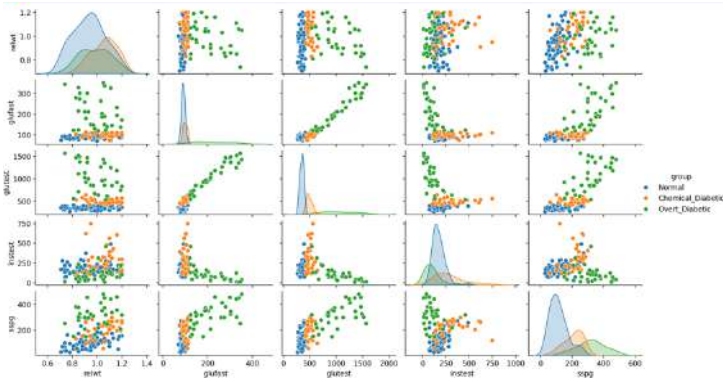
Si può effettuare un'analisi rapida e capire ad esempio che:

- glufast e glutest sono in correlazione;
- glufast o glutest sono in una possibile correlazione con relwt, anche se l'associazione è debole;
- relwt e sspg non sono in associazione.

Notiamo che pairplot() ha scartato la variabile group, perché? La variabile group è categoriale e negli scatter plot vanno solo variabile numeriche. Possiamo però utilizzare la variabile group per rendere il grafico più leggibile, nel seguente modo:

```
sbn.pairplot(data, height=1.5,
aspect=2, hue='group')
plt.show()
```

In questo modo, i punti hanno dei colori diversi a seconda della categoria di appartenenza, che varia a seconda del valore di 'group'.



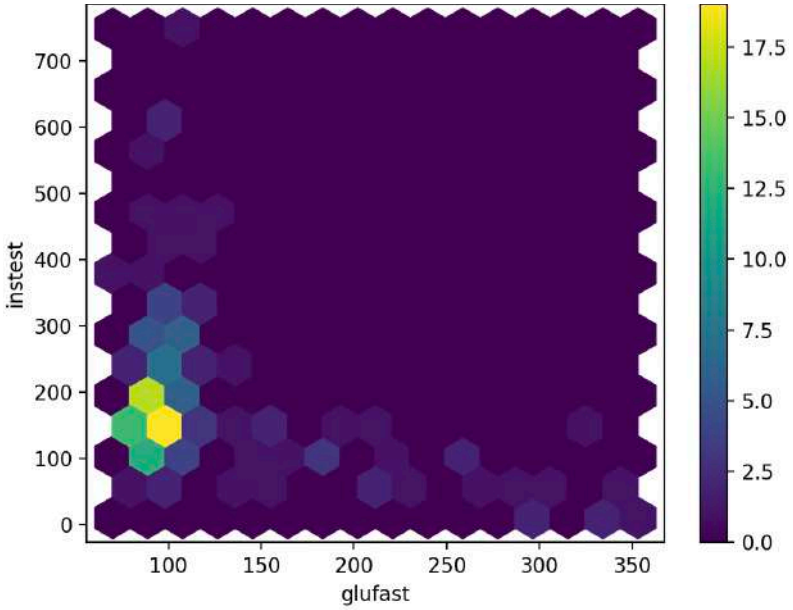
Un problema che è presente in grafici pairplot, si ha quando i punti si sovrappongono. Se abbiamo diversi cluster di punti, non possiamo stimare la densità, perché i punti diventano un ammasso nero(o colorato nel caso che viene fatta una colorazione per categoria), per questo motivo si utilizzano gli **hexbin plot 2 D**.

Gli hexbin plot 2D divide lo spazio XY in celle esagonali, dove ogni cella viene colorata in base al numero di osservazioni simili che raggruppa. In questo modo si vede bene dove ci sono più dati concentrati.

Si utilizzano gli esagoni perché rispetto a:

- i quadrati hanno solo 4 vicini alla stessa distanza, invece gli esagoni ne hanno 6 e inoltre le linee diagonali dei quadrati sono visivamente distrattivi;
- i cerchi non riescono a tassellare l'intero spazio senza sovrapporsi o lasciare buchi; invece, gli esagoni riescono a ricoprire tutto lo spazio.

Vediamo come per gli altri grafici, che per creare il grafico hexbin utilizziamo **.plot.hexbin()**.



Proprio come nell'analisi di una singola variabile usiamo il KDE per ottenere una curva continua della distribuzione, possiamo applicare lo stesso principio a due variabili per ottenere una distribuzione bivariata continua.

Il **grafico di densità 2D** rappresenta le regioni con maggiore o minore concentrazione di punti tramite un gradiente di colore, e le curve di livello collegano punti che hanno uguale densità, come in una mappa topografica.

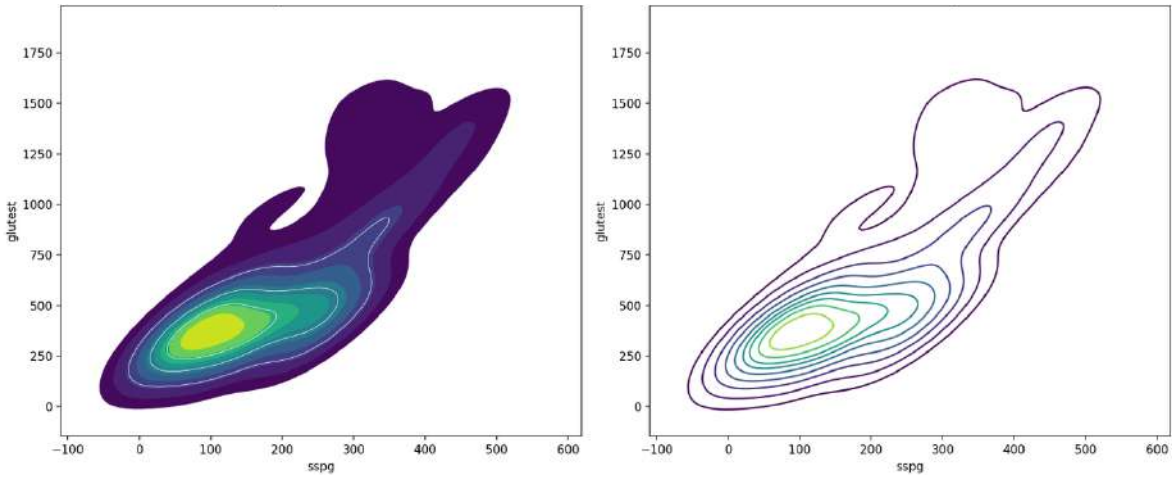
Rispetto all'hexbin, non si basa su una griglia discreta di celle, ma su una stima continua della densità, offrendo una rappresentazione più fluida e precisa della distribuzione dei dati.

La funzione per riprodurre il grafico di densità 2D è contenuta all'interno della libreria seaborn e si utilizza in questo modo:

```
sbn.kdeplot(data=data,x='sspg',y='glutest', fill='true',cmap='viridis', ax=axes[0])
```

Attraverso fill mostriamo i vari livelli di intensità, colorando attraverso i colori specificati in cmap.

Questo tipo di grafico presenta diversi vantaggi, come la rappresentazione continua della distribuzione dei dati. Tuttavia, in presenza di valori anomali (outlier), può introdurre alcune distorsioni: gli outlier tendono infatti ad “allungare” le code della distribuzione, facendo apparire come presenti delle frequenze anche in regioni dove in realtà non esistono osservazioni.



Covarianza

Per le distribuzioni univariate, si è analizzata la varianza ovvero il livello di dispersione delle osservazioni attorno alla propria media. Si può utilizzare una metrica simile per due variabili, dove se sono correlate, sono distribuite in modo simile.

Il concetto di varianza è generalizzato dalla covarianza nel caso di dati bivariati. La covarianza studia quanto le variabili variano insieme.

La formula è la seguente:

$$\boldsymbol{Cov(X,Y) = \frac{1}{N} * \sum_{i}^{n} (x^{(i)} - \overline{x}) * (y^{(i)} - y)}$$

Dove:

- N è il numero totale di osservazioni;
- $x^{(i)}, y^{(i)}$sono i valori dell’i-esima osservazione di X e Y
- $\overline{x}, \overline{y}$ sono le relative medie di X, Y

Se volessimo applicare la formula della covarianza ad un solo campione, allora dovremmo utilizzare la seguente formula:

$$\boldsymbol{Cov(X,Y) = \frac{1}{N-1} * \sum_{i}^{n} (x^{(i)} - \overline{x}) * (y^{(i)} - y)}$$

Così riduce i bias all’interno della stima, dato che non lavora con tutta la popolazione.

A seconda del valore della covarianza, possiamo dire con certezza se due feature sono associate, antiassociate oppure indipendenti.

Il segno è dato dalle somme del prodotto: $(x^{(i)} - \overline{x}) * (y^{(i)} - y)$

La concordanza, discordanza si verifica attraverso il seguente modo:

- Se $x^{(i)} e\ y^{(i)}$ hanno i segni concordi, ovvero si trovano ambedue o sopra o sotto le relative medie allora il prodotto sarà positivo, indicando la correlazione;
- Se $x^{(i)} e\ y^{(i)}$hanno i segni discordi, allora il prodotto sarà negativo, indicando un'anti correlazione;
- Se $x^{(i)} e\ y^{(i)}$hanno segni vicino allo zero, allora saranno influenti nel calcolo della covarianza.

Quindi i valori del primo e terzo quadrante contribuiscono positivamente, invece, i valori del secondo e quarto quadrante contribuiscono negativamente. I valori vicino all'origine non contribuiscono.

Possiamo anche calcolare la matrice delle covarianze, ovvero la matrice che contiene il valore della covarianza di ogni singola coppia di feature all'interno del dataset.

La formula è la seguente:

$$\sum \boldsymbol{ij} = \boldsymbol{Cov(X_i, X_j)}$$

Per calcolare la covarianza o la matrice di covarianza è necessario che non ci siano variabili categoriali ma solo numeriche.

La matrice della covarianza si calcola nella seguente maniera:

```
covariance_matr = data.drop('group',axis=1).cov()
print(f"{covariance_matr}")
```

La covarianza però è limitata all'unità di misura delle variabili ed è difficile con il solo valore calcolato se due variabili sono in correlazione(cov €]-∞,+∞[).

Il coefficiente di correlazione di Pearson risolve il problema dell'intervallo esteso, normalizzando la covarianza e rendendola adimensionale. Così, il valore è sempre compreso tra -1 e 1, dove:

- -1 correlazione perfetta negativa(antiassociate, X cresce Y descresce o viceversa);
- 0 nessuna correlazione lineare;
- 1 correlazione perfetta positiva(associate, X cresce Y cresce).

Essendo normalizzato il valore, si possono fare dei confronti più semplici tra coppie di variabili. La formula per calcolare il **coefficiente di correlazione di Pearson** è la seguente:

$$p(x, y) = \frac{Cov(X, Y)}{\sigma_x \sigma_y}$$

ovvero $p(x, y) = \frac{Cov(X,Y)}{\sigma_x \sigma_y} = \frac{1}{n}\sum(\frac{(x_i - \bar{x})}{\sigma_x} * \frac{(y_i - \bar{y})}{\sigma_y}) = Cov(z(X), z(Y))$

quindi l'operazione che si svolge è applicare lo z-scoring alla covarianza.

1. 0 <= p(x,y) <= 0.3 correlazione debole;
2. 0.3 <= p(x,y) <= 0.7 correlazione moderata;
3. 0.7 <= p(x,y) <= 1.0 correlazione forte

Si ricorda che la covarianza o il coefficiente di correlazione di Pearson è utilizzata solo per misurare le relazioni lineari di due variabili continue. Solo nel caso in cui abbiamo una variabile che assume due valori "1/0", si può calcolare **coefficiente di correlazione punto-biseriale**.

Si analizza come si comporta una variabile continua con una variabile dicotomica. Se il valore della variabile cambia molto al variare della variabile dicotomica, allora la covarianza è alta.

Il coefficiente di correlazione di Pearson misura soltanto correlazioni lineari. Ma possono esserci anche correlazioni diverse da quelle lineari, che si possono calcolare attraverso il **coefficiente di correlazione di Spearman(funzione non parametrica-> lavora anche con dati ordinali o classificaziones)**.

Le possibili correlazioni di cui parliamo possono essere quadratiche, logaritmiche, esponenziali, ecc., l'importante è che sia monotona; quindi, se X cresce allora anche Y crescerà e viceversa.

La funzione le seguenti operazioni:

- prendere il valore delle variabili e li ordina;
- assegnare ad ogni valore un rango in base alla posizione, dal più piccolo al più grande;
- confronta i ranghi e non i valori.

La verifica avviene tra gli ordinamenti di ogni osservazione e non sui valori.

La formula per calcolare il coefficiente è la seguente:

$$\boldsymbol{R = 1 - \frac{6\sum_{i=1}^{n} d_i^2}{n(n^2-1)}}$$

Dove d -> è la differenza tra ranghi per la i-esima osservazione = $R(X_i) - R(Y_i)$.

Funziona bene sia con dati continui e sia con dati ordinali, dato che sostituisce i valori con i ranghi.

Libreria in python: **from scipy.stats import spearmanr**; funzione: **spearmanr(colonna1, colonna2)**

Un'altra misura di correlazione non parametrica è il **coefficiente di correlazione di Kendall** o **τ(tau)di Kendall**.

Una coppia si considera concordante se prese n coppie (X_i,Y_i) e (X_j,Y_j) con $i < j$ allora $(X_i-X_j)(Y_i-Y_j) > 0$ che c'è una monotonia o crescente o decrescente. In sostanza se $X_i>X_j$ allora $Y_i>Y_j$ e viceversa.

Una coppia si considera discordante nel caso contrario.

La formula per calcolare $\boldsymbol{\tau}$ è la seguente:

$$\tau = \frac{numero\ coppie\ concordanti - numero\ coppie\ discordanti}{\frac{n}{2}(n-1)}$$

Dove $\frac{n}{2}(n-1)$è il numero di coppie totali.

Come abbiamo visto precedentemente, anche questa funzione appartiene alla libreria di python **scipy.stats**. La importiamo nella seguente maniera:

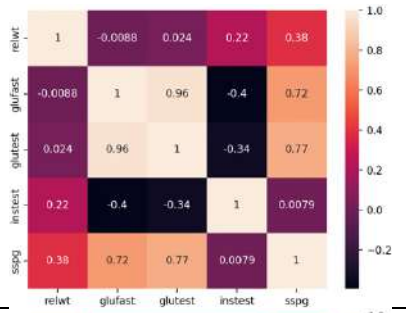
```
from scipy.stats import kendalltau
print(f"Kendall value between relwt and sspg{kendalltau(data['relwt'],data['sspg'])}")
```

Possiamo raggruppare tutte queste funzioni che calcolano la correlazione in un'unica funzione **.corr()**, appartenente alla funzione pandas.

Questa funzione crea una matrice di correlazione dove a secondo del tipo di correlazione ci sono coefficienti diversi. Poi si può stampare la matrice attraverso un grafico il cui codice è contenuto all'interno della libreria seaborn. Questo grafico è **heatmap** e quanto segue è una tabella con la rappresentazione grafica delle matrici e il codice correlato.

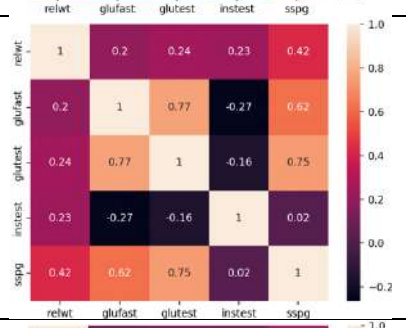
```
sbn.heatmap(data.drop('group', axis =1).corr(), annot=True)
plt.show()
```

Questo è il grafico che mostra la matrice di correlazione contenente il coefficiente di correlazione di Pearson. Vediamo che lungo la diagonale principale ci sono i feature in correlazione con sé stessi. Da questa matrice possiamo evincere una correlazione lineare tra instest e sspg.



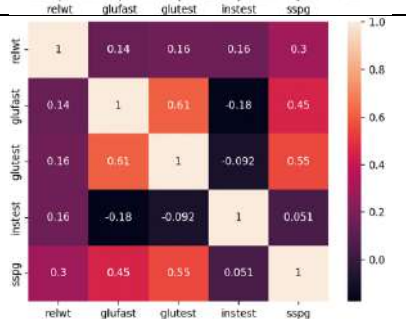

```
sbn.heatmap(data.drop('group', axis
=1).corr(method='spearman'), annot=True)
plt.show()
```

Questa è la matrice calcolata attraverso il metodo spearman. Essa non cerca linearità, bensì una monotonia nella distribuzione delle due variabili. I valori presenti nella matrice sono simili alla matrice precedente.




```
sbn.heatmap(data.drop('group', axis =1).corr(method='kendall'),
annot=True)#annot serve a mostrare i valori dentro le celle
plt.show()
```

Questa è la matrice calcolata attraverso il metodo kendall. Essa come la precedente cerca monotonia tra le variabili, verificando a coppie concordanza e discordanza.
I valori presenti nella matrice sono simili ai precedenti, con valori un po' più appiattiti.



Distribuzione dei dati

Si può assegnare un valore di probabilità ad un esito di una variabile casuale. È utile assegnarlo a tutti i valori che può assumere la variabile, e quest'operazione prende il nome di distribuzione di probabilità.

Il calcolo della probabilità per ogni valore di un intervallo discreto, si chiama **funzione di massa di probabilità (probability mass function PMF)**, perché ogni punto può essere visto come una piccola massa di probabilità.

Il calcolo della probabilità per ogni valore di un intervallo continuo, si chiama **funzione di densità di probabilità(probability density function PDF)**, perché dato che non si può calcolare la probabilità di un punto preciso, si calcola la probabilità di rientrare in un intervallo.

Una volta che scopriamo che una variabile casuale X è caratterizzata da una distribuzione di probabilità P, diremo che **X ~ P**.

P è la distribuzione di probabilità che dice:

- quali valori assume X;
- con quale probabilità X assume i valori.

La PMF, ovvero la probabilità per variabili discrete è la seguente funzione:

$$P:\Omega \longrightarrow [0,1]$$

e segue la proprietà:

$$\sum_{x \in \Omega} \boldsymbol{P(x)} \ = \ \mathbf{1}$$

Come già avevamo visto, in python per calcolare la probabilità di massa per una variabile, si fa attraverso la funzione:

variabile.value_counts(normalize=True).sort_index()

Se la variabile discreta può essere ordinata, possiamo definire **la funzione di distribuzione cumulativa(cumulative distributive function CDF)** in questo modo:

$$F(X) = P(X \ <= \ x) \ = \sum_{y<x} P(y)$$

In pratica si applica la ECDF che si calcola con la seguente formula:

$$\boldsymbol{F(X)} = \frac{\boldsymbol{numero\ di\ osservazioni\ minori\ di\ x}}{\boldsymbol{totale\ osservazioni}}$$

È un approccio più pratico, fatto attraverso i dati, perché nella CDF dovremmo sapere a prescindere la distribuzione dei dati.

In python invece si calcola con la seguente formula: **ecdf = np.arange(1, len(feature)+1)/ len(feature)**

In modo tale che si crei un array del genere: [1/n,2/n,...,n/n] che poi va associato all'array ordinato. Quindi se avessimo tipo: [1,1,1,2,3] allora ecdf = [1/5,2/5,3/5,4/5,5/5], infine nel plot in corrispondenza di 1 avremmo 3/5.

Per le variabili continue, la probabilità si calcola tramite un'integrale applicato sulla funzione di densità di probabilità. La formula quindi è la seguente:

$$f: \Omega \longrightarrow [0,1]$$

$$\int_{-\infty}^{+\infty} f(x)dx \ = \ 1$$

Possiamo definire quindi la probabilità attraverso l'integrazione:

$$\boldsymbol{P(a \leq x \leq b) = \int_{a}^{b} f(x)dx}$$

Dato che ci sono numeri infinitesimali, si utilizza l'integrale per raccogliere sotto lo stesso intervallo.

Se dovessimo poi mostrare la densità di probabilità in un grafico, potremmo intraprendere due scelte:

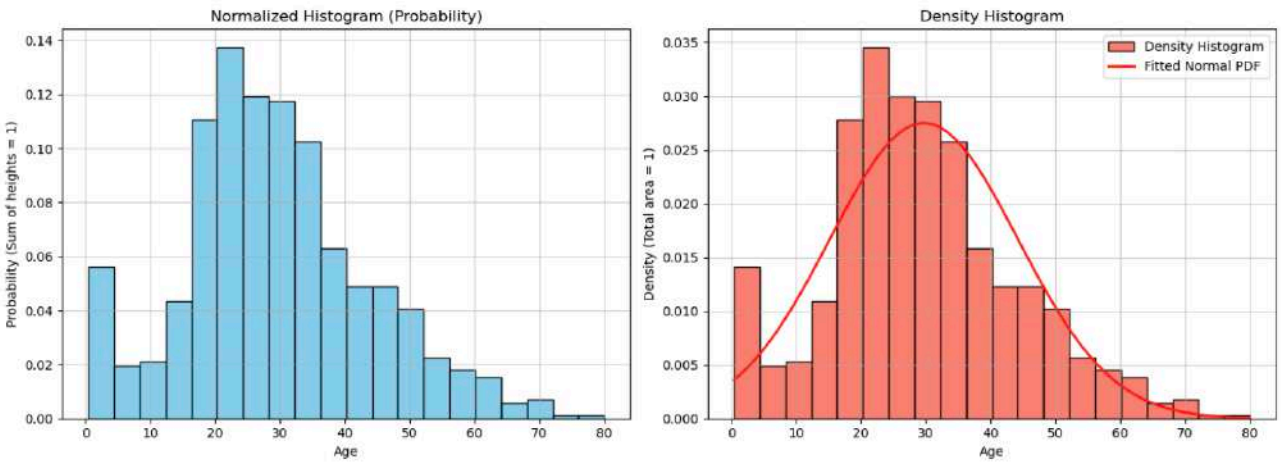
- istogramma normalizzato -> probabilità corrispondente all'altezza associata all'intervallo;
- densità normalizzata -> probabilità corrispondente all'area associata all'intervallo.

La differenza sta nella non precisione dell'istogramma normalizzato. Sappiamo che la probabilità è un'integrale e che quindi calcola tutte le osservazioni in un'area. Attraverso l'istogramma stiamo considerando discreto un valore continuo.

Infatti:

- $h_j = \frac{c_j}{n}$ dove c_j e h_j sono rispettivamente numero di osservazioni e altezza(probabilità) associata ad un bin b_j. Possiamo vedere come non rispetta $\sum_{j=1}^{n} Aj \ = \ \sum_{j=1}^{n} h_j w_j \ = \ \sum_{j=1}^{n} \frac{c_j}{n} w_j = \frac{1}{n}\sum_{j=1}^{n} c_j w_j \neq 1$-> si è dimostrato come l'istogramma non è preciso.
  n.b. = w è l'ampiezza del bin b_j
- nella densità normalizzata abbiamo che $h_j = \frac{c_j}{w_j n}$ così abbiamo che:

$$\sum_{j=1}^{n} A_j = \sum_{j=1}^{n} h_j w_j = \sum_{j=1}^{n} w_j \frac{c_j}{w_j n} = \frac{1}{n}\sum_{j=1}^{n} w_j * \frac{c_j}{w_j} = \frac{1}{n}\sum_{j=1}^{n} c_j = 1$$



Per calcolare la CDF associata ad una distribuzione continua su un intervallo, si integra nel seguente modo: $\boldsymbol{\int_{-\infty}^{x} f(x)dx}$

Esistono diverse distribuzioni di probabilità comuni che possono essere usate per descrivere eventi casuali. Quando abbiamo sufficienti prove che una distribuzione descrive i nostri dati, la possiamo semplicemente applicare.

Distribuzione uniforme della probabilità dei dati discreti

Quando abbiamo che tutti gli eventi sono equiprobabili, allora distribuzione della probabilità è la seguente:

$P(X = a_i) = \frac{1}{k}$ con k il numero di valori che può assumere la variabile in uscita

Ad esempio, i dadi seguono questa distribuzione, infatti:

$P(X = 1) = \frac{1}{6}, P(X = 2) = \frac{1}{6}, P(X = 3) = \frac{1}{6}, P(X = 4) = \frac{1}{6}, P(X = 5) = \frac{1}{6}, P(X = 6) = \frac{1}{6}$ con k = 6

Distribuzione di Bernoulli

La distribuzione di Bernoulli è una distribuzione su una variabile casuale binaria, ovvero una variabile che può assumere solo i valori [0,1].

La distribuzione è molto semplice, infatti:

$$\boldsymbol{P(X = 1) = \phi}$$

$$\boldsymbol{P(0) = 1 - \phi}$$

Quindi la distribuzione è controllata da un singolo valore ϕ € [0,1].

Distribuzione binomiale

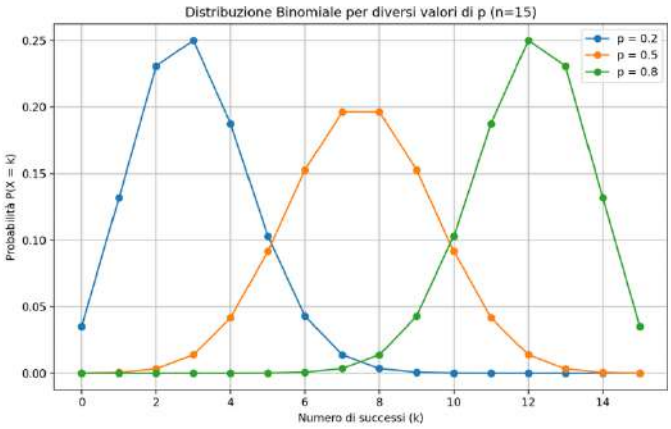
La distribuzione binomiale risponde alla domanda qual è la probabilità che si hanno k successi con n prove indipendenti.

La formula per calcolarla è la seguente:

$$\boldsymbol{P(k) = \binom{n}{k} p^k (1 - p)^k}$$

Dove:

- $\binom{n}{k}$ è la formula del calcolo combinatorio che può essere riscritta così $\frac{n!}{k!(n-k)!}$
- p è la probabilità che si verifichi l'evento



Nei grafici della distribuzione binomiale, sull'asse delle ascisse è riportato il numero di successi ottenuti nelle prove, mentre sull'asse delle ordinate sono rappresentate le probabilità con cui ciascun evento si verifica.
Per probabilità piccole, come 0,2, la curva si concentra a sinistra, vicino allo zero; per probabilità grandi, come 0,8, la curva si sposta a destra, verso il numero massimo di prove; infine, per probabilità intermedie, come 0,5, la curva risulta centrata e simmetrica.

Distribuzione Multinoulli/categoriale

La distribuzione categoriale studia la probabilità su una variabile casuale che può assumere un numero finito di stati.

La distribuzione sa già in anticipo:

- i possibili esiti che la variabile può assumere;
- le probabilità che un esito si verifica.

Quindi avremmo un vettore di probabilità dove ogni componente è la probabilità che si verifica l'esito j

$$P = \{P_1, \ldots, P_n\}$$

$$\sum_{j=1}^{n} Pj = 1$$

Distribuzione multinomiale

La distribuzione multinomiale è una generalizzazione della distribuzione binomiale, dove una variabile può assumere più di due valori. Si conta quante volte succede ciascuno dei k esiti in n prove indipendenti.

Si ha:

- n prove indipendenti;
- p vettore probabilità $\{p_1, \ldots, p_k\}$
- X vettore successi desiderati $\{x_1,\ldots,x_n\}$

Si vuole quindi calcolare $P(x_1,\ldots,x_n)$ e si fa attraverso la seguente formula:

$$\frac{\boldsymbol{n}!}{\boldsymbol{x_1}!\,\boldsymbol{x_2}!\ldots\boldsymbol{x_n}!}(\boldsymbol{p_{x1}})^{\boldsymbol{x1}}(\boldsymbol{p_{x2}})^{\boldsymbol{x2}}\ldots(\boldsymbol{p_{xn}})^{\boldsymbol{xn}}$$

Ad esempio, supponendo di avere un dado non truccato e i seguenti dati:

- 15 prove
- p $=\{\frac{1}{6},\frac{1}{6},\frac{1}{6},\frac{1}{6},\frac{1}{6},\frac{1}{6}\}$
- X = {2,4,3,0,5,1} quindi vogliamo trovare esattamente 2 volte 1, 4 volte 2 ...

Calcoliamo la probabilità di X attraverso la formula:

P(2,4,3,0,5,1) = $\frac{15!}{2!4!3!0!5!1!}\ (\frac{1}{6})^2(\frac{1}{6})^4(\frac{1}{6})^3(\frac{1}{6})^0(\frac{1}{6})^5(\frac{1}{6})^1 = \frac{1307674368000}{34560} * 0{,}027 * 0{,}0007 * 0{,}004 * 0{,}00012 * 0{,}116$ = numero infinitesimale.

Distribuzione Gaussiana

Sinora si è parlato di distribuzione di probabilità legata a valori discreta, e si usano le PMF funzioni di massa di probabilità. Per funzioni che lavorano con valori reali, serve una PDF, cioè una funzione che descrive la densità di probabilità su un intervallo di valori. Una delle distribuzioni più rilevanti studiate per valori reali è la distribuzione gaussiana.

La distribuzione è totalmente descritta da due numeri:

- μ media -> centro della campana;
- $\sigma\ deviazione\ standard$ -> misura quanto i valori si disperdono intorno alla media;

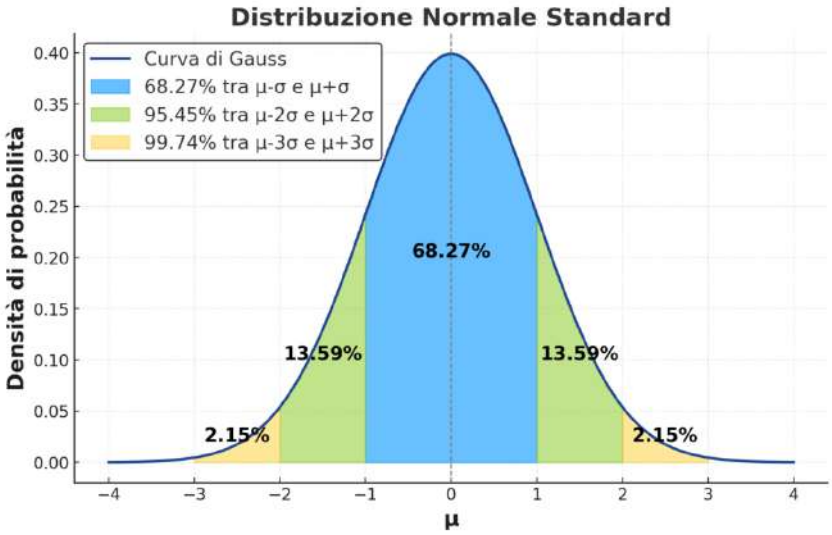
La formula per calcolare la distribuzione gaussiana è la seguente:

$$\boldsymbol{N(x, \mu, \sigma^2)} = \sqrt{\frac{\mathbf{1}}{\mathbf{2\pi\sigma^2}}} \boldsymbol{e}^{\frac{-(x-\mu)^2}{2\sigma^2}}$$

Si utilizza molto, soprattutto quando non si sa molto sulla distribuzione reale perché c'è il **teorema sul limite centrale** che dice che:

la somma o media di più variabili casuali indipendenti che hanno la stessa distribuzione tendono ad avere una distribuzione normale, anche se le singole variabili non sono normali.

Anche se le distribuzioni di partenza non sono normali, la somma delle distribuzioni o la media assumerà una distribuzione normale.



Distribuzione Gaussiana Multivariata

Abbiamo visto come la distribuzione Gaussiana si può applicare a variabili continue e analizza come esse si comportano singolarmente, infatti l'analisi è univariata.

Per generalizzare la distribuzione gaussiana a più variabili e averne una multivariata, si avrà:

- $\mu = \begin{matrix} \mu_1 \\ \cdots \\ \mu_n \end{matrix}$ è un vettore contenente tutte le medie
- $\Sigma = matrice\ di\ covarianza$

La formula sarà la seguente:

$$N(\mathbf{x}; \mu, \boldsymbol{\Sigma}) = \sqrt{\frac{1}{(2\pi)^d \det(\Sigma)}} e^{\left(-\frac{1}{2}(\mathbf{x}-\mu)^T \Sigma^{-1} (\mathbf{x}-\mu)\right)}$$

Statistiche chiave per la distribuzione dei dati

Alcune statistiche ci permettono di riassumere le distribuzioni attraverso funzioni che calcolano tendenze centrali, correlazioni tra due feature ed eventuali confronti attraverso funzioni che standardizzano i dati.

Una tra queste statistiche chiave è il **valore atteso** ovvero il valore medio teorico che si calcola per una variabile casuale tenendo in conto delle relative probabilità.

Si fa una media pesata dei valori e varia a seconda delle variabili analizzate, continue o discrete.

La formula per calcolare il valore atteso nelle variabili discrete è la seguente:

$$E_{X\sim P}[X] = \sum_{\boldsymbol{x}\in\boldsymbol{\Omega}} \boldsymbol{x} * \boldsymbol{P}(\boldsymbol{x})$$

La formula per calcolare il valore atteso nelle variabili continue è il seguente:

$$E_{X\sim P}[\boldsymbol{X}] = \int_{\boldsymbol{x}\in\boldsymbol{X}} \boldsymbol{x}\boldsymbol{f}(\boldsymbol{x})\boldsymbol{dx}$$

dove f(x) è la densità di probabilità di x.

Nel contesto delle variabili casuali per calcolare la varianza e la deviazione standard si utilizzano le seguenti formule:

- $\boldsymbol{Var_{X\sim P}[X] = E_{X\sim P}[(X - E_{X\sim P}[X])^2]}$ non si deve fare alcuna divisione perché già si sa che se gli eventi sono o non equiprobabili, vengono lo stesso pesati e quindi avranno un loro peso nel calcolo;
- $\boldsymbol{std_{X\sim P}[X] = \sqrt{Var_{X\sim P}[X]}}$ la deviazione standard è conseguenza della varianza.

Anche la covarianza è influenzata dal calcolo della varianza e misura quando due variabili sono linearmente correlate.

La formula verrà riscritta nel seguente modo:

$$\boldsymbol{Cov_{X\sim P, Y\sim P}(X,Y) = E_{X\sim P, Y\sim P}[(X - E_{X\sim P}[X])^2 * (Y - E_{Y\sim P}[Y])^2]}$$

Un altro modo per caratterizzare una distribuzione è attraverso l'uso dell'entropia, che misura l'incertezza media o l'imprevedibilità associata ai valori di una variabile casuale.

Prima di parlare dell'entropia, dobbiamo analizzare la **self information**, ovvero quanto sorprendente o informativo è un evento.

$$\boldsymbol{I(x) = -\log_2 P(X)}$$

Il segno meno, serve a rendere positivo il risultato dato che 0< $\log_2 x$ <1 €]-∞,0[, e si utilizza la base 2 perché parliamo di bit.

Per eventi che hanno probabilità alta si impiegano pochi bit, viceversa se hanno basse probabilità. Ad esempio il lancio della moneta:

$$I(croce) = -\log_2(0.5) = 1 \text{ bit}$$

Se due eventi sono indipendenti, cioè P(A,B) = P(A)*P(B) allora I(A,B) = $-\log_2(P(A,B))$= $-\log_2(P(A)) - \log_2(P(B))$

L’**entropia H(X)** di una variabile casuale X **misura** quanta **incertezza** o imprevedibilità c’è nei suoi valori. È la media pesata della self-information di tutti i singoli eventi, pesata secondo la loro probabilità.

Le formule per i due casi, è la seguente:

- valori discreti: $-\sum_{x€\Omega}(P(x) * \log_2(P(x))$
- valori continui: -$\int_{x€\Omega} f(x) \log_2(f(x))dx$

Quando l’entropia ha un valore:

- alto -> valori quasi tutti uguali in probabilità, difficile intuire il prossimo valore;
- basso -> concentrazione probabilità in pochi valori, tutto il resto ha probabilità bassa;

Se la distruzione dei valori è piatta, allora l’entropia è alta -> lancio della moneta.

Se la distribuzione è concentrata, allora l’entropia è bassa -> il sole sorge(evento certo e molto sbilanciato su un valore[SI]).

Inferenza statistica

Abbiamo visto come si descrive un campione di dati attraverso concetti astratti dateci dalla teoria della probabilità. Quello che si vuole descrivere però non è un solo campione, ma l'intera popolazione.

La cosa più intuitiva da fare è lavorare con la popolazione direttamente, ma questo è possibile in dataset piccolissimi. Per questo motivo, si deve campionare un sottoinsieme dell'intere popolazione e cercare di dedurre le proprietà della popolazione dal campione. Questa operazione è detta **inferenza statistica**. L'analisi dei dati mediante queste tecniche è detta **analisi inferenziale**.

La prima fase dell'analisi inferenziale è il campionamento. Il modo più semplice per effettuare un campionamento è farlo in modo casuale. Si fanno due assunzioni per effettuare il campionamento casuale:

- **unbiasedness(imparzialità)**: ogni elemento ha la stessa probabilità di essere scelto per il campione;
- **indipendence(indipendenza)**: la selezione di un elemento non influenza le selezioni degli altri elementi.

più grande è il campione, più rispetta il comportamento della popolazione.

Durante il campionamento quindi potremmo incorrere in determinati problemi come:

- **bias di selezione**: il campione non è rappresentativo della popolazione da cui dovrebbe venire, perché alcune categorie siano sovrarappresentate o sottorappresentate -> ad esempio, un sondaggio sulle politiche adottate dal governo in carica fatto solo nelle piazze del Lazio -> non rappresenta il resto d'Italia;
- **bias di risposta**: osservazioni distorte a causa della non accuratezza e non sincerità dei partecipanti. I motivi potrebbero essere tanti come: desiderio di apparire in un certo modo, incomprensione delle domande o distrazione. -> ad esempio, domande lunghe e ambigue potrebbero confondere il partecipante.
- **bias di copertura(coverage bias)**: quando il mezzo di raccolta non raggiunge in modo equo la popolazione -> ad esempio, raccolta informazioni per mezzo di instagram che esclude chi non utilizza questo social(non ha uno smartphone o dispositivo avanzato, per scelta).

A volte la popolazione in esame è eterogenea ed effettuare un campionamento casuale potrebbe non fare emergere le caratteristiche di ogni sottogruppo. Ad esempio, supponendo che la categoria voto è influenzata dalla categoria professione, se non analizzassimo buona parte delle professioni potremmo avere i risultati alterati.

Per risolvere il problema possiamo utilizzare il **campionamento stratificato(stratified sampling)**.

Il campionamento stratificato prevede che:

- si divide la popolazione in gruppi omogenei basati su variabili rilevanti, come la professione;
- si esegue un campionamento casuale per ogni gruppo.

Per effettuare questo campionamento si richiede una conoscenza approfondita sulla popolazione, ma la rappresentatività migliora.

Le librerie pandas e numpy semplificano il campionamento in python. Se volessimo campionare da una distribuzione, allora si potrebbe procedere come segue:

```
import numpy as np
from matplotlib import pyplot as plt
samples = np.random.normal(mean:= 0, std:=1, size= 100)
samples = pd.Series(samples)
samples.plot.density()
plt.show()
```

Oltre alla distribuzione normale, potremmo plottare altre distribuzioni come: esponenziale, uniforme, laplaciana e molte altre descritte nel capito precedente.

Se volessimo invece campionare da un dataframe, si potrebbe procedere come segue:

```
#random sampling
import pandas as pd
titanic = pd.read_csv(link, index_col='PassengerId')
sample = titanic.sample(100, replace=True) #replace permette il ripescaggio
```

```
#stratified sampling
import pandas as pd
titanic = pd.read_csv(link, index_col='PassengerId')
sample = titanic.groupby('Pclass').apply(lambda x:x.sample(10))
```

Presi dei campioni, possiamo calcolare statistiche come la media e osservare che tendono a convergere ai valori ottenuti applicando le stesse operazioni sull'intera popolazione. Questo è possibile grazie al **teorema del limite centrale**, secondo il quale le somme o le medie di campioni indipendenti tendono ad assumere una distribuzione normale, indipendentemente dalla distribuzione originaria della popolazione.

Il valore atteso e la deviazione standard delle medie campionarie permettono di valutare la precisione e l'affidabilità della stima della media della popolazione ottenuta da un singolo campione.

Poiché la deviazione standard della popolazione (σ) è sconosciuta, utilizziamo la deviazione standard del campione (s) come stima. Tuttavia, questa descrive la variabilità **dei dati nel campione**, non quella delle medie campionarie.

Quindi non ci da un indice di dispersione dalla media reale, ma da quella campionaria. Per sapere quanto la campionaria si disperde dalla media vera possiamo utilizzare la formula dello **standard error**, ossia:

$$SE(\bar{X}) = \frac{\sigma}{\sqrt{n}}$$

SE è inversamente proporzionale a $\sqrt{n}$ quindi all'aumentare del numero di osservazioni dentro un campione diminuisce SE.

Importante è capire che se raddoppiamo le osservazioni all'interno del campione, l'errore si riduce del 30%, e per dimezzarlo dobbiamo quadruplicare i dati. Per ridurre notevolmente dobbiamo aggiungere molte più osservazioni di quanto potremmo aspettarci.

Gosset, chimico e matematico, si pose il problema di stimare caratteristiche della popolazione a partire da campioni piccoli. Il problema era l'incertezza dovuta alla piccola dimensione del campione. Per questo motivo sviluppò una nuova distribuzione, simile alla normale nel suo andamento a campana, ma leggermente più appiattita e con le code più spesse, in modo da tenere conto della maggiore variabilità degli estremi.

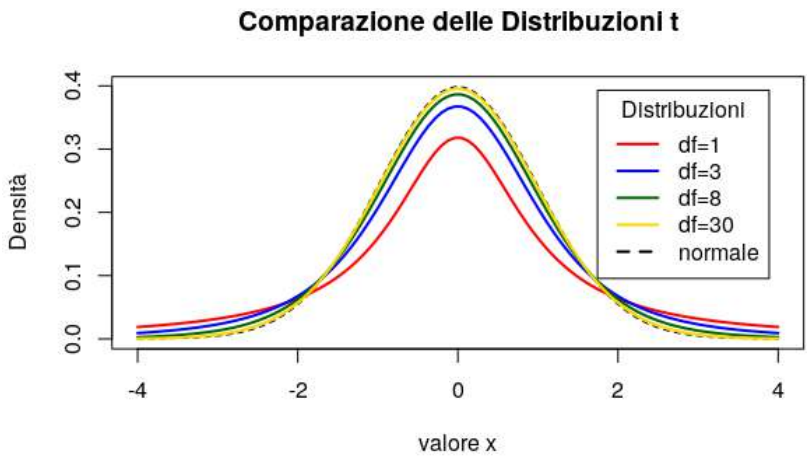
Questa distribuzione è la t di student e la formula è la seguente:

$$\boldsymbol{t = \frac{\bar{X} - \mu}{\sigma/\sqrt{n}} = \frac{\bar{X} - \mu}{SE}}$$

dove:

- $\bar{X}$ è la media calcolata;
- μ è la media ipotizzata della popolazione;
- SE è l'errore standard, ovvero la deviazione della media campionaria rispetto la media reale;
- t è il valore che ci dice se siamo vicini alla media ipotizzata(t valori bassi) o siamo lontani(t valori alti).

t sarà un multiplo dello standard error, infatti esso ci indicherà quanto si allontana da dalla media in termini di standard error.



La media μ della popolazione stimata a partire da una media campionaria $\bar{x}$, potrebbe non essere esattamente precisa, e può variare a seconda del campione studiato.

Per questo motivo al posto di avere una media puntuale dovremmo stimare un intervallo detto **intervallo di confidenza** che vede dei valori entro cui è possibile trovare la media della popolazione.

Quando la varianza della popolazione non è nota, questo intervallo si costruisce utilizzando la distribuzione t di Student, che tiene conto della variabilità aggiuntiva introdotta dal campione.

Sappiamo per il teorema del limite centrale che le medie campionarie seguono una distribuzione normale, e che quindi:

- 68,3% dei campioni cade sull'intervallo $[\mu - \sigma, \mu + \sigma]$
- 95% dei campioni cade sull'intervallo $[\mu - 2\sigma, \mu + 2\sigma]$
- 99,7% dei campioni cade sull'intervallo $[\mu - 3\sigma, \mu + 3\sigma]$

Quindi sappiamo con la confidenza del 68,3% che:

$$\bar{x} \in [\mu - \sigma, \mu + \sigma] \leftrightarrow \mu \in [\bar{x} - \sigma, \bar{x} + \sigma]$$

per il 68,3% dei campioni, abbiamo che:

$$P([\bar{x} - \sigma \leq \mu \leq \bar{x} + \sigma] = 68{,}3\%$$

Questo che abbiamo ottenuto è un intervallo di confidenza. La media campionaria è già calcolata, ma non abbiamo la deviazione standard. Nell'intervallo di confidenza possiamo sostituire la deviazione standard con lo standard error della media come segue:

$$[\bar{x} - SE(\bar{x}), \bar{x} + SE(\bar{x})]$$

L'intervallo di confidenza trovato ha però un livello di confidenza piuttosto basso, poiché lascia fuori il 31,7% dei casi. Stimato un livello di confidenza p, possiamo moltiplicare la deviazione standard per β e trovare i campioni presenti in p% dei casi:

$$P(\bar{x} - \beta\sigma \leq \mu \leq \bar{x} + \beta\sigma) = p$$

utilizzando l'SE:

$$[\bar{x} - \beta SE(\bar{x}), \bar{x} + \beta SE(\bar{x})]$$

Adesso giungiamo al collegamento con la t-student. Per stimare quanto siamo lontani dalla media della popolazione, possiamo utilizzare la seguente formula:

$$[\bar{x} - tSE(\bar{x}), \bar{x} + tSE(\bar{x})]$$

Il valore di t proviene dalla distribuzione t di Student, che tiene conto della variabilità del campione e del livello di confidenza scelto.

Non si può calcolare direttamente, perché richiederebbe la media della popolazione (sconosciuta), ma si ottiene tramite tabelle o software statistici in base ai gradi di libertà e alla probabilità cumulativa (p.c.f.) della distribuzione.

Finora si sono costruiti intervalli di confidenza utilizzando formule basate sul teorema del limite centrale, ovvero che una statistica segue la distribuzione normale.

Ma cosa accade se il campione è troppo piccolo e non segue la normale?

Per operazioni statistiche più complesse di media o mediana, potrebbe essere difficile avere formule semplici.

Per questo motivo entra in gioco il **bootstrapping**, un metodo computazionale potente che permette di stimare la distribuzione campionaria di qualsiasi statistica, utilizzando il solo campione che si ha a disposizione.

Il bootstrapping prevede in fasi:

- Prelievo del campione e sostituzione della popolazione;
- Estrazione dei nuovi campioni dal campione della stessa misura. Alcuni valori possono essere ripetuti e altri non essere presenti nel campione preso in considerazione - **ricampionamento**;
- Calcolo della statistica di interesse migliaia di volte a campioni dati dal ricampionamento;
- Costruzione della **distribuzione campionaria empirica**(trovata attraverso i dati).

```
from scipy import stats
sample = titanic['Age'].dropna().sample(150).to_numpy()
sample = (sample,)
bootstrap_result = stats.bootstrap(sample,np.mean,n_resamples=1000,confidence_level=0.95)
confidence_interval = bootstrap_result.confidence_interval
print(f"real mean: {titanic['Age'].mean()}")
print(f"confidence interval: [{confidence_interval.low:.2f},{confidence_interval.high:.2f}]")
```

```
real mean: 29.69911764705882
confidence level interval: [30.54,32.95]
```

Il **bootstrapping** è quindi una valida alternativa per trovare l'intervallo di confidenza, senza che si calcoli t student o che si assuma che la distribuzione sia normale. Semplicemente se si ha un campione generalmente rappresentativo della popolazione, allora: si può ricampionare migliaia di volte, calcolare la statistica desiderata, osservare la distribuzione e prendere i percentili associali all'intervallo di confidenza.

Quando effettuiamo la stima della media della popolazione da un campione dobbiamo distinguere:

- **estimatore** -> funzione utilizzata per effettuare la stima della media -> si indica con T($\bar{X}$) ;
- **stima** -> valore concreto ottenuto da un campione -> varia a seconda del campione.

Il **bias** è la differenza tra il valore ottenuto attraverso l'estimatore e la vera media della popolazione:

$$\mathbf{BIAS(T(\bar{X}))} = \mathbf{E[T(\bar{X})] - \phi}$$

Se $\mathbf{E[T(\bar{X})] = \phi}$ allora $\mathbf{BIAS(T(\bar{X})) = 0}$

dove $\boldsymbol{\phi}$ assume il valore reale della statistica applicata alla popolazione.

Il bias quindi misura la differenza media tra il valore atteso dell'estimatore e il valore vero della popolazione.

In base al valore del bias abbiamo le seguenti interpretazioni:

- se $\mathbf{BIAS(T(\bar{X})) = 0}$, il valore **non è distorto**;
- se $\mathbf{BIAS(T(\bar{X})) > 0}$, il valore è **sovrastimato**;
- se $\mathbf{BIAS(T(\bar{X})) < 0}$, il valore è **sottostimato.**

Un'altra metrica utilizzabile nel caso in cui si lavori con campioni è la **varianza**.
In questo contesto, la varianza misura quanto i valori stimati da campione a campione oscillano attorno al loro valore medio atteso.

La formula è la seguente:

$$\boldsymbol{Var(T(\bar{X}))} = \boldsymbol{E[(T(\bar{X}) - E[T(\bar{X})])^2]}$$

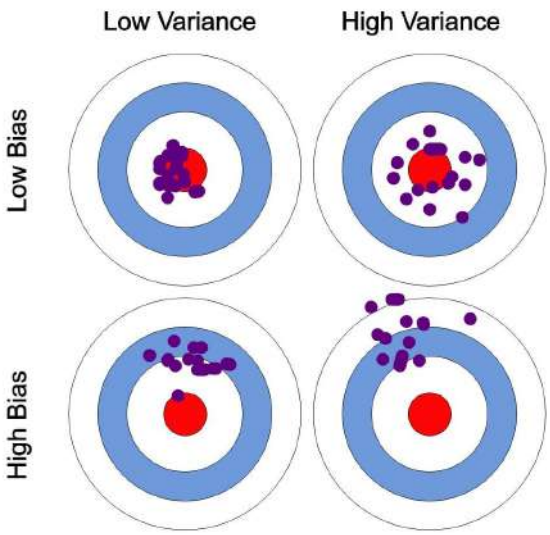
A seconda dei valori della varianza possiamo dedurre che:

- se $\boldsymbol{Var(T(\bar{X}))}$ è **bassa** allora le **stime** ottenute dai campioni sono **stabili** e **simili**;
- se $\boldsymbol{Var(T(\bar{X}))}$ è **alta** allora le **stime cambiano molto** da campione a campione;

Idealmente noi vogliamo un estimatore con **bias basso e varianza bassa**. Ciò vuol dire che il valore atteso si trova vicino al valore dell'operazione eseguita sulla popolazione(bias basso), inoltre, tutti i campioni differiscono poco tra di loro(bassa).

In pratica però abbiamo la possibilità di visualizzare 4 scenari:

- **low bias, low variance**: i valori stimati sono raggruppati intorno al valore calcolato sulla popolazione;
- **low bias, high variance**: il valore atteso è vicino al valore calcolato sulla popolazione ma le stime si oscillano molto attorno al valore atteso;
- **high bias, low variance**: il valore atteso è lontano dal valore calcolato sulla popolazione ma le stime sono stabili;
- **high bias, high variance**: il valore atteso è lontano dal valore calcolato sulla popolazione e le stime non sono stabili e simili.



Quando lavoriamo con gli intervalli di confidenza, a partire da un campione cerchiamo di stimare il valore di un'intera popolazione.

Attraverso un **test di ipotesi** si verifica un'affermazione specifica su una variabile. Formuliamo un'ipotesi iniziale (**ipotesi nulla**) e cerchiamo di dimostrare che sia falsa; se non ci sono prove sufficienti per rigettarla, la assumiamo valida.

Per esempio, nel caso del Titanic, potremmo chiederci se chi stava in prima classe o chi era più giovane avesse più probabilità di salvarsi. Analizzando i campioni, cerchiamo di verificare il contrario dell'ipotesi: se non troviamo evidenze, l'ipotesi iniziale rimane accettata.

Utilizziamo un esempio per capire:

Abbiamo un forno che fa biscotti, e li raggruppa in pacchi da 1000g. Se prendessimo 500 pacchi con i seguenti valori:

$\bar{x} = 1000.0005$ $\qquad$ $s(\sigma)_{n-1} = 0.01g$

L'ipotesi nulla(H_0) = la media dei pacchetti è 1000g.

L'ipotesi alternativa(H_a) = la media dei pacchetti è differente da 1000g.

Ora dobbiamo imporre un **livello di significatività**, ovvero una soglia predefinita che indica **quanto siamo disposti a rischiare di rifiutare l'ipotesi nulla H_0quando in realtà è vera**.

$$\alpha = 0.05$$

Possiamo al massimo rischiare di rifiutare l'ipotesi con una probabilità del 5%.

Ora ci chiediamo quanto la nostra media campionaria si discosta dalla media assunta $\mu = 1000g$

Per rispondere alla domanda, utilizzeremo la t-student come segue:

$$t = \frac{\bar{x} - \mu}{s/\sqrt{n}} = \frac{1000.0005 - 1000}{0.01/\sqrt{500}} = \frac{0.0005}{0.01/22.36} = \frac{0.0005}{0.00044} = 1.12$$

Questo numero ci dice in termini di standard error quanto la media campionaria si allontana dalla media assunta per la popolazione.

Adesso possiamo calcolare il **p-value**, ovvero la **percentuale di valori che sono uguali o estremi del valore t che abbiamo trovato**. Ovviamente dobbiamo verificare sia valori maggiori sopra la media che sia valori al di sotto della media.

P(Z > |1.12|) ≈0.13 a destra, per verificare entrambi i lati della distribuzione avremmo

p-value = 13%*2 = 26%

p-value > livello di sginificatività

Possiamo accettare l'ipotesi, perché:

- se p-value < α, allora si rifiuta perché il risultato osservato è molto improbabile sotto l'ipotesi nulla, è più plausibile l'ipotesi alternativa.
- se p-value $\geq \alpha$, allora si accetta dato che il risultato osservato non è improbabile sotto l'ipotesi nulla e non abbiamo prove per dimostrare il contrario.

Test ipotesi step-by-step:

- dai dati che si hanno, calcolare la t-student;
- stabilire un livello di significatività per l'ipotesi nulla;
- calcolare il p-value verificando i valori uguali a t o più estremi;
- confrontare p-value con livello di significatività.

Il test statistico serve per rifiutare l'ipotesi oppure per accettarla. Siccome il test viene effettuato sul campione e non su tutta la popolazione allora non si è sicuri al 100%.

Possiamo quindi effettuare due errori:

- **Errore di primo tipo o falso positivo** -> si rifiuta l'ipotesi nulla anche se è vera. Si cade nel livello di significatività α, ovvero la probabilità che accettiamo come rischio di rifiutare l'ipotesi anche se è vera.
- **Errore di secondo tipo o falso negativo** -> si accetta l'ipotesi nulla anche se è falsa. Si è fallito nel rifiutare l'ipotesi nulla, quindi si accetta anche se è falsa.

Nel test di ipotesi possiamo decidere quale deviazione analizzare, ovvero si possono avere due situazioni sotto-citate:

- **Test a due code**: si effettua quando vogliamo consultare la percentuale di elementi che sono estremi o uguali al valore della t-student presenti in entrambe le code. le code. Ad esempio, controllare se la media(o qualsiasi altra operazione) campionaria è diversa dalla media della popolazione indipendentemente dalla coda di appartenenza.
- **Test a una coda**: si effettua quando vogliamo solo consultare una delle due code. Ad esempio, controllare se la media campionaria è minore o maggiore della media della popolazione.

```
#VERIFICA SE LA MEDIA È UGUALE A 30 ATTRAVERSO TEST DI IPOTESI
samples = titanic['Age'].sample(500, replace=True).dropna()
null_hypothesis_age = 28
t_stat, p_value = stats.ttest_1samp(samples,null_hypothesis_age)#si calcola la t-student
e il p-value

alpha = 0.05
if p_value < alpha :
    print("Ipotesi nulla rigettata")
else:
    print("Impossibile rigettare l'ipotesi nulla")
print(f"t-statistic: {t_stat:.2f}")
print(f"P-value: {p_value:.4f}") #il p-value avrà automaticamente il valore per il test
a due code
```

vediamo come attraverso stats appartenente a scipy e attraverso alla funzione ttest_1samp possiamo calcolare p_value e t student.

Possiamo anche calcolare la differenza della media campionaria attraverso il seguente codice:

```
t_stat,p_value = stats.ttest_ind(samplem,samplef)
```

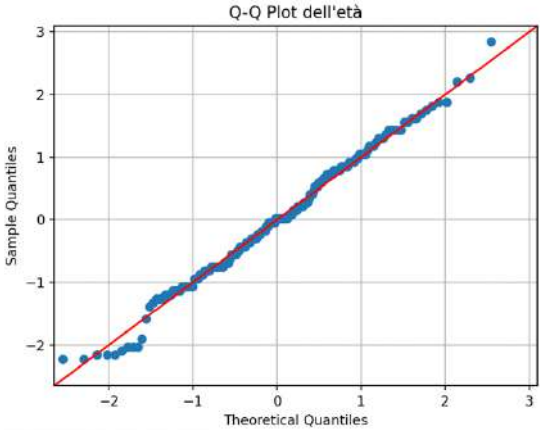
Possiamo utilizzare il test delle ipotesi quando dobbiamo verificare se due variabili sono in correlazione oppure no. Si utilizza il chi-quadro di Pearson, e attraverso il p-value che ritorna poter dire se sono in relazione.

VERIFCARE SE LA DISTRIBUZIONE È GAUSSIANA

Non è sempre detto che una distribuzione sia normale, quindi, è importante verificare che la distribuzione sia normale prima di presumere che sia vero. Possiamo verificarlo attraverso **skew**(che verifica se ci sono code allungate a destra o sinistra) e **kurtosis**(verifica l'ampiezza delle code e la concentrazione dei valori attorno alla media -> verifica se è appuntito o schiacciato) funzioni che sono presenti in scipy.stats, ma possono anche essere utilizzati vari test.

Questi test sono i **grafici quantile-quantile**, dove si mettono a confronto i quanti dei dati empirici con i quantili di una distribuzione di riferimento, tipo la normale. Si hanno quindi le diagonali per i quantili delle distribuzioni di riferimento e le curve per i quantili dei dati empirici, e se le curve si allontanano dalla retta allora vuol dire che ci sono delle deviazioni.

```
from statsmodels.graphics.gofplots import
qqplot

età = titanic['Age'].dropna()

qqplot(età, line='45', fit=True)
plt.title("Q-Q Plot dell'età")
plt.grid(True)
plt.show()
```



Altrimenti per campioni piccoli, possiamo anche verificarlo con il **test di Shapiro**. Confronta i dati osservati con ciò che ci dovremmo aspettare se i dati fossero tratti da una gaussiana.

Quindi importiamo la funzione Shapiro da scipy.stats, che calcola p_value e t student dalla distribuzione. Poi effettueremo il confronto con α, livello di significatività e se minore allora non segue la gaussiana, altrimenti il contrario.

```
from scipy.stats import shapiro
t_stat,p_value = shapiro(titanic['Age'])
alpha = 0.05
if p_value > alpha:
    print("segue la gaussiana")
else:
    print("non segue la gaussiana")
```

Questa funzione si può utilizzare per campioni che vanno fino a 2000 osservazioni.

Per verificare se un campione molto grande segue la normale, si utilizza il **test k-quadrato di d'Agostino**.

L'ipotesi nulla come per il test precedente è che il campione segue la normale.

```
from scipy.stats import normaltest
t_stat,p_value = normaltest(titanic['Age'])
alpha = 0.05
if p_value > alpha:
    print("segue la gaussiana con d'agostino")
else:
    print("non segue la gaussiana con d'agostino")
```

INTRODUZIONE ALL'ANALISI PREDITTIVA

Nei capitoli precedenti abbiamo visto come:

- riassumere i dati(media, mediana, ecc.);
- studiare le relazioni tra variabili;
- verificare se le conclusioni potessero essere generalizzate alla popolazione.

L'analisi fatta nei capitoli a seguire si concentrerà sulla **modellazione predittiva**, che si concentra su:

- comprendere le relazioni tra le variabili;
- costruire i modelli che permettano di fare predizione sui dati non osservati.

Il primo passo per raggiungere gli obiettivi sopraccitati è la creazione di un modello predittivo. Un **modello** nell'analisi dati è una semplificazione della realtà, come una mappa geografica per il globo.
Anche se i modelli non sono perfettamente rappresentativi della realtà, essi sono utili. Assumendo che tutti i modelli sono sbagliati, il modello più giusto è il modello che è più semplice o utile per raggiungere il nostro obiettivo, non quello più rappresentativo.
Ad esempio, quando utilizziamo una gaussiana per riassumere i dati stiamo utilizzando un modello. Se la gaussiana è applicata ad una colonna contenente il BMI, potremmo notare valori negativi, non fisiologicamente corretti, o arrotondati.
Esso è molto utilizzato perché senza consultare tutti i dati, sappiamo il valore tipico e quando i valori variano da esso, permettendoci di sapere molto sui dati su cui stiamo lavorando.
Quando si effettua un'analisi predittiva, assumiamo di osservare:

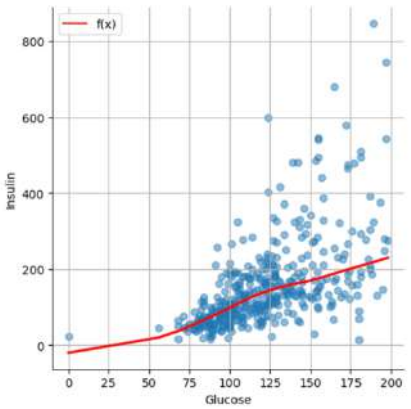
- **Y** la variabile che vogliamo predire, che solitamente viene chiamata **variabile dipendente** o **variabile di risposta**;
- **$X = \{X_1, X_2, \dots, X_n\}$** un insieme di variabili, chiamate **predittori**, che useremo per predire Y.

Possiamo quindi esprimere un modello predittivo nel seguente modo:

$$\boldsymbol{Y = f(x) + \varepsilon}$$

Dove:

- **f** è il **modello predittivo**, che si deve trovare;
- $\boldsymbol{\varepsilon}$ è il **termine di errore**, dato che i nostri modelli non possono fare previsioni perfette.



Come possiamo vedere nell'immagine, alcuni punti si trovano lontani dalla curva ottenuta dal modello.
Questo accade perché, nella pratica, i dati reali non seguono mai perfettamente l'andamento teorico descritto dal modello.
Ogni osservazione può infatti essere influenzata da numerosi fattori non considerati nel modello, come errori di misura, variabili non incluse o semplicemente la naturale variabilità del fenomeno osservato.
Per questo motivo viene introdotto il concetto di errore, rappresentato dal termine ε.

Un modello può essere costruito per due motivi:

- **Comprendere** -> Perché? -> si cercano di capire le cause e i meccanismi dietro un fenomeno. Si mira ad ottenere spiegazioni profonde per intraprendere decisioni informate. Si cerca di analizzare la scatola nera e trovare i parametri del modello, utili poi a capire quali variabili influenzano o sono influenzate.
- **Prevedere** -> Cosa? -> obiettivo principale è anticipare eventi o comportamenti per agire in modo strategico. Il valore del modello risiede nella sua capacità di prevedere le osservazioni

future. Non serve a capire i dettagli della scatola nera, ma a capire quale valore assumerà Y se viene aggiunto un nuovo valore di X.

Un obiettivo non esclude l'altro anzi spesso i modelli supportano sia l'inferenza statistica che la previsione. Comprensione e previsione hanno portato allo sviluppo di due approcci diversi, anche se gli strumenti matematici utilizzati sono condivisi.

Approccio statistico "GLASS BOX"

L'approccio **glass box** si basa sulla comprensione e inferenza, che ci permettono di guardare all'interno e capire i meccanismi.
La domanda principale è: ciò che guardo è dato dal caso oppure è statisticamente effettiva, ovvero che si verifica anche per altri campioni. Si basa fortemente sulle ipotesi fatte sui dati e utilizza strumenti visti prima come: valore p, intervalli di confidenza e test di ipotesi. L'interpretabilità è il punto fondamentale, infatti, un modello non interpretabile è inutile.

Approccio Machine Learning "BLACK BOX"

L'approccio **black box** si basa interamente su previsioni e accuratezza. Spesso si tratta il modello come una scatola nera, dove ci è utile sapere solo la risposta e non come ci si è arrivati.
L'accuratezza è fondamentale nelle previsioni dei dati. Per ottenere le previsioni, si fa una suddivisione dei dati in set di addestramento e simula uno scenario futuro. Il modello si basa sulle prestazioni dei test, analizzando accuratezza e tasso di errore. L'interpretabilità è sono un vantaggio e non un requisito, infatti, si preferisce un modello accurato e complesso piuttosto che semplice e non accurato.

Ad esempio, uno studioso di geopolitica potrebbe seguire obiettivi diversi:

- **Approccio statistico**: se deve studiare i rischi di crisi internazionali, analizza i fenomeni passati e trova un pattern per dire con certezza perché sono causate. Ad esempio, gli interessi sulle materie prime potrebbe essere una delle cause.
- **Approccio machine learning**: se si devono anticipare le prossime guerre, si può creare un modello predittivo basato su dati storici(spesa in sicurezza, alleanze, cultura, ecc.) per stimare una possibile guerra tra i due stati analizzati. Non si spiega il perché, ma è utile a predire dove e quando.

Il contrasto tra questi due modelli evidenza una sfida pratica: compromesso tra interpretabilità e complessità nel modello. In genere più un modello aumenta di complessità(diventa più utile e accurato nel fare predizioni) diminuisce di interpretabilità(si capisce meno il perché di una decisione).
Si hanno quindi:

- **Modelli semplici**(complessità bassa): regressione lineare -> potrebbero essere talmente semplici da avere bias alti e non saper catturare interazioni complesse;
- **Modelli complessi**(complessità alta): alberi decisionali, reti neurali -> può catturare bias piccoli meno comprensibili all'uomo.

La scelta del modello, quindi, è vincolata da questo compromesso(trade-off complessità-interpretabilità).
Nel machine learning si hanno due tipi di apprendimento:

- **Supervisionato**: Il modello impara dai dati etichettati, ovvero per ogni supervisione abbiamo:

- le variabili di input o predittori -> X
- il valore corretto da predire -> variabile di risposta -> Y

l'obiettivo è imparare la relazione che sussiste tra le variabili X e Y, cioè la funzione $\boldsymbol{h}(\boldsymbol{X}) \approx \boldsymbol{Y}$, in modo che il modello possa predire Y per nuovi dati.

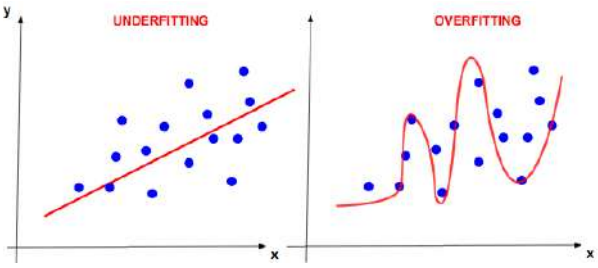
- $\hat{\boldsymbol{y}}$ è il **valore predetto** dal modello;
- **y** è il valore di riferimento chiamato **ground truth**.

- **Non supervisionato**: apprendimento senza una risposta corretta da predire, si conoscono solo le caratteristiche di input. L'obiettivo è quindi quello di trovare strutture, schemi e raggruppamenti nascosti all'interno dei dati.

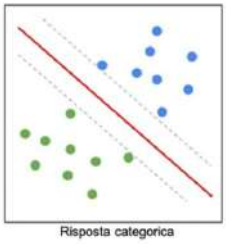
Apprendimento supervisionato

L'apprendimento supervisionato si divide in:

- **Regressione(predire numeri)**: in un problema di regressione si cerca di predire il valore di una variabile numerica continua -> cerca una funzione matematica che si adatta al meglio ai dati osservati -> cerca quindi di ridurre la differenza tra i valori predetti e i valori osservati. Il concetto nasce da Galton, con la regressione verso la media. Questo strumento è diventato utile a capire le relazioni tra variabili e predire dati futuri.



- **Classificazione(predire categorie)**: in un problema di classificazione si cerca di predire una categoria o l'etichetta discreta. L'obiettivo, quindi, è trovare un limite o confine che separi al meglio i diversi gruppi etichettati.



Apprendimento non supervisionato

L'apprendimento non supervisionato si divide in:

- **Clustering (ricerca di gruppi)**-> l'attività consiste nel raggruppare automaticamente punti simili in cluster, a seconda delle caratteristiche. L'obiettivo del clustering è di trovare gruppi nascosti che non hanno etichette preesistenti. Ad esempio, avendo in un'azienda potremmo suddividere in cluster i clienti, nel seguente modo:

- **Riduzione della dimensionalità** -> processo con cui si riduce il numero di caratteristiche all'interno di un dataset cercando di conservare il più possibile le informazioni originali. Verranno quindi escluse: variabili ridondati, correlate e che quindi possono essere descritte da altre caratteristiche. Questo permette di visualizzare dataset complessi in grafici 2d o 3d e velocizza altri modelli di machine learning. Un esempio visto in data mining è la PCA(principal component analysis)
- **Rilevamento delle anomalie** -> consiste nell'identificare i punti rari molto diversi rispetto alla maggior parte dei dati. È non supervisionato perché non esiste un'etichetta per i dati anomali.

Si può effettuare una classificazione dei modelli anche a seconda del modo in cui gestiscono la propria complessità, ovvero se il numero di parametri è fisso o variabile.

Modelli a complessità fissa – Modelli parametrici

Un modello parametrico assumo una forma predefinita per la funzione che descrive i dati e ha un numero fisso di parametri, indipendentemente dai dati.

- imposta la struttura prima di vedere i dati;
- apprende trovando i valori migliori per questi parametri.

Esempi di modelli parametrici:

- **Regressione lineare**: si ha la seguente equazione $Y = \beta_0 + \beta_1 X_1 + \ldots + \beta_n X_n$
  il modello impara solo i coefficienti β_i, con i numero fisso anche al variare della quantità dei dati;
- **Regressione logistica**: simile alla regressione lineare ma ritorna un valore di probabilità tra 0 e 1;
- **Naive Bayes**: impara un insieme fisso di probabilità associate a ciascuna feature.

Questi modelli sono semplici hanno i vantaggi e gli svantaggi dei modelli con approccio statistico.

Modelli a complessità flessibile – Modelli non parametrici

Un modello non parametrico fa poche o nessuna assunzione sulla forma della funzione.

- il numero di parametri cresce al crescere dei dati
- il modello si adatta ai dati

Un esempio di modello non parametrico:

- **K-Nearest-Neighbors(K-NN)**: si predicono dei punti a partire dai K punti vicini del dataset, quindi, se si raddoppiano i dati da cercare si raddoppia la memoria necessaria per memorizzare e usare i dati.

I vantaggi sono dati dall'accuratezza e dalla flessibilità, ma sono lenti da addestrare e meno interpretabili.

La scelta dei due modelli è fortemente legata al **trade-off bias-varianza**, infatti:

- si può avere un modello con bias alto e varianza bassa, che cerca in modo grossolano di passare in tutti i punti;
- si può avere un modello con bias basso e varianza alta, che passa molto bene in tutti i punti di training ma non riesce ad adattarsi a punti nuovi.

Osserviamo come funziona l'apprendimento supervisionato in questo caso, visto che i concetti di apprendimento ed errore di supervisione sono definiti in modo formale.

Si considerano $\mathbf{X}$ e$\mathbf{Y}$ **due spazi di oggetti** che rappresentano gli **spazi campionari** di due variabili casuali X e Y. Le realizzazioni di queste variabili sono indicate come x € X e y € Y.
L'obiettivo di un modello predittivo è trovare una funzione h tale che:

$$\boldsymbol{h}: \mathbf{X} \rightarrow \mathbf{Y}$$

il quale ritorna un output $\hat{\boldsymbol{y}}$ dato un oggetto x, dove $\hat{\boldsymbol{y}}$ **rappresenta il valore predetto** e **y il valore vero**.
Possiamo quindi analizzare un esempio come il riconoscimento di e-mail spam. Se l'e-mail contiene più errori allora verrà classificata come spam, altrimenti no.
Il modello potrebbe essere definito come segue:

$$h(x) = \begin{cases} 1\ se\ f(x) > 0(spam) \\ 0\ altrimenti(non\ spam) \end{cases}$$

Nell'apprendimento automatico, quasi mai si ha una funzione così esplicita, bensì si utilizza un **modello parametrico**, il cui processo di apprendimento tenta di trovare il valore migliore.
L'insieme di tutte le funzioni h che possiamo ottenere cambiando i parametri, si chiama **spazio delle ipotesi** $\mathbf{H}$.
La domanda che sorge è, abbiamo trovato i parametri migliori?
Quando si vuole trovare la migliore funzione h all'interno dello spazio delle ipotesi H, dobbiamo capire quanto è buona una funzione.
Utilizziamo dei parametri, quali:

- **L($\hat{\boldsymbol{y}}, \boldsymbol{y}$), funzione di perdita** che serve a misurare l'errore tra la predizione e il valore reale. Più grande è il valore e maggiore è la predilezione.
- **R(h) =** $\boldsymbol{E}_{(x,y)\sim P(X,Y)}[\boldsymbol{L}(\boldsymbol{h}(\boldsymbol{x}), \boldsymbol{y})]$ è la perdita media attesa su tutti i dati i dati futuri. Si calcola come il valore medio atteso rispetto alla distribuzione vera dei dati P(X,Y).

L'apprendimento statistico deve quindi:

$$\boldsymbol{h}^* = \boldsymbol{argmin R}(\boldsymbol{h}) \qquad \textbf{con h € H}$$

ovvero dobbiamo trovare la funzione che minimizza il rischio e quindi che sbaglia meno in media.
Il problema è che non possiamo sempre calcolare il rischio, dato che alcune volte non si conosce la distribuzione.
In ogni caso, abbiamo a disposizione di un dataset chiamato **Training Set (TR)**, la cui distribuzione possiamo assumere che sia un campione rappresentativo di P(X,Y).
Possiamo riscrivere la formula del rischio nel seguente modo:

$$\boldsymbol{R}_{emp}(\boldsymbol{h}) = \frac{\mathbf{1}}{\boldsymbol{N}} \sum_{\boldsymbol{i}=\mathbf{1}}^{\boldsymbol{n}} \boldsymbol{L}(\boldsymbol{h}(\boldsymbol{x}_i), \boldsymbol{y})$$

Questa formula è chiamata **Empirical Risk(rischio empirico)** e riconduce ad un principio molto importante nel machine learning, ovvero l'**Empirical Risk Minimizzation**(minimizzazione rischio empirico). Dato che non possiamo conoscere h, dobbiamo quindi trovare una funzione che minimizzi il rischio empirico.

$$\hat{\boldsymbol{h}} = \boldsymbol{argmin R}_{emp}(\boldsymbol{h}) \qquad \textbf{con h € H}$$

Del modello dobbiamo anche analizzare la **capacità**, ovvero l'abilità di adattarsi a funzioni complesse. Un modello è flessibile o complesso quando lo spazio delle ipotesi H è molto ricco.
Si hanno quindi due tipi di modelli:

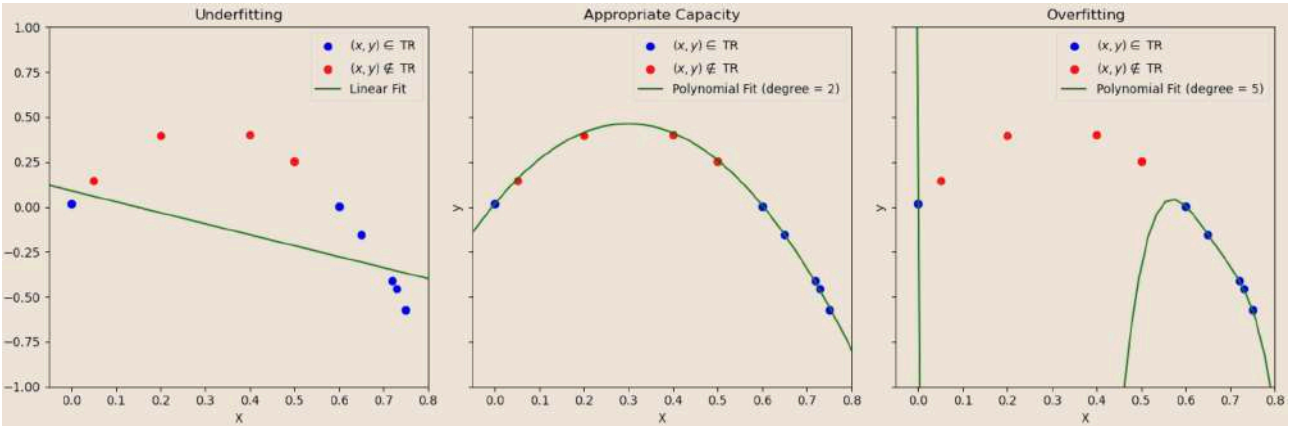
- **Modello a bassa capacità**: ossia un modello con lo spazio delle ipotesi povero, che contiene solo funzioni semplici -> esempio: $\beta_0 + \beta_1 X$(funzione lineare) -> attraverso funzioni così semplici non si riesce a catturare il vero schema dei dati -> **underfitting**.

- **Modello ad alta capacità**: ossia un modello con lo spazio delle ipotesi ampio, che contiene funzioni molto articolate -> ad esempio un polinomio di grado 10 -> rischia di adattarsi anche al rumore dato che è molto flessibile -> **overfitting**.

Quando applichiamo un modello, non sappiamo la qualità di adattamento sulla popolazione.
Possiamo però applicare delle metriche di valutazione per valutare il modello sul training set, e sulla base di ciò stabilire quanto un modello sia meglio rispetto ad un altro.
Per la regressione lineare potremmo utilizzare delle metriche come l'MSE, dove i dati presenti hanno tutti valori numerici. La formula dell'MSE o errore quadratico medio è la seguente:

$$\boldsymbol{MSE} = \frac{\mathbf{1}}{\boldsymbol{N}} \sum_{\boldsymbol{i}=\mathbf{1}}^{\boldsymbol{n}} (\boldsymbol{y} - \boldsymbol{h}(\boldsymbol{x}))^{\mathbf{2}}$$

Se l'MSE è basso, vuol dire che i valori previsti sono uguali o simili ai valori reali, altrimenti il contrario.
Non necessariamente però un MSE basso è indice di un buon modello, dato che se fa overfitting potrebbe adattarsi al rumore e non adattarsi a nuovi dati.



Anche quando analizziamo il nostro modello, dobbiamo tener conto del tradeoff bias-varianza.
Possiamo scomporre l'errore totale atteso in questo modo:

$$\boldsymbol{E}[\boldsymbol{y_0} - \hat{\boldsymbol{f}}(\boldsymbol{x_0})^{\mathbf{2}}] = [\boldsymbol{Bias}(\hat{\boldsymbol{f}}(\boldsymbol{x_0}))]^{\mathbf{2}} + \boldsymbol{var}(\hat{\boldsymbol{f}}(\boldsymbol{x_0})) + \boldsymbol{var}(\boldsymbol{\varepsilon})$$

Dove:

- Bias misura quanto il modello si discosta sistematicamente dal valore reale.
- Un bias elevato indica che il modello è troppo rigido (fa ipotesi troppo semplici) e non cattura la complessità dei dati -> **underfitting**.
- Varianza misura quanto le previsioni del modello variano al variare del training set.
- Una varianza elevata indica che il modello è troppo flessibile e si adatta anche al rumore dei dati -> **overfitting**.
- Var(ε) rappresenta l'errore irriducibile, cioè il rumore intrinseco dei dati che nessun modello può eliminare.

La varianza non si eleva al quadrato perché è già una misura quadratica.
Il compromesso consiste nel trovare un modello con un bias sufficientemente basso ma una varianza non eccessiva, in modo da minimizzare l'errore totale atteso.
Per controllare la capacità di un modello, dobbiamo distinguere due tipi di parametri:

- **I parametri del modello**: ovvero i valori che il modello impara dai dati durante il processo di training. Sono interni al modello e sono gli output dell'algoritmo di apprendimento.
- **Iper-parametri**: sono dei parametri che vengono utilizzati per configurare l'algoritmo di apprendimento. Sono parametri esterni che possono rendere più complesso un modello o viceversa.

Il vero scopo non è ottenere buone prestazioni sui dati del training set, ma generalizzare su dati nuovi mai visti(Golden Rule). L'attività di scelta del modello ottimale è chiamata **model selection** e scegliere significa trovare un equilibrio tra semplicità e flessibilità, tra interpretabilità e complessità.
La model selection è anche fortemente influenzata dai due approcci: **statistico** e **predittivo**.
Se seguissimo l'approccio statistico, allora ci serviremmo di metrica come:

- **p-value**: per testare la significatività delle variabili;
- **adjusted R^2**: per bilanciare bontà di adattamento e semplicità.

Se seguissimo l'approccio predittivo, dovremmo considerare:

- non possiamo sempre utilizzare le loss function(MSE, Empirical Risk), perché si utilizzano in fase di training, dato che servono all'ottimizzazione e non possono non essere utili per il nostro obiettivo, quindi, migliorare l'accuratezza, ridurre l'errore medio ecc. ;
- Una formalizzazione è questa:
    - $\boldsymbol{p}: \boldsymbol{Y^N X\, Y^N} \rightarrow \boldsymbol{R}$ dove si prende un insieme di valori reali, le predizioni del modello e si restituisce un numero reale che rappresenta la qualità del livello

Per verificare se il nostro modello è in grado di generalizzare, ossia è in grado di fare previsioni corrette anche su dati nuovi, si effettua la **validazione**.
Si effettua la validazione per capire se il modello ha effettivamente capito il fenomeno o ha solo memorizzato i dati di training.
L'**holdout validation** la forma più semplice di validazione in machine learning. Consiste nel dividere il dataset in due parti:

- **Training set**: usato per allenare il modello;
- **Test set**: usato per valutare il modello;

Si allena il modello solo sul training set, ad esempio, con Empirical Risk Minimization e poi si testa sul test set, che vede i dati al suo interno come dati mai visti.
Questo serve a simulare la perfomance futura del modello, però se il dataset è piccolo, la valutazione può essere distorta.
Un altro tipo di validazione è il **K-fold Cross Validation**, dove il dataset viene divisa in k parti uguali e si fanno k esperimenti:

- Ogni volta si usa 1 fold come test e gli altri k-1 come training;
- Si media la perfomance su tutti i test.

In questo caso si possono utilizzare i dati sia per allenare che per testare, quindi, lo possiamo applicare anche a dataset più piccoli.
Un caso estremo di K-fold è il **Leave-One-Out Cross Validation** dove K = N(dove N è il numero di osservazioni), cioè ogni fold contiene un solo esempio.
Per ogni osservazione del dataset:

- si lascia fuori e si userà come test;
- si utilizzano le altre n-1 osservazioni per allenare il modello;
- si valuta il modello su quell'unica osservazione lasciata fuori.

Si ripete per tutte le osservazioni, per questo motivo è efficiente con dataset piccoli e quando serve una valutazione accurata.
Quando abbiamo gli iper-parametri per la regolarizzazione, non si possono scegliere basandoci sul test set, dato che esso viene utilizzato per valutare le prestazioni finali, e così è come barare.
Per questo problema si utilizzano tre insiemi di dati distinti, ovvero:

1. **Training set**, che serve per addestrare il modello e far apprendere i parametri interni;
2. Validation set, che serve per scegliere gli iper-parametri migliori e si scelgono i migliori per il validation set;
3. Test set, che serve per valutare il modello finale, dopo aver scelto gli iper-parametri finali e addestrato il modello definitivo.

Regressione Lineare

La **regressione lineare** è un potente strumento per modellare e comprendere le relazioni tra variabili e fare previsioni sui dati non osservati. Essa mira a modellare una **relazione lineare** tra una **variabile dipendente** e una o più **variabili indipendenti**.
I modelli di regressione mirano a studiare la relazione tra due variabili X e Y, definendo un modello matematico f tale che:

$$Y = f(x) + \epsilon$$

Dove:

- **f** è una funzione deterministica che viene utilizzata per prevedere i valori di Y dai valori di X -> non è la funzione vera, ma solo la stima;
- $\boldsymbol{\epsilon}$ è un termine di errore, che cattura ciò che non sono stati catturati dalla funzione;
- **Y** non può essere interamente previsto da **X**.

Il **problema ha una stocasticità intrinseca** che non può essere modellata all'interno della funzione deterministica f.
Si costruiscono i modelli per avere una forma analitica esplicita delle relazioni tra le variabili, ovvero che si conosce un'equazione matematica precisa.
La regressione lineare mira a modellare una relazione lineare tra due variabili X e Y. Possiamo quindi immaginare una retta che attraversa i dati, la cui equazione è la seguente:

$$\boldsymbol{Y \approx \beta_0 + \beta_1 X}$$

Spesso si introduce una variabile "rumore " che cattura la casualità. Possiamo togliere quindi l'approssimazione nel seguente modo:

$$\boldsymbol{Y = \beta_0 + \beta_1 X + \varepsilon}$$

Dove:

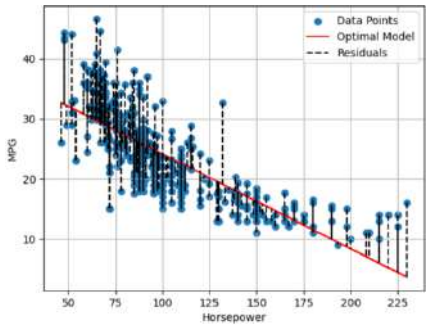
- **X** è la variabile indipendente o regressore;
- **Y** è la variabile dipendente o variabile regredita;
- $\boldsymbol{\beta_0, \beta_1}$ sono chiamati coefficienti o parametri del modello.

I parametri dei dati devono essere appropriati perché forniscono un modello che li rappresenta bene. I parametri influenzano l'inclinazione e la crescita della linea, chiamata **linea di rigressione**.
Per verificare se il nostro modello è un buon modello, deve essere verificata la condizione che da X si prevede bene Y. Sappiamo già che la relazione non è perfettamente lineare come il modello, quindi, possiamo calcolare gli errori di ogni previsione come segue:

$$\boldsymbol{e_i = y_i - \widehat{y_i}}$$

Ciò che calcoliamo sono i residui positivi o negativi e come indicatore globale utilizzeremo la somma residua dei quadrati(RSS), come segue:

$$\boldsymbol{RSS = e_1^2 + e_2^2 + \ldots + e_n^2}$$



La figura a sinistra rappresenta una linea di rigressione che passa per dei punti, corrispondenti ai dati osservati. Le parti tratteggiate rappresentano i **residui** e si possono anche calcolare attraverso la seguente formula:

$$\boldsymbol{RSS = (y_1 - \beta_0 - \beta_1 x_1 + \varepsilon)^2 + \ldots + (y_n - \beta_0 - \beta_1 x_n + \varepsilon)^2}$$

Se questi valori sono minimizzati, la retta si adatta meglio ai dati.

Però in poche parole vogliamo trovare:

($\widehat{\beta}_0\widehat{\beta}_1$) = argmin RSS($\beta_0\beta_1$) ovvero intercetta e coefficiente angolare che generanno la retta la cui somma dei residui è minima

$\frac{\partial RSS}{\partial \beta_0} = 0$ $\frac{\partial RSS}{\partial \beta_1} = 0$ attraverso il parziale fissiamo uno dei due coefficienti

stiamo cercando i valori di β_0e $\beta_1$che minimizzano la funzione RSS, cioè la somma dei quadrati degli errori tra i valori osservati y_ie quelli stimati $\widehat{y}_i = \beta_0 + \beta_1 x_i$.

Si può dimostrare attraverso i calcoli che:

$\widehat{\beta}_1 = \frac{\sum_{i=1}^{n}(x_1-\overline{x})(y_1-\overline{y})}{\sum_{i=1}^{n}(x_1-\overline{x})^2} = \frac{Cov(X,Y)}{Var(X)}$ misuriamo come y cambia al variare di x (inclinazione della linea)

- Se $\hat{\beta}_1 = 0.5$ e se x = 5 allora y = 2.5
- Se $\hat{\beta}_1 = 2$ e se x = 5 allora y = 10

$\widehat{\beta}_0 = \overline{y} - \widehat{\beta}_1\overline{x}$ è l'intercetta della regressione, ovvero il punto y quando x è 0.

Possiamo anche utilizzare il la minimizzazione del rischio empirico(ERM) allo stesso modo della RSS. Infatti, utilizzando la lost function abbiamo che:

L(h(x_i)-y_i) = (h(x_i)-y_i)^2 = e_i^2

Possiamo definire il rischio empirico come segue:

$$R_{emp} = \frac{1}{N}\sum_{i=1}^{n}(h(x_i) - y_i)^2 = \frac{1}{N}RSS$$

Quindi:

$$\widehat{h} = argmin\, R_{emp}(h) = argmin\, \frac{1}{N}\sum_{i=1}^{n}(h(x_i) - y_i)^2 = argmin\, \frac{1}{N}RSS = argmin\, RSS$$

Si toglie $\frac{1}{N}$ in RSS perché diventa solo un fattore di scala. Quindi da un punto di vista del machine learning il **problema si riduce a trovare l'insieme ottimale dei parametri β minimizzando il rischio empirico**, che corrisponde alla minimizzazione della somma residua dei quadrati.

Quando noi calcoliamo la retta di regressione per un singolo campione, potremmo incorrere in dati imprecisi. Calcolando la media di tutti i campioni, la retta di regressione ottenuta dalla media si avvicina di più alla retta di regressione della popolazione.

Si possono considerare le formule sopra come degli estimatori, e per questo motivo possiamo effettuare test di ipotesi sui coefficienti. Attraverso un test statistico possiamo verificare se β_1è diverso da 0. Se non fosse così allora avremmo non associazione tra le due variabili.

Potremmo quindi avere le seguenti ipotesi:

$$H_0 = Non\ c'è\ associazione\ tra\ le\ due\ variabili\ X\ e\ Y \iff \beta_1 = 0 \rightarrow ipotesi\ nulla$$
$$H_1 = C'è\ associazione\ tra\ le\ due\ variabili\ X\ e\ Y \iff \beta_1 \neq 0 \rightarrow ipotesi\ alternativa$$

Si calcolano le statistiche t e si verifica se p >α o viceversa.

I test di ipotesi però ci dicono soltanto se c'è una relazione statistica significativa, ma non ci dicono quanto bene il modello si adatti ai dati o accurate siano le sue previsioni.

Per misurare le prestazioni, utilizziamo diverse metriche utilizzate nelle due diverse categorie, ovvero:

- **metriche per la comprensione** -> misura quanto bene il modello spiega bene i dati per cui è stato addestrato -> **RSE**;
- **metriche per la previsione** -> misura quanto bene il modello riesce a predire i dati -> **MSE**;

Delle stime statistiche utilizzate come metriche per la comprensione sono:

- **Residual Standard Error RSE**-> è la stima statistica della deviazione standard dell'errore irriducibile.

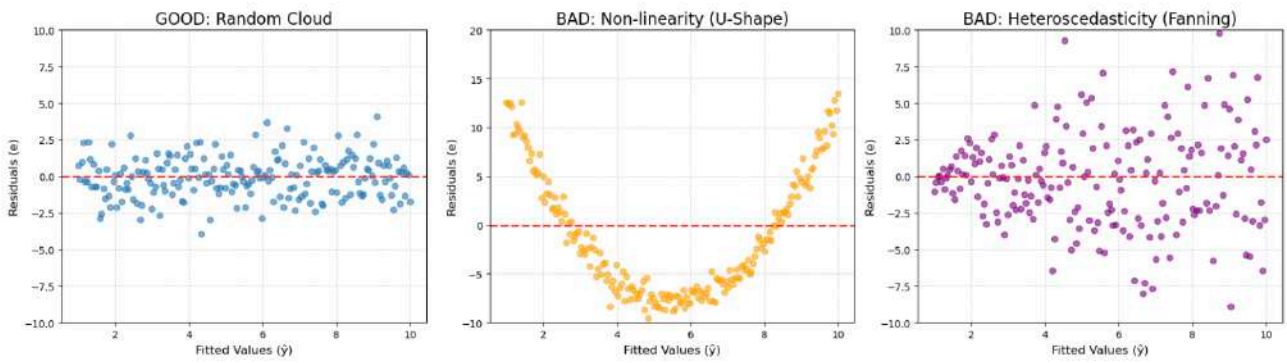
$$RSE = \sqrt{\frac{RSS}{n-2}} = \sqrt{\frac{1}{n-2}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

dove si sottraggono due gradi di libertà che sono dati dai parametri($\hat{\beta}_0\hat{\beta}_1$)(ogni stima che si impone nelle formule corrisponde ad un grado di libertà sottratto). L'RSE è una misura assoluta della mancanza di adattamento del modello, più alto è più il modello non si adatta.

- $\boldsymbol{Statistica\ \mathcal{R}^2}$-> detta anche coefficiente di determinazione è una misura che dice quanto la retta si adatta bene ai dati. In valore non è una misura assoluta bensì relativa e spiega la variabilità dei dati.
  TSS = $\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$**-> total sum squares(baseline)** confronta quanto i dati y variano intorno alla loro media senza che x influenzi la stima;
  RSS misura la varianza dopo la regressione(x influenza y)
  Allora abbiamo che:
  $$\mathcal{R}^2 = \frac{TSS - RSS}{TSS} = 1 - \frac{RSS}{TSS}$$
  Se $\mathcal{R}^2 \approx 0$ il nostro modello non è migliore di una semplice ipotesi sulla media;
  Se $\mathcal{R}^2 \approx 1$ il nostro modello spiega la varianza della variabile.
  Per una regressione lineare $\mathcal{R}^2 = p(x,y)^2$ è il coefficiente di correlazione

Queste metriche attraverso un singolo numero riassumono un livello di errore, tuttavia non indicano dove sta l'errore.
Per verificare se queste metriche non hanno semplicemente restituito un buon parametro e comunque il modello è imperfetto, possiamo utilizzare il **grafico dei residui e il grafico adattato** (trovato attraverso la regressione).
Quello che ci aspettiamo di vedere è una forma rettangolare di punti rumorosa senza pattern, attorno alla linea 0. Ciò significa che il modello ha catturato il trend principale, e gli errori rimanenti sono rumore casuale.
I problemi legati al modello nascono quando:

- Si vede una curva tipo U o un altro pattern(non una retta), e significa che la relazione X e Y non è lineare(utilizzare regressione polinomiale).
- Si vede una forma a ventaglio o a cono, dovuta dal fenomeno di eteroschedasticità, ovvero che i residui aumentano di dispersione all'aumentare dei valori predetti. La varianza degli errori non è costante, ma variabile a seconda della posizione di X.



Possiamo anche verificare se i residui sono distribuiti normalmente attraverso i Q-Q plot.
Quando un modello non riesce a spiegare completamente la varianza di una feature, le cause potrebbero essere:

- Stocasticità che impedisce di apprendere una funzione accurata per prevedere Y da X;
- La relazione tra X e Y non è lineare e non si può prevedere Y accuratamente;
- La previsione di Y dipende anche da altre variabili.

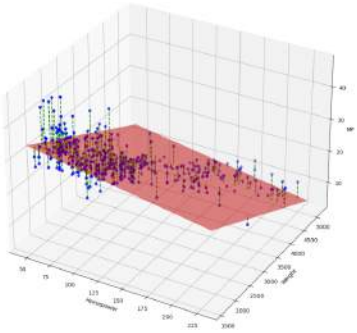
Il terzo punto è comune e facile da correggere. Supponendo di lavorare con un dataset che contiene informazioni relative alle macchine, allora possiamo vedere come alcune variabili non dipendono interamente da una sola feature, ma da un insieme di esse.
Ad esempio:

- una feature che contiene il valore di metri per litro potrebbe essere caratterizzata da:
  - cavalli;
  - peso;
  - potenza;
- una feature potenza potrebbe dipendere da:
  - cavalli;

Possiamo quindi trovare un modello che utilizzi più feature per prevedere i valori di una colonna. Questo si ottiene aggiungendo un coefficiente aggiuntivo per la nuova variabile:

$$\boldsymbol{Y = \beta_0 + \beta_1 X_1 + \ldots + \beta_n X_n}$$

Il modello ottenuto è chiamato **regressione lineare multipla** e serve appunto ad individuare la relazione lineare tra una variabile dipendente Y e più variabili indipendenti $X_1,\ldots,X_n$.
Il modello di regressione multipla basato su due variabili ha un'interpretazione geometrica. Infatti, l'equazione: $\boldsymbol{Z = \beta_0 + \beta_1 X + \beta_2 Y}$ identifica un piano nello spazio 3D.



Come nel caso precedente le parti tratteggiate sono i residui e il miglior adattamento del modello cerca di minimizzare nuovamente la somma dei quadrati dei residui.
Questo modello effettua previsioni selezionando Z, ossia il valore che interseca il piano per valori di X e Y.
In generale quando consideriamo n variabili, il regressore lineare sarà un iperpiano(n-1)-dimensionale nello spazio n-dimensionale.
L'interpretazione statistica di un modello di regressione lineare multipla:

$$\boldsymbol{Y = \beta_0 + \beta_1 X_1 + \ldots + \beta_n X_n}$$

si può interpretare come quando il regressore è lineare semplice. Abbiamo che:

- $\boldsymbol{\beta_0}$ è il valore che assume la variabile, quando tutte le variabili indipendenti sono poste a 0;
- $\boldsymbol{\beta_1}$ è il valore di incremento di Y quando X_i incrementa di un'unità, ponendo tutto il resto come costanti.

Per stimare i coefficienti possiamo utilizzare nuovamente la funzione di costo come la somma residua dei quadrati:

$$\boldsymbol{RSS(\beta_0,\ldots,\beta_n) = \sum_{i=1}^{m} (y_i - (\beta_0 + \beta_1 x_1^{(i)} + \ldots + \beta_n x_n^{(i)}))^2}$$

$\boldsymbol{\widehat{\beta_0},\ldots,\widehat{\beta_1} = arg\ min\ RSS(\beta_0,\ldots,\beta_n)}$ sono le stime multiple dei coefficienti che minimizzano la funzione di perdita.
Per trovare questi valori ottimali, si può utilizzare la notazione matriciale. Quindi date m osservazioni, avremmo m equazioni:

$$\boldsymbol{y^{(1)} = \beta_0 + \beta_1 x_1^{(1)} + \ldots + \beta_n x_n^{(1)} + e^{(1)}}$$

$$\boldsymbol{y^{(2)} = \beta_0 + \beta_1 x_1^{(2)} + \ldots + \beta_n x_n^{(2)} + e^{(2)}}$$

$$\ldots$$

$$\boldsymbol{y^{(m)} = \beta_0 + \beta_1 x_1^{(m)} + \ldots + \beta_n x_n^{(m)} + e^{(m)}}$$

Che possiamo riscrivere in forma matriciale come segue:

$$y = X\beta + e$$

$$\mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix}, \mathbf{X} = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} & \ldots & x_n^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} & \ldots & x_n^{(2)} \\ \vdots & \vdots & \vdots & & \\ 1 & x_1^{(m)} & x_2^{(m)} & \ldots & x_n^{(m)} \end{bmatrix}, \beta = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_n \end{bmatrix}, \mathbf{e} = \begin{bmatrix} e^{(1)} \\ e^{(2)} \\ \vdots \\ e^{(m)} \end{bmatrix}$$

Dove: X è la matrice dei dati(matrice di progettazione), βè la matrice dei coefficienti da stimare ed e è il vettore degli errori detti residui.

La formula per trovare $\hat{\beta}$coefficienti ottimali è:

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

Questa formula non sempre si può applicare dato che:

- Alcune colonne di X sono linearmente dipendenti da altre, cioè questa si ottiene attraverso una combinazione lineare. Il determinante sarà uguale a 0 e non sarà invertibile.
- Ci sono troppe variabili rispetto le osservazioni;
- Dati duplicati o costanti -> può causare problemi perché non aggiunge informazione.

Test F

Quando si adatta un regressore lineare multiplo, è comune eseguire un test statistico che serve per verificare se almeno uno dei regressori è diverso da 0 (nella popolazione).

Questo test è chiamato **F-test** ed è definito nel seguente modo:

- **H_0:** $\boldsymbol{\beta_1 = \ldots = \beta_n = 0}$
- $\boldsymbol{H_\alpha: \exists j \text{ tale che } \beta_j \neq 0}$

L'ipotesti nulla è che tutti i coefficienti siano nulli nella popolazione. Se ciò è vero, il regressore nullo non è affidabile e lo dovremmo scartare. L'ipotesti alternativa è che almeno uno di questi coefficienti sia diverso da 0.

Come nei test d'ipotesi fatti precedentemente:

- Se il p-value è piccolo allora l'ipotesi nulla si scarta, quindi, se p$\leq \alpha$;
- Se il p-value è grande allora si accetta l'ipotesi nulla, quindi, se se p$\geq \alpha$;

Quando abbiamo più variabili indipendenti che condizionano una variabile dipendente, potremmo scartare quelle con relazioni scadenti, perché le stime degli altri coefficienti potrebbe risultare alterate.

Un metodo per rimuovere queste variabili è attraverso la **selezione a ritroso** o **eliminazione a ritroso**, togliendo quelle che condividono il p-value più alto.

Inoltre, possiamo anche osservare che alcune volte in una regressione lineare semplice una feature è significativa, dato che aiuta a spiegare un'altra feature, ma può non esserlo in una regressione multipla. Questo fenomeno è il sintomo di **collinearità** tra i predittori.

La collinearità si verifica quando due variabili sono fortemente correlate tra loro.

Quando un predittore può essere approssimato da una combinazione lineare di altri predittori, allora si parla di **multicollinearità**.
Quando siamo in presenza di questi fenomeni, il fenomeno non riesce a distinguere bene l'effetto di variabili correlate e la matrice utilizzata per calcolare i coefficienti diventa invertibile.
Nel caso della regressione lineare semplice, il coefficiente di determinazione è il quadrato del coefficiente di pearson. Riprendiamo la formula: $\boldsymbol{R^2 = \rho(x, y)^2}$
Nel caso della regressione lineare multipla, invece, si utilizza:

$$\boldsymbol{R^2 = p(Y, \hat{Y})^2}$$

Dove $\hat{Y}$ sono i valori predetti del modello e Y sono i valori reali.
Le variabili categoriche (come "tipo di carburante" o "sistema di alimentazione") vengono trasformate in numeriche usando variabili fittizie (detti anche dummy variables).

- Ogni livello della variabile viene rappresentato da una colonna con valori 0 o 1
- Se ci sono n livelli, si creano n – 1 colonne per evitare ridondanza

Vediamo come funziona la regressione lineare in python.
Importiamo le librerie che gestiscono la regressione lineare singola:

```
import statsmodels.api as sm
import statsmodels.formula.api as smf
```

Per calcolare il modello, basta richiamare la funzione .ols() che indica quale modello applicare e utilizzare .fit() per allenare il modello sui dati a disposizione.

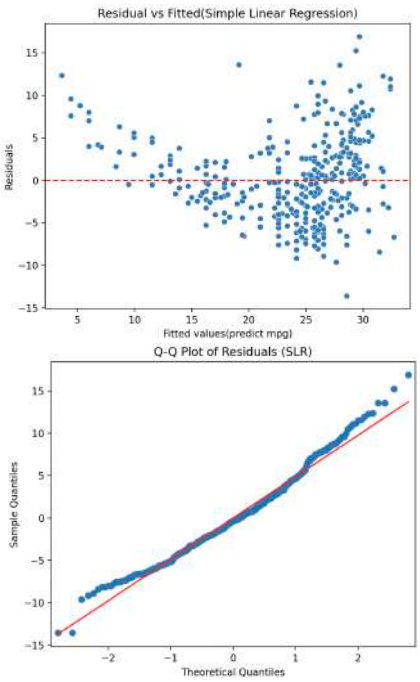
```
model_slr = smf.ols("mpg ~ horsepower",data=mpgclean).fit()
print(model_slr.summary())
```

in questo modo si applica il modello sui dati indicati e attraverso .summary(), vediamo tutte le informazioni parametriche che ci danno informazioni utili sulle due variabili.
Estraiamo infine i dati stimati e i residui, ovvero le differenze tra i dati stimati e quelli reali. Si prosegue nel seguente modo:

```
fittedvalues_slr = model_slr.fittedvalues
residuals_slr = model_slr.resid
```

Possiamo confrontare i valori stimati e i valori reali e come si distribuiscono i valori stimati in questo modo:

```
ax1 = plt.subplot(1,2,1)
sns.scatterplot(x=fittedvalues_slr,y=residuals_slr,ax=ax1)
ax1.axhline(0, color='red',linestyle='--')
ax1.set_title('Residual vs Fitted(Simple Linear Regression)')
ax1.set_xlabel('Fitted values(predict mpg)')
ax1.set_ylabel('Residuals')


ax2 = plt.subplot(1,2,2)
sm.qqplot(residuals_slr,line='s',ax=ax2)
ax2.set_title('Q-Q Plot of Residuals (SLR)')
plt.tight_layout()
plt.show()
```



Abbiamo visto con solo una feature, ma come funziona con molte più feature?

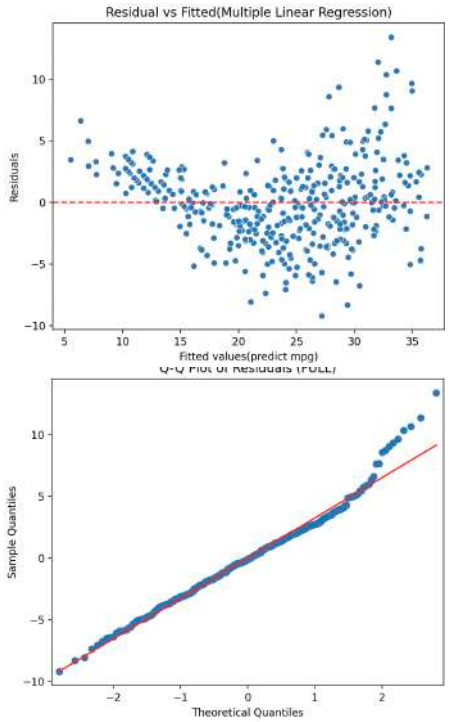
```
allfeature = 'mpg ~ cylinders + displacement + horsepower + weight + acceleration + model_year + C(origin)'
model_full = smf.ols(allfeature,data=mpgclean).fit()
print(model_full.summary())
```

Come per la regressione lineare singola, anche nella regressione lineare multipla possiamo utilizzare la funzione ols e la funzione fit. Si specificano tutte le variabili predittrici e infine si applica la funzione. Attraverso le informazioni parametriche ritornate da summary e le conoscenze pregresse, possiamo stimare quali variabili non sono utili al fine di predirre mpg. Le informazioni restituite sono le seguenti:

	coef	**std err**	**t**	**P>|t|**	**[0.025**	**0.975]**
Intercept	-15.3246	4.602	-3.330	0.001	-24.374	-6.276
C(origin)[T.japan]	0.2232	0.566	0.394	0.694	-0.890	1.336
C(origin)[T.usa]	-2.6300	0.566	-4.643	0.000	-3.744	-1.516
cylinders	-0.4897	0.321	-1.524	0.128	-1.121	0.142
displacement	0.0240	0.008	3.133	0.002	0.009	0.039
horsepower	-0.0182	0.014	-1.326	0.185	-0.045	0.009
weight	-0.0067	0.001	-10.243	0.000	-0.008	-0.005
acceleration	0.0791	0.098	0.805	0.421	-0.114	0.272
model_year	0.7770	0.052	15.005	0.000	0.675	0.879

Possiamo rimuovere le variabili che hanno il p value alto, semplicemente ricalcolando il modello solo sulle variabili di interesse.

```
allfeature = 'mpg ~ cylinders + displacement + horsepower + weight +
model_year + C(origin, Treatment(reference="usa"))'
model_full = smf.ols(allfeature,data=mpgclean).fit()
print(model_full.summary())
fittedvalues_full = model_full.fittedvalues
residuals_full = model_full.resid
plt.figure(figsize=(12,5))
ax1 = plt.subplot(1,2,1)
sns.scatterplot(x=fittedvalues_full,y=residuals_full,ax=ax1)
ax1.axhline(0, color='red',linestyle='--')
ax1.set_title('Residual vs Fitted(Multiple Linear Regression)')
ax1.set_xlabel('Fitted values(predict mpg)')
ax1.set_ylabel('Residuals')


ax2 = plt.subplot(1,2,2)
sm.qqplot(residuals_full,line='s',ax=ax2)
ax2.set_title('Q-Q Plot of Residuals (FULL)')
plt.tight_layout()
plt.show()
```



Oltre la regressione lineare

Osservando la regressione lineare in termini statistici, abbiamo avuto modo di constatare due problemi importanti in questi modelli semplici:

- Le forme tipo a U o a ventaglio dimostra un andamento lineare, cioè che il **modello** è **sotto adattato**(bias elevato), perché la relazione nel mondo reale non è una linea.
- Il fenomeno della **collinearità** e **multicollinearità**, che rende i valori p instabili, confondendo l'interpretazione.

Rimanendo nel mondo statistico proviamo a correggere il modello. Un modello lineare semplice presuppone che l'effetto di ciascun predittore su Y sia **additivo**. Ciò significa che l'effetto di 1 unità di una variabile è costante, indipendentemente da altre variabili.
Se volessimo verificare che l'effetto di una variabile cambia in base ad un'altra variabile dobbiamo aggiungere il **termine di interazione** del modello. Un termine di interazione, in regressione lineare, serve a catturare il caso in cui l'effetto di una variabile dipenda da un'altra.
Supponendo di avere la seguente formula:

$$migliaperlitro = \beta_0 + \beta_1 cavalli + \beta_2 peso$$

se volessimo aggiungere un termine di interazione avremmo la seguente formula:

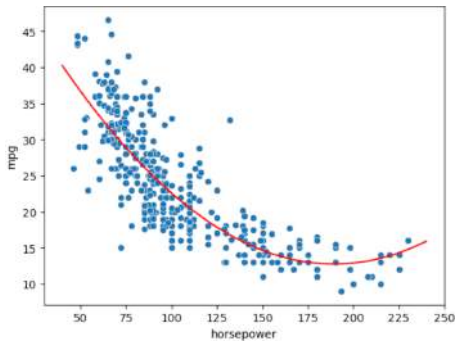
$$migliaperlitro = \beta_0 + \beta_1 cavalli + \beta_2 peso + \beta_3 (cavalli * peso)$$

Se infine il valore R^2 aumenta, allora il modello di interazione è più adatto.

$$migliaperlitro = \beta_0 + (\beta_1 + \beta_3 * peso) * cavalli + \beta_2 peso$$

Così si guadagna più precisione, ma a scapito di una semplice interpretabilità immediata. Passando da un modello del tipo "glass box", proseguendo in questo modo si arriva ad un modello più "black box".
Quando attraverso scatterplot o simili notiamo che la relazione non è lineare, e ad esempio quadratica, possiamo adattare i nostri modelli a relazioni quadratiche come segue:

$$y = \beta_0 + \beta_1 X_1 + \beta_2 X_1^2$$



Il modello rimane lineare perché X_1^2 può essere visto come un parametro normale. Basta assegnarlo ad una variabile Z e possiamo continuare a trovare i coefficienti attraverso il metodo OLS(minimi quadrati lineari).
Il modello diventa non lineare quando i coefficienti sono contenuti in altre funzioni come logaritmi e simili.
Nonostante il modello sia lineare può rappresentare curve(la linearità riguarda solo i coefficienti e non le variabili). In generale, si può adattare un modello polinomiale ai dati, scegliendo un grado adatto, avendo una **regressione polinomiale**. Si possono avere anche termini di interazione nella relazione polinomiale. Così facendo si trasforma la regressione lineare in un modello non lineare, anche se è ancora risolvibile attraverso i minimi quadrati ordinari, perché **non è lineare nella variabile ma nei coefficienti**.

Complicando il nostro modello, per un'accuratezza predittiva maggiore, abbiamo attraversato il confine tra inferenza statistica e machine learning. A discapito dell'interpretabilità abbiamo migliorato la precisione con cui si predicono i dati.
Come visto nei paragrafi precedenti, quando l'obiettivo è la previsione, non ci interessa la perfomance sul set di training ma solo sul set di test. Non sarà più utile quindi calcolare R^2 o RSE.

Le metriche utilizzate per valutare il modello sul set di test sono:

- MSE(ERRORE QUADRATICO MEDIO) -> L'MSE è la media degli errori quadratici sul set di test. È la versione predittiva del rischio empirico. Non è intuitivo dato che gli errori sono al quadrato.

$$\boldsymbol{MSE} = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

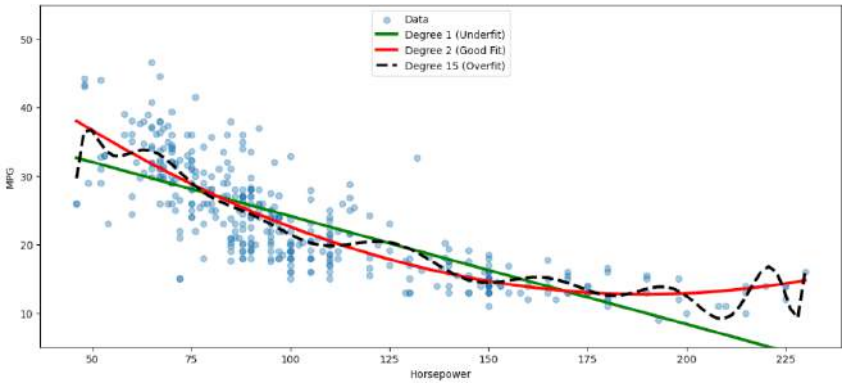
  dove:
  y è il valore del set di test
  $\hat{y}$ è il valore predetto
  Un buon modello ha quindi un piccolo MSE, dato che è una misura di errore.
- RMSE(RADICE ERRORE QUADRATICO MEDIO) -> L'RMSE è la soluzione alla difficile intuitività dell'MSE, ed è la metrica più comune per la regressione predittiva. È la radice quadrata dell'MSE e può essere interpretato come l'errore in media. Gli errori più grandi contribuiscono di più e quindi possiamo capire se ci sono valori troppo distanti.

$$\boldsymbol{RMSE} = \sqrt{\boldsymbol{MSE}} = \sqrt{\frac{1}{m}\sum_{j=1}^{m} (y_j - \hat{y}_j)^2}$$

- MAE(ERRORE MEDIO ASSOLUTO) -> Alternativa al RMSE, esso è altamente interpretabile e non penalizza errori più grandi. Esso può essere interpretato come l'errore medio assoluto e la formula è la seguente:

$$\boldsymbol{MAE} = \frac{1}{m}\sum_{j=1}^{m} |y_j - \hat{y}_j|$$

Attraverso l'utilizzo delle polinomiali si è risolto il problema del sottodimensionamento, ma quanto dovrebbe essere il grado massimo del polinomio. Se un polinomio è un polinomio di grado alto, ha una capacità enorme, e può oscillare per adattarsi a ogni singolo punto.
Questo potrebbe causare overfitting, ovvero la situazione in cui la varianza è alta, e il modello comprende il rumore dei dati di addestramento e non il segnale sottostante.
Sostanzialmente si adatterà ai dati di training ma fallirà su qualsiasi nuovo dato.



Il modello lineare è in underfitting, invece il polinomio di 15 grado è in overfitting. La domanda che dovremmo porci è: come facciamo a scegliere il grado giusto del polinomio?

Regolarizzazione

Un approccio di apprendimento automatico puro, che ci permette di avere una soluzione alla domanda precedente è la **regolarizzazione**.
La regolarizzazione aggiunge una **penalità** alla funzione di costo per impedire che diventi troppo complessa. L'idea che sta alla base è che se si spinge ad avere coefficienti β piccoli, allora forziamo il modello a trovare una curva più semplice e liscia, anche se presenta molte caratteristiche.

$$\boldsymbol{RSS + penalit}\text{à}$$

La regolarizzazione diventa anche una soluzione indiretta della multicollinearità. Infatti, essa stabilisce la matrice, consentendo al modello di trovare coefficienti sensati e stabili anche in presenza di multicollinearità.
La **regressione Rigde** è il tipo di regolarizzazione più comune. Aggiunge una **penalità proporzionale alla somma dei quadrati dei coefficienti**.
Queta è anche chiamata L_2 norma e la funzione di costo diventa:

$$\boldsymbol{RSS_{L2} = \sum_{i=1}^{m} (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^{n} \beta_j^2}$$

Dove:

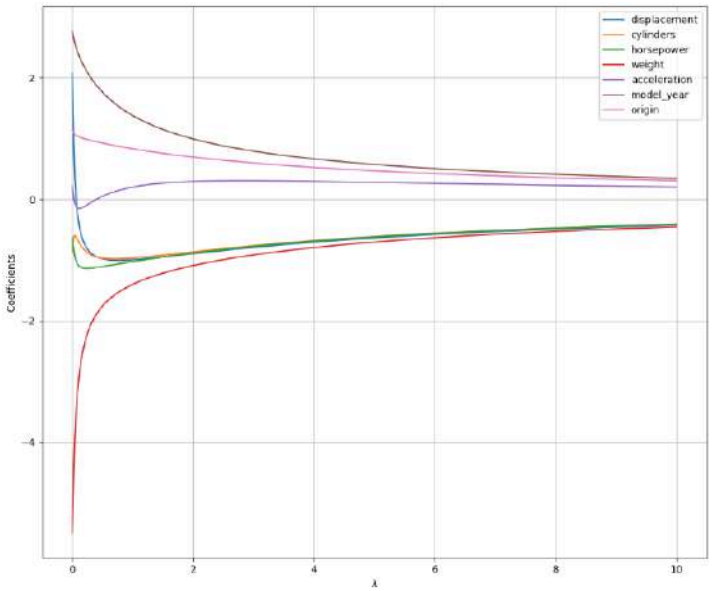
- **RSS** spinge il modello ad attarsi ai dati;
- $\boldsymbol{\lambda \sum_{j=1}^{n} \beta_j^2}$ spinge i coefficienti verso i valori piccoli;
- l'iper-parametro $\boldsymbol{\lambda}$ regola questo compromesso.

$\boldsymbol{\lambda = 0}$ **questo è OLS standard(minimi quadrati ordi nari)**
$\boldsymbol{\lambda \to \infty}$ **la penalità ha tutto il peso dell'RSS.**
La regressione ridge riduce i coefficienti vicino allo 0, ma mai 0. Per applicarla bisogna standardizzare prima le feature(z-score) dato che la feature misurata in kilometri verrà penalizzata diversamente dai millimetri.

Aumentando il valore di λ, i coefficienti vengono gradualmente ridotti verso 0.

Il modello decide quali coefficienti ridurre per un dato valore di λ, in modo che si trovino pesi più piccoli.

Viene ridotta la varianza delle stime dei coefficienti, ma la distorsione(bias) aumenta, quindi, trovare un buon λ permette di controllare il compromesso bias-varianza.



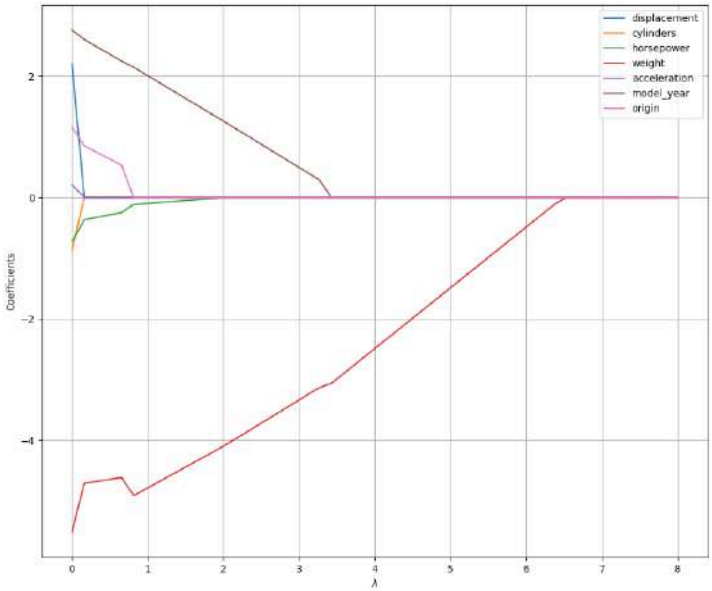
I coefficenti β non possono essere letti come in una regressione lineare standard, ma si può capire quali sono le variabili più influenti per la predizione.

Un'alternativa alla regressione Ridge è la regressione Lazo che ha una proprietà diversa, ma utile. Aggiunge una penalità proporzionale alla somma dei valori assoluti dei coefficienti. Questa regressione è anche chiamata $\boldsymbol{L_1 norma}$ e la funzione di costo diventa:

$$\boldsymbol{RSS_{L1}} = \sum_{i=1}^{m} (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^{n} |\beta_j|$$

Attraverso la funzione del valore assoluto durante la fase di ottimizzazione per trovare le soluzioni, i coefficienti inutili vengono impostati esattamente a 0.
Così:

- si riduce la complessità;
- si migliora l'interpretabilità;
- si può ridurre il dataset, mantenendo solo le colonne con coefficienti diversi da zero.



Possiamo applicare quindi i due tipi di regressione, ricordando però che il valore che le caratterizza è il valore $\boldsymbol{\lambda}$.

Questo iper-parametro λ ci permette di passare da un modello OLS standard a un modello che tiene conto della penalità sui coefficienti.
È importante capire che, regolando λ, stiamo manipolando indirettamente il compromesso tra bias e varianza.
Scegliendo il valore ottimale di λ, otteniamo il modello che minimizza l'errore totale, bilanciando accuratezza e semplicità.
In questo modo, la regolarizzazione ci permette di costruire modelli potenti ma stabili, che generalizzano bene sui dati nuovi senza cadere nell'overfitting.

Per mettere in pratica su python la regressione polinomiale, regolarizzata o non regolarizzata, prima dobbiamo dividere il nostro dataset in:

- train set;
- test set;
- validation set.

Le azioni successive saranno:

- addestramento modello;
- predizione dei dati;
- calcolo misure di valutazione.

Prima parte: divisione del dataset e standardizzazione.

```
from sklearn.datasets import fetch_california_housing

data  = fetch_california_housing()

X = data.data
y = data.target #valore da predire, valore medio di una casa in un quartiere


from sklearn.model_selection import train_test_split
validation_prop = 0.2 #dividiamo in validation set
test_prop = 0.2 #dividiamo in test set
#il resto sarà training set

#SEPARIAMO DA TRAINING SET E VALIDATION SET I VALORI UTILIZZATI PER IL SET DI TEST
X_trainval, X_test, y_trainval, y_test =
train_test_split(X,y,test_size=test_prop,random_state=42)
#SEPARIAMO DA TRAINING SET I VALORI UTILIZZATI PER IL VALIDATION SET
X_train, X_val, y_train, y_val =
train_test_split(X_trainval,y_trainval,test_size=test_prop/(1-
test_prop),random_state=42)


from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

scaler.fit(X_train)#apprende media e deviazione standarddel dataset


X_train = scaler.transform(X_train)#applica lo z-scoring a train set
X_val = scaler.transform(X_val)#applica lo z-scoring a test set
X_test = scaler.transform(X_test)#applica lo z-scoring a validation set
```

Seconda parte: addestramento modello, predizione dei dati e calcolo misure di valutazione.
Regressione Polinomiale:

```
from sklearn.preprocessing import PolynomialFeatures
def trainval_polynomial(degree):

    pf = PolynomialFeatures(degree)

    pf.fit(X_train)# non addestra il modello, bensì si calcola tutte le combinazioni
polinomiali
    X_train_poly = pf.transform(X_train)# trasformazione in polinomi
    X_val_poly = pf.transform(X_val)


    polyreg = LinearRegression()#si utilizza la regressione lineare però con polinomi al
posto di feature
    polyreg.fit(X_train_poly,y_train)


    y_poly_train_pred = polyreg.predict(X_train_poly) #si verifica per il training set
    y_poly_val_pred = polyreg.predict(X_val_poly) #si verifica per il validation set

    mae_train = mean_absolute_error(y_train, y_poly_train_pred)
    mse_train = mean_squared_error(y_train, y_poly_train_pred)
    rmse_train = np.sqrt(mean_squared_error(y_train, y_poly_train_pred))

    mae_val = mean_absolute_error(y_val, y_poly_val_pred)
    mse_val = mean_squared_error(y_val, y_poly_val_pred)
    rmse_val = np.sqrt(mean_squared_error(y_val, y_poly_val_pred))

    return mae_train, mse_train, rmse_train, mae_val, mse_val, rmse_val


for d in range(1,4):
    print("DEGREE: {} \n      {:>8s} {:>8s} {:>8s}\nTRAIN {:8.2f} {:8.2f} {:8.2f} \nVAL
{:8.2f} {:8.2f} {:8.2f}\n\n".format(d,"MAE", "MSE", "RMSE", *trainval_polynomial(d)))
```

REGRESSIONE POLINOMIALE

DEGREE: 1

	MAE	MSE	RMSE
TRAIN	0.53	0.51	0.72
VAL	0.53	0.53	0.73

DEGREE: 2

	MAE	MSE	RMSE
TRAIN	0.46	0.42	0.65
VAL	0.48	0.91	0.95

DEGREE: 3

	MAE	MSE	RMSE
TRAIN	0.42	0.34	0.58
VAL	23.48	2157650.15	1468.89

Regressione Lazo:

```
from sklearn.linear_model import Lasso
def trainval_polynomial_lasso(degree, alpha):
    pf = PolynomialFeatures(degree)

    pf.fit(X_train)
    X_train_poly = pf.transform(X_train)
    X_val_poly = pf.transform(X_val)

    polyreg = Lasso(alpha=alpha)
    polyreg.fit(X_train_poly, y_train)

    y_poly_train_pred = polyreg.predict(X_train_poly)
    y_poly_val_pred = polyreg.predict(X_val_poly)

    mae_train = mean_absolute_error(y_train, y_poly_train_pred)
    mse_train = mean_squared_error(y_train, y_poly_train_pred)
    rmse_train = np.sqrt(mean_squared_error(y_train, y_poly_train_pred))

    mae_val = mean_absolute_error(y_val, y_poly_val_pred)
    mse_val = mean_squared_error(y_val, y_poly_val_pred)
    rmse_val = np.sqrt(mean_squared_error(y_val, y_poly_val_pred))

    return mae_train, mse_train, rmse_train, mae_val, mse_val, rmse_val

print("LSSO, DEGREE: 2")
for alpha in [0.02,0.03,0.04]:
    print("Alpha: {:0.2f} \n      {:>8s} {:>8s} {:>8s}\nTRAIN {:8.2f} {:8.2f} {:8.2f}
\nVAL   {:8.2f} {:8.2f} {:8.2f}\n\n".format(alpha,"MAE", "MSE", "RMSE",
*trainval_polynomial_lasso(2,alpha)))
```

LSSO, DEGREE: 2

Alpha: 0.02

	MAE	MSE	RMSE
TRAIN	0.52	0.51	0.71
VAL	0.54	1.19	1.09

Alpha: 0.03

	MAE	MSE	RMSE
TRAIN	0.55	0.55	0.74
VAL	0.55	0.59	0.77

Alpha: 0.04

	MAE	MSE	RMSE
TRAIN	0.56	0.57	0.76
VAL	0.57	0.59	0.77

Sebbene si possano eseguire in modo indipendente le operazioni di standardizzazione, espansione delle caratteristiche polinomiali e regressione, attraverso uno strumento di scikit-learn, possiamo automatizzare i processi. Questa automatizzazione è permessa attraverso la Pipeline.

```
from sklearn.pipeline import Pipeline
polynomial_regressor = Pipeline([
    ('scaler', StandardScaler()),
    ('polynomial_expansion', PolynomialFeatures()),
    ('ridge_regression', Ridge())
])


polynomial_regressor.set_params(polynomial_expansion__degree=2,
ridge_regression__alpha=300)

polynomial_regressor.fit(X_train, y_train)
y_val_pred = polynomial_regressor.predict(X_val)

mean_absolute_error(y_val, y_val_pred), mean_squared_error(y_val, y_val_pred),
np.sqrt(mean_squared_error(y_val, y_val_pred))
```

Attività di classificazione, misure di valutazione e K-nearest Neighbor

I modelli di classificazione rappresentano un'altra classe di modelli predittivi. Insieme alla regressione, la classificazione è un'attività principale dell'apprendimento supervisionato.
Nel caso della regressione, abbiamo visto che i valori appartenevano a $\Re^m$, nel caso della classificazione, i valori di output sono discreti, solitamente indicati come "classi".

Y = {0, … ,M-1}

dove M è il numero di classi.
Quindi avremmo un training set per addestrare e valutare il modello che sarà definito così:

$$\boldsymbol{D} = \{(\boldsymbol{x_i}, \boldsymbol{y_i})\}_{\boldsymbol{i=1}}^{\boldsymbol{N}}$$

dove:

- x_i € $\Re^m$
- y_i € $\{0, \ldots, M-1\}$ generalmente chiamate **etichette**.

I valori predetti attraverso un **classificatore** h sono $\hat{y} = h(x)$ e vengono chiamate **etichette previste**.
Si può ottimizzare il modello h minimizzando il rischio empire. Una possibile funzione di loss è la seguente:

$$\mathbf{L(y, \hat{y})} = \begin{cases} \mathbf{1}\ \boldsymbol{se}\ \ \hat{\boldsymbol{y}} \neq \mathbf{y} \\ \mathbf{0}\ \boldsymbol{se}\ \ \hat{\boldsymbol{y}} = \mathbf{y} \end{cases}$$

$$\boldsymbol{R_{emp}} = \frac{\mathbf{1}}{\boldsymbol{N}} \sum_{\boldsymbol{i=1}}^{\boldsymbol{N}} \mathbf{L}(\mathbf{y}_i, \hat{\boldsymbol{y}}_i) = \frac{\boldsymbol{numero\ di\ predizioni\ sbagliate}}{\boldsymbol{N}}$$

Il rischio empirico per la classificazione è anche chiamato **tasso di errore**, ed è un numero € [0,1].

$$\hat{\boldsymbol{h}} = \boldsymbol{arg\ min R_{emp}(h)}$$

Un esempio di classificatore è il rilevamento di e-mail spam.
Definire le misure di valutazione è utile per:

- Guidare l'addestramento;
- Ottimizzare gli iper-parametri;
- Valutare che l'algoritmo funzioni sul testing set.

Si considera l'insieme delle etichette reali e l'insieme delle etichette previste:

$$Y_{TE} = \{y^{(i)} | (x^{(i)}, y^{(i)}) \in TE\}_i$$
$$\hat{Y}_{TE} = \{h(x^{(i)}) | (x^{(i)}, h(x^{(i)})) \in TE\}_i$$

Dove TE è il test set.

Le misure di valutazione sono:

- **accuratezza** -> misura di prestazione comune e si definisce come la percentuale di etichette per le quali l'algoritmo ha previsto l'etichetta corretta:

  $\boldsymbol{ACCURACY(Y_{TE}, \hat{Y}_{TE})} = \frac{\boldsymbol{y^{(i)}: y^{(i)} = \hat{y}^{(i)}}}{|\boldsymbol{Y_{TE}}|}$ numero compreso sempre tra 0 e 1
- **tasso di errore**-> strettamente collegato all'accuratezza, misura la percentuale di etichette sbagliate prodotto dall'algoritmo:

  ERROR RATE$\boldsymbol{(Y_{TE}, \hat{Y}_{TE})} = \frac{\boldsymbol{y^{(i)}: y^{(i)} \neq \hat{y}^{(i)}}}{|\boldsymbol{Y_{TE}}|} = \mathbf{1} - \boldsymbol{ACCURACY(Y_{TE}, \hat{Y}_{TE})}$

Il problema dell'accuratezza è che conta quanti errori ha commesso l'algoritmo e non di quale tipo.
In generale possiamo commettere 2 tipi di errore:

- **Tipo 1**: consideriamo un elemento come appartenente alla classe presa in considerazione, ma in realtà non ne fa parte -> **FALSO POSITIVO(FP)**;
- **Tipo 2**: consideriamo un elemento come non appartenente alla classe presa in considerazione, ma in realtà ne fa parte -> **FALSO NEGATIVO (FN)**;

Possiamo anche avere le previsioni corrette come: **vero positivo** e **vero negativo**.
Possiamo inserire i numeri in questa matrice, detta **matrice di confusione**, per avere una visione completa sul calcolatore, e poi calcolare l'accuratezza.

CONFUSION MATRIX		PREDICTED LABELS	
		POSITIVE	NEGATIVE
TRUE LABELS	POSITIVE	TP	FN
	NEGATIVE	FP	TN

CONFUSION MATRIX		PREDICTED LABELS	
		POSITIVE	NEGATIVE
TRUE LABELS	POSITIVE	TP	FN
	NEGATIVE	FP	TN

$$Accuracy = \frac{TP + TF}{TP + FN + FP + TN}$$

Si fa la somma tra i casi veri positivi e veri negativi e infine si effettua una divisione per i casi totali.
Un **classificatore ingenuo** è un classificatore che su varie classi riconosce la **classe maggioritaria**.
Esempio:

- Si hanno 10000 casi da classificare e le classi sono {0,1}. Dopo aver applicato il classificatore si ha che:
  - 9500 sono di classe 1 ed effettivamente sono di vero positivo;
  - 500 sono di classe 0 ma sono stati classificati di classi 1, quindi avendo dei falsi positivi;

È un buon classificatore sapendo che l'accurancy = $\frac{9500+0}{9500+500+0+0} = 0{,}95$? No, perché non riconosce mai la classe 0 e dice sempre la risposta più probabile.
La matrice di confusione si utilizza per capire se c'è un problema con il classificatore in caso di dati sbilanciati, ma per capire se ci sono comportamenti anomali si possono utilizzare anche misure complementari.
Queste misure sono:

- **Precisione**, che calcola quanti positivi, erano effettivamente casi di veri positivi e si calcola in questo modo:

  $$precision = \frac{TP}{TP + FP}$$

  Misura la qualità delle predizioni positive.
  Es. 80% delle predizioni è vero(i casi positivi devono essere veramente positivi).
- **Richiamo**, che misura quanti positivi sono stati correttamente classificati come positivi e si calcola come segue:

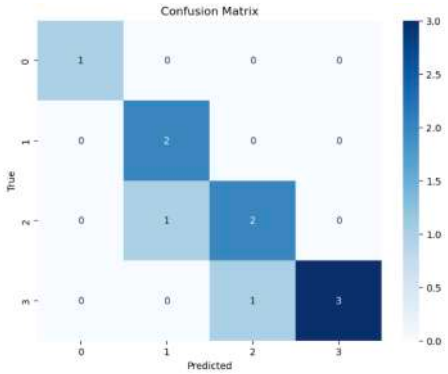
  $$recall = \frac{TP}{TP + FN}$$

  Misura della capacità di trovare tutte le predizioni positive.
  Es. trova 55% dei casi veri totali(il numero di casi positivi trovati deve essere alto).

Solitamente si devono applicare entrambe le misure, anche se spesso si trovano valori contrastanti tra di loro.
Possiamo invece considerare le due metriche congiuntamente, attraverso una media armonica. La misura che li considera congiuntamente è l'F_1-score ed è definito come segue:

$$\boldsymbol{F_1 = 2 * \frac{precision * recall}{precision + recall}}$$

La matrice di confusione può essere generalizzata al caso in cui ci siano M classi. In tal caso la matrice è di dimensioni MxM e l'elemento C_{ij} indica il numero di elementi appartenenti alla classe i che sono stati classificati come appartenenti alla classe j.



Lungo la diagonale principale, ci sono i veri positivi, invece in tutto il resto delle celle sono presenti tutti gli errori di classificazione. Possiamo verificare le varie misure delle classi calcolandole singolarmente, come se fosse un problema binario.

Molti classificatori non producono spesso una classe, ma producono un punteggio:

- **Probabilità**
- **Punteggio**

Questo numero è indicativo su quanto il modello sia sicuro che l'osservazione sia positiva. Per ottenere la classe finale, si deve stabilire una soglia, es. 0,5 -> positivo.
La scelta della soglia influisce sul risultato che si ottiene:

- **Una soglia bassa** -> il modello diventa più sensibile(trova più positivi);
- **Una soglia alta** -> il modello diventa più restrittivo.

La curva ROC permette di valutare un classificatore indipendentemente dalla soglia scelta. Infatti, in questo modo possiamo osservare tutti i possibili comportamenti del modello.
Consideriamo quindi:

c(x)

essere una funzione di predizione di confidenza, da un vettore in input x e ad esempio nel caso del regressore logistico:

$$\boldsymbol{c(x) = \sigma(\beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n)}$$

La funzione di classificazione sarà definita come segue:

$$\boldsymbol{h_\theta = [\sigma(\beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n) \geq \theta]}$$

Dove:

- **[]** le parantesi sono le **parentesi di Iverson** -> restituiscono un valore vero o falso;
- $\boldsymbol{\theta}$ € $\boldsymbol{R}$ è una soglia di un valore reale -> valore di probabilità;

A seconda della soglia scelta avremmo un numero di valori:

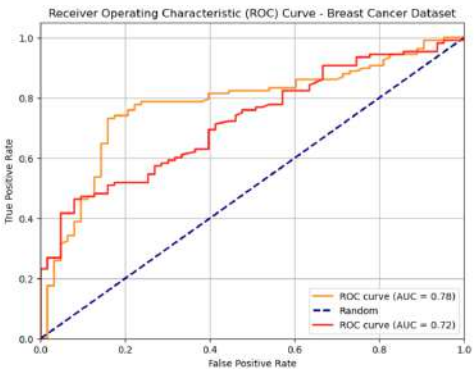
- **veri positivi $\boldsymbol{TP_\theta}$;**
- **falsi positivi $\boldsymbol{FP_\theta}$;**
- **veri negativi $\boldsymbol{TN_\theta}$;**
- **falsi negativi $\boldsymbol{FN_\theta}$.**

Definiremo quindi il tasso di veri positivi TPR e il tasso di falsi positivi FPR come segue:

- **TPR(true positive rate) =** $\boldsymbol{\frac{TP_\theta}{TP_\theta + FN_\theta}}$ come il richiamo, veri positivi su tutti gli elementi positivi;
- **FPR(false positive rate) =** $\boldsymbol{\frac{FP_\theta}{FP_\theta + TN_\theta}}$ falsi positivi su tutti gli insiemi di veri negativi;

Se la soglia è bassa quindi, allora sia il valore di TPR e FPR è pari a 1, visto che tutti gli elementi vengono considerati positivi, altrimenti se è bassa allora saranno pari a 0, visto che tutti gli elementi vengono considerati negativi.

La curva ROC si ottiene scegliendo un valore soglia θ e tracciando un punto 2D(TNR,FPR). I punti (0,0)(tutto negativo) e (1,1)(tutto positivo) appartengono alla curva.
La linea tratteggiata rappresenta un classificatore random, la cui area sotto la curva è 0.5. L'AUC(area under the curve), infatti è l'area sotto la curva e serve per valutare l'efficacia di un classificatore.



Qualsiasi curva che si trova al di sotto della linea randomica, indica un classificatore che applica una soglia non adeguata.

- Se AUC = 1, classificatore perfetto;
- Se AUC = 0.5 classificatore inutile;
- Se AUC < 0.5 classificatore sbagliato(inverti previsioni).

Vediamo come creare un semplice classificatore in python:

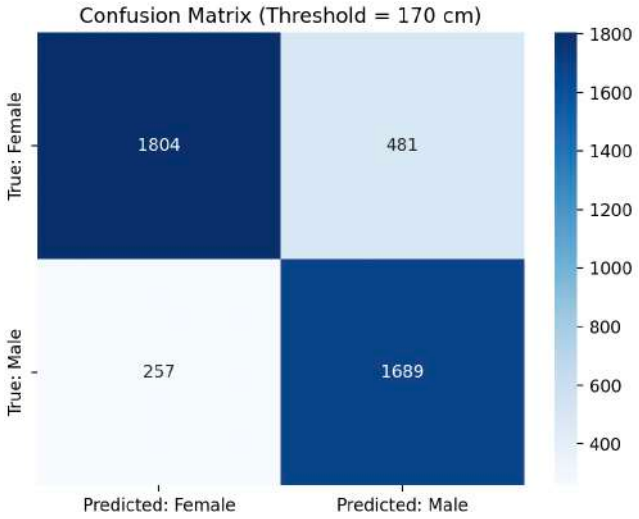
- Carichiamo il dataset:

```
data = pd.read_csv("http://antoninofurnari.it/downloads/height_weight.csv")
from sklearn.metrics import confusion_matrix
```

- Creiamo soglia e matrice di confusione:

```
soglia = 170 #impostiamo la soglia
male_pred = (data['height'] >= soglia) #classificazione
male_gt = (data['sex'] == 'M') #veri positivi
cm = confusion_matrix(male_gt,male_pred) #creiamo la matrice di confusione
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
        xticklabels=['Predicted: Female', 'Predicted: Male'],
       yticklabels=['True: Female', 'True: Male'])
plt.title(f'Confusion Matrix (Threshold = {soglia} cm)')
plt.show()
print(classification_report(male_gt, male_pred))
```

Infine, otterremo questo risultato:

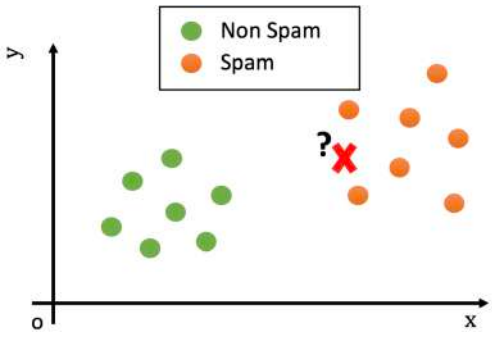


La soglia migliore dipende dall'obiettivo, potremmo volere alta sensibilità con un TPR alto e un FPR alto(screening medico -> poi si verifica con un esame più specifico(tipo per il cancro)), oppure alta specificità(filtro antispam -> si può non rilevare elemento positivo) basso FPR.
Può essere una via di mezzo oppure per trovarla in modo più semplice attraverso la **statistica J di Youden**, che massimizza la TPR e la specificità.

J = TPR+(1-FPR)

La classificazione per soglia funziona con una sola caratteristica, infatti abbiamo visto che possiamo classificare se è uomo o donna attraverso il solo confronto dell'altezza.

Un classificatore intuitivo e non parametrico basato sul concetto di similarità tra punti dati è il K-Nearest Neighbor. Introduciamo prima l'algoritmo 1-NN, ossia il punto più vicino è quello da esaminare per assegnargli la stessa classe.



Per determinare la classe possiamo utilizzare una funzione di distanza definita così:

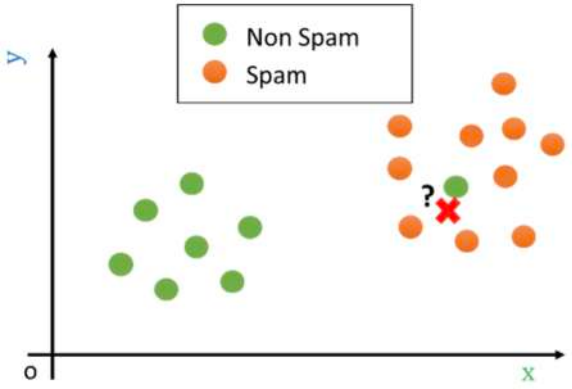
$$\boldsymbol{d(x, x')}$$

d potrebbe essere qualsiasi tipo di distanza ad esempio quella euclidea.
Quindi possiamo assegnare la classe a x' quella dell'esempio più vicino come segue:

h(x') = arg min {d(x,x') | (x,y) € TR}

Questo è chiamato algoritmo **next-neighbor** (o **1-NN**).
L'algoritmo **next-neighbor** presuppone che i punti dati siano vicini tra loro nel set di addestramento, questo è vero quando lo spazio di addestramento è ideale per la classificazione, ossia i punti sono puliti e semplici.

Spesso però i test di addestramento contengono **valori anomali**, ovvero che i punti dati non seguono fedelmente la distribuzione degli altri punti dati.



In questo caso non si dovrebbe guardare al punto più vicino nello spazio, bensì **all'intorno** nel punto dati.
In questo caso considerando l'intorno, possiamo ragionevolmente classificare il nostro punto come spam.
Stabilire però il raggio, soprattutto in spazi meno densi potrebbe portare ad includere anche 0 elementi.
Per questo motivo si considerano raggi contenenti al massimo k elementi, dove k è un iper-parametro dell'algoritmo.

Dato un punto x', l'intorno dei punti di addestramento di dimensione k centrato a x' è definito così:

$$\boldsymbol{N_k(x') = N(x', R_k(x'))}$$

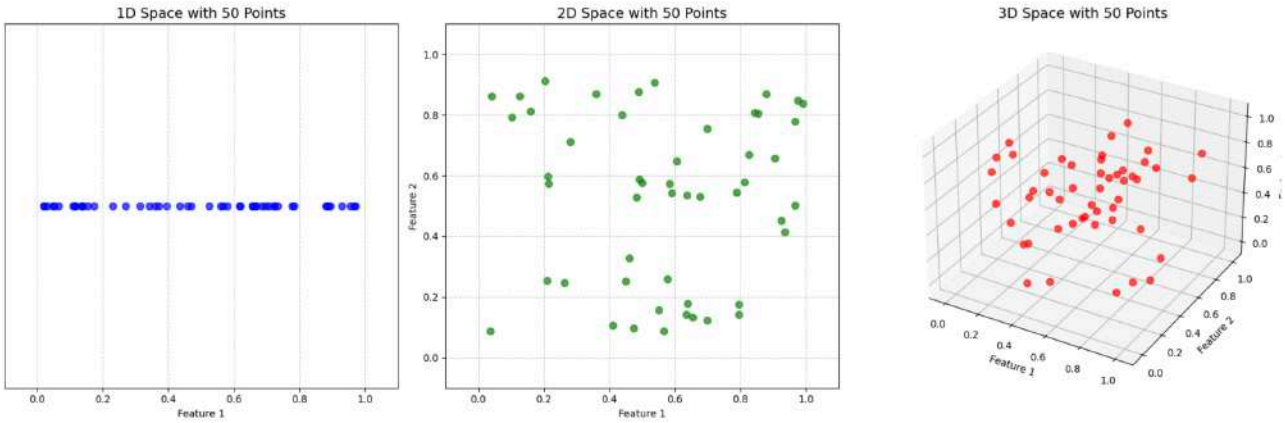
dove:
$\boldsymbol{R_k(x') = sup\{r : |N(x', r)\backslash\{x'\} \leq K\}}$ **-> si prende il raggio più grande così possiamo arrivare al k-esimo punto più lontano**

Infine, definiamo l'algoritmo di classificazione K-Nearest Neighbor come segue:

$$\boldsymbol{h(x') = mode\{y \mid (x, y) \in N(x';\ TR, K\}}$$

dove mode è la funzione modalità statistica, che restituisce l'elemento più frequente di un insieme.
Quindi la classe viene scelta in base all'elemento più frequente tra i K più vicini.
K è un iper-parametro dell'algoritmo, che può essere scelto arbitrariamente oppure ottimizzato usando un set di convalida.

K-NN soffre però del problema noto come “**Maledizione della densità**”, ossia che più le dimensioni aumentano e più il concetto di distanza perde di significato.



Possiamo quindi notare che il concetto di distanza all’aumentare delle dimensioni, perde di significatività. Si osserva che:

- I punti sulla retta sono molto vicini(1d);
- I punti in un piano sono più sparsi(2d);
- I punti nello spazio sono sparsi e lo spazio sembra apparentemente vuoto(3d);

Aumentando le dimensioni, tipo di 50,100, ecc., i punti si allontaneranno sempre di più.
L’aumento delle dimensioni porta a considerare due problemi:

- **sparsità** -> il vicino più prossimo potrebbe essere molto lontano, e il vuoto che separa un punto da un altro potrebbe mettere in dubbio se i due punti appartengono alla stessa classe;
- **distanza perde significato** -> la distanza tra il punto più vicino e il punto più lontano potrebbe diventare la stessa, ed è per questo motivo che il concetto di vicino più prossimo diventa casuale e privo di senso.

Per verificare se ancora le distanze sono significative, lo possiamo fare attraverso una misura di dispersione delle distanze, che si calcola come segue:

$$\boldsymbol{relative\ spread}(\boldsymbol{x}') = \frac{\boldsymbol{d_{max}}(\boldsymbol{x}') - \boldsymbol{d_{min}}(\boldsymbol{x}')}{\boldsymbol{d_{max}}(\boldsymbol{x}')}$$

- **Se lo spread è alto**, vuol dire che c’è **una differenza marcata tra il punto più vicino e quello più lontano**;
- **Se lo spread è basso**(spread = 0) vuol dire che **distanza massima e distanza minima diventano sempre più uguali**.

I classificatori sono generalmente divisi in due tipologie diverse:

- **generativi** -> apprende la distribuzione di probabilità di ogni classe e crea la storia, contenendo all’interno informazioni statistiche e altre informazioni apprese. Dopodiché utilizza questa conoscenza e il teorema di bayes per trovare la probabilità che sia di classe y dato un nuovo x.
  Un esempio è il Naive Bayes;
- **discriminativi** -> si occupa di trovare il confine che separa una classe A da una classe B , ad esempio K-Nearest Neighbors o regressione logistica.

K-NN può anche essere utilizzato per la regressione lineare.

Regressione Logistica

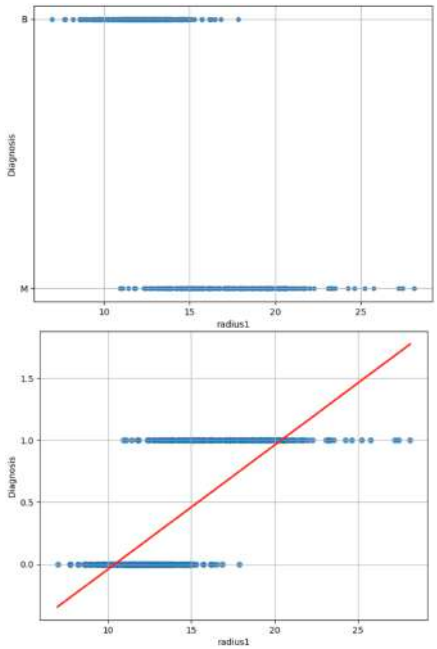
Abbiamo osservato come un modello non parametrico per la classificazione tipo il K-NN abbia problemi legati a:

- Scalabilità scarsa con molti attributi -> la distanza perde di significato;
- Richiede la memorizzazione dell'intero dataset;
- Si deve confrontare ogni punto con tutto il resto.

I modelli parametrici non hanno questo problema, infatti, una volta addestrati:

- Non è necessario conservare i dati di addestramento;
- Si possono utilizzare per classificare nuovi esempi;

Tuttavia, modelli come la regressione lineare non sono in grado di modellare relazioni tra variabili indipendenti continue(predittori numerici) e variabili dipendenti qualitative(variabili regredite non numericamente misurabili).



Ad esempio, possiamo osservare come esista una relazione tra radius1 e diagnosi, infatti, all'aumentare di 'radius1 ' la diagnosi ci dice che un tumore è benigno o maligno.
Possiamo utilizzare il regressore lineare per la variabile indipendente diagnosi, trasformando la variabile stessa in variabile continua e specificare che:

- Se il valore è minore a1 e maggiore uguale a 0 allora è uguale a B
- Se è maggiore uguale a 1 allora è uguale a M.

Così però stiamo assumendo che il regressore lineare assume la forma:

- f: $\mathbb{R} \rightarrow \mathbb{R}$

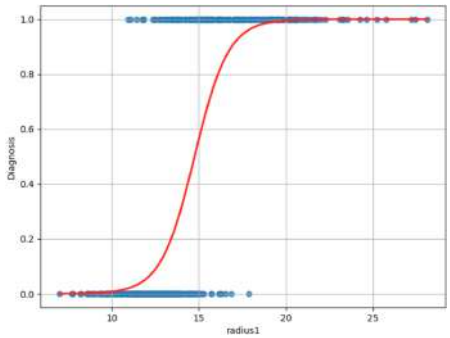
invece, la variabile diagnosi:

- f: $\mathbb{R} \rightarrow \{0,1\}$

Quindi, mentre con un regressore lineare si volevano prevedere valori continui, ora si vogliono assegnare delle osservazioni in contenitori discreti.

Al posto di classificare direttamente, si potrebbe scegliere di modellare la probabilità, ovvero di passare da {0,1} a [0,1].

Dovremmo avere qualcosa che saturi valori più piccoli di 0 a 0 e valori grandi a 1.



Dobbiamo in pratica trovare una formulazione del regressore lineare che trasformi l'output in una funzione non lineare.
Possiamo utilizzare la **formula logistica**, che permette di interpretare facilmente un modello in modo probabilistico.
La funzione logistica è la seguente:

$$\boldsymbol{f(x) = \frac{1}{1 + e^{-x}}}$$

il grafico prodotto, ossia quello a sinistra è la sigmoide. La funzione è limitata tra 0 e 1 ed è derivabile.

Definiamo quindi il nostro modello di regressione logistico, come segue:

$$\boldsymbol{P(Diagnosi = 1|x) = f(\beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n) = \frac{1}{1 + e^{-(\beta_0+\beta_1 x_1+\cdots+\beta_n x_n)}}}$$

Attraverso varie trasformazioni possiamo vedere come:

$\boldsymbol{e^{(\beta_0+\beta_1 x_1+\cdots+\beta_n x_n)} = \frac{p}{1-p}}$ -> questo termine si chiama **odds** e mostra il numero di volte in cui crediamo che l'esempio sarà positivo sul numero di volte in cui crediamo sia negativo.

Infine, abbiamo che **log($\boldsymbol{\frac{p}{1-p}}$) = $\boldsymbol{\beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n}$**-> **logit della probabilità**.

Questo è utile perché modelliamo il logit(**regressione logit**) della probabilità e non la probabilità direttamente.

La funzione sigmoide trasforma un numero reale in una probabilità, il logit fa l'opposto. La regressione lineare modella il logit della probabilità come una funzio\ne lineare delle feature.

Per adattare meglio il modello e trovare i parametri β, definiremo una funzione di costo. Anche se abbiamo adattato la funzione logistica come una lineare, non conosciamo la probabilità vera degli elementi. Infatti, le osservazioni forniscono solo esempi di input x ed etichette corrispondenti y.

P(y|x; β) = $f_\beta(x) = \frac{1}{1+e^{-\beta^T x}} = \sigma(\beta^T x)$

Possiamo scrivere:

$$P(y|x; \beta) = f_\beta(x)$$

$$P(y|x; \beta) = 1 - f_\beta(x)$$

Assumendo che tutti gli eventi siano indipendenti, possiamo calcolare la verosimiglianza totale, ovvero che tutti le osservazioni si verifichino insieme, dati i parametri β.

Sappiamo che la probabilità che due eventi indipendenti si verifichino, ovvero che P(x,y) = P(x)*P(y). Allora:

L($\boldsymbol{\beta}$) =$\boldsymbol{\prod_{i=1}^{n} P(y^{(i)}|x^{(i)}; \beta) = \prod_{i=1}^{n} f_\beta(x)^{y^{(i)}}(1 - f_\beta(x^{(i)}))^{1-y^{(i)}}}$

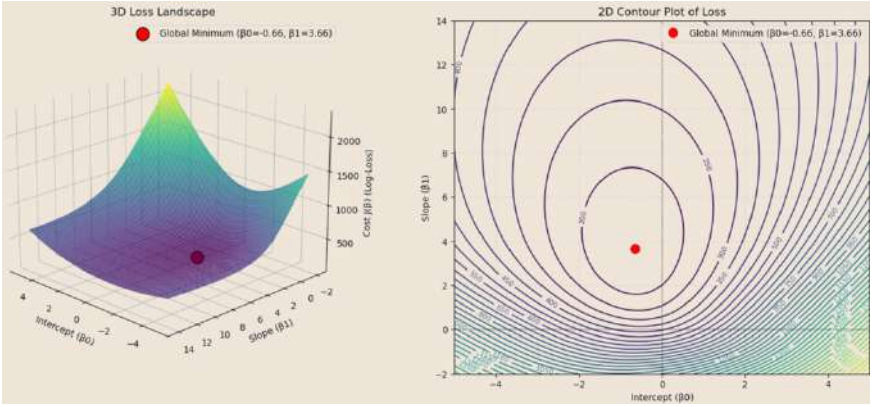
Per derivare sarà difficile farlo, per questo motivo attraverso il logaritmo i prodotti si trasformano in somme. La funzione di costo sarà quindi definita nel seguente modo:

$$\boldsymbol{J(\beta) = -\sum_{i=1}^{n} [y^{(i)} log\sigma(\beta^T x^{(i)}) + (1 - y^{(i)}) log(1 - \sigma(\beta^T x^{(i)}))]}$$

La funzione di costo della regressione logistica è convessa, cioè ha una sola soluzione globale.

Questo significa che, indipendentemente da dove partiamo (stima iniziale), la discesa del gradiente porterà sempre alla stessa soluzione ottimale.

Calcoleremo quindi la derivata e troveremo punto di massimo e minimo. Prenderemo il punto di minimo perché vorremmo minimizzare l'errore del modello.



Applicando quello che sappiamo dei regressori lineari ai regressori logistici, possiamo scrivere:

$$\boldsymbol{x = 0 \Longrightarrow \log \frac{p}{1-p} = \beta_0}$$

e quindi:

$$\boldsymbol{x = 0 \Longrightarrow \frac{p}{1-p} = e^{\beta_0}}$$

Cioè quando tutte le variabili non sono nulle e^{β_0} è l'odd di y = 1.

Inoltre, possiamo dire che:

$$\boldsymbol{\frac{odd(p \mid x+1)}{odd(p \mid x)} = \beta_1}$$

Aumentando quindi una variabile x corrisponde di un aumento moltiplicativo degli odd di e^{β_1}.

Quest'analisi può essere estesa al caso di un regressore logistico multiplo:

$$\textbf{P(y=1|x)} = \boldsymbol{\sigma(\beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n)}$$

- se $x_i = 0 \; \forall i$ allora l'odd di $\boldsymbol{\beta_0 = 0}$;
- Se $e^{\beta_i} = 1.05$, allora un aumento di 1 unità di $x_i$provoca un aumento del 5% degli odds di avere $y = 1$.

I valori hanno interpretazioni simili a quelle ottenute nel caso della regressione lineare. Discostano perché nella regressione lineare trattiamo i coefficienti, invece nella regressione logistica i coefficienti logit.

Non possiamo utilizzare R^2per valutare il nostro modello, infatti, nel modello logistico non si prevede un valore continuo, ma una probabilità.

Non si minimizzano i residui quadratici, bensì, si minimizza la **logaritmo-verosimiglianza negativa**(J(B)).

Per ottenere una metrica simile "da 0 a 1" per la bontà di adattamento si è sviluppato una **"Pseudo $\boldsymbol{R^2}$"** o **"$\boldsymbol{R^2}$McFadden"**. Il punto di partenza è lo stesso di R^2, ossia quanto è migliore il nostro modello rispetto ad uno di base stupido(tipo uno che prevede sempre il valore medio).

Contestualizzando nello "Pseudo R^2" si confronta la verosimiglianza del nostro modello con la verosimiglianza logaritmica di un modello di base.

$$R^2McFadden = 1 - \frac{LL_{model}}{LL_{null}}$$

Dove:

- LL_{model} è la verosimiglianza logaritmica del modello trovato;
- LL_{null} è un modello che conosce solo la situazione a priori, dove annulla tutte le variabili e contiene solo l'intercetta β_0 -> la probabilità sarà uguale per tutti.

Sono tutti e due i valori negativi, e il valore di LL_{model} è maggiore, dato che si avvicina più a 0. Il valore assoluto di LL_{null} è maggiore in valore assoluto di LL_{model}.

- Se $R^2McFadden \rightarrow \mathbf{0}$, allora il modello non migliora rispetto al modello null;
- Se $R^2McFadden \rightarrow \mathbf{1}$, allora il modello spiega bene la varianza dei dati.

In python si fa così:

```
import pandas as pd
import numpy as np
from matplotlib import pyplot as plt
from statsmodels.datasets import get_rdataset
from statsmodels.formula.api import logit

biopsy = get_rdataset('biopsy',package='MASS')

df = pd.DataFrame(biopsy.data)

df['cl'] = df['class'].replace({'benign':0,'malignant':1})

model = logit('cl ~ V1 + V2 + V3 + V4 + V5 + V6 + V7 + V8 + V9', df).fit()
print(model.summary())
model = logit('cl ~ V1 + V3 + V4 + V5 + V6 + V7 + V8 + V9', df).fit()
print(model.summary())

model = logit('cl ~ V1 + V3 + V4 + V6 + V7 + V8', df).fit()
print(model.summary())

print(np.exp(model.params))

print()
print("In percentuale:")
for ind,perc in np.exp(model.params).items():
    perc = (perc -1 )
    print(f"{ind} : {perc:.2%}")
```

```
In percentuale:
Intercept : -99.99%
V1 : 86.36%
V3 : 41.84%
V4 : 40.15%
V6 : 46.02%
V7 : 60.21%
V8 : 27.53%
```

REGRESSIONE PREDITTIVA MULTICLASSE E VISTA PREDITTIVA

Spesso si vuole studiare la relazione tra variabili indipendenti categoriali o continue e una variabile dipendente categoriale non binaria(il predicato assume solo il valore vero o falso).

Quando una variabile può assumere più di due valori, possiamo definire il **modello di regressione logistica multinomiale**.

Supponendo di avere una variabile che possa assumere tre valori:

- Basso;
- Medio;
- Alto;

Si sceglie uno dei valori Y come classe base. K è il numero di classi e Y =1 è la classe base(baseline). Nel regressore logistico avevamo:

$$\log\left(\frac{P(y=1|X)}{1-P(y=1|X)}\right) = \beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n$$

Nel caso di più variabili, invece di modellare le probabilità, un regressore logistico multinomiale modella il logaritmo del rapporto tra una data classe k e la classe di base 1. La formula è la seguente:

$$\log\left(\frac{P(Y=k \mid X=x)}{P(Y=1|X=x)}\right) = \beta_{k0} + \beta_{k1} X_1 + \ldots + \beta_{kn} X_n$$

Si definisce quindi una combinazione lineare diversa per ogni classe k, quindi, abbiamo bisogno di (n+1)*(k-1) parametri (n+1 perché si considerano tutte le feature + l'intercetta e k-1 perché la classe baseline non si considera-> log(1)= 0).

Si può dimostrare che:

- $P(Y = k|X = x) = \frac{e^{\beta_k^T X}}{1+\sum_{l=2}^{k} e^{\beta_l^T X}}$
- $P(Y = k|X = x) = \frac{1}{1+\sum_{l=2}^{k} e^{\beta_l^T X}}$

```
diabetes_data =
sm.datasets.get_rdataset("Diabetes","heplots").data
print(diabetes_data.head(5))
diabetes_data2 = diabetes_data.copy()
diabetes_data2['group'] =
diabetes_data2['group'].map({
    'Normal': 0,
    'Chemical_Diabetic': 1,
    'Overt_Diabetic': 2
}).astype(int)
model = smf.mnlogit("group ~
glutest",data=diabetes_data2)
result = model.fit()
print(result.summary())
print("\nModel Likelihood Ratio Test:")
print(result.llr, result.llr_pvalue)
```

```
  group=1      coef    std err          z      P>|z|     [0.025      0.975]
------------------------------------------------------------------------------
Intercept   -90.3738     42.527     -2.125      0.034   -173.726     -7.022
glutest       0.2153      0.101      2.128      0.033      0.017      0.413
------------------------------------------------------------------------------
  group=2      coef    std err          z      P>|z|     [0.025      0.975]
------------------------------------------------------------------------------
Intercept  -109.8266     43.021     -2.553      0.011   -194.147    -25.506
glutest       0.2471      0.102      2.428      0.015      0.048      0.447
```

Dai valori appresi, possiamo dire che quando glutest = 0, allora la probabilità che si ha il diabete chimico è:

$$\frac{e^{-903738}}{1} \approx 0$$

quindi, la probabilità che sia 'chemical_diabetic' è 0, ed è più probabile che il soggetto sia normale.

Ma come interpretiamo i parametri in modo geometrico?

Nella regressione lineare si cerca una curva che si "adatti ai dati", la regressione logistica trova un iperpiano che separa i dati.

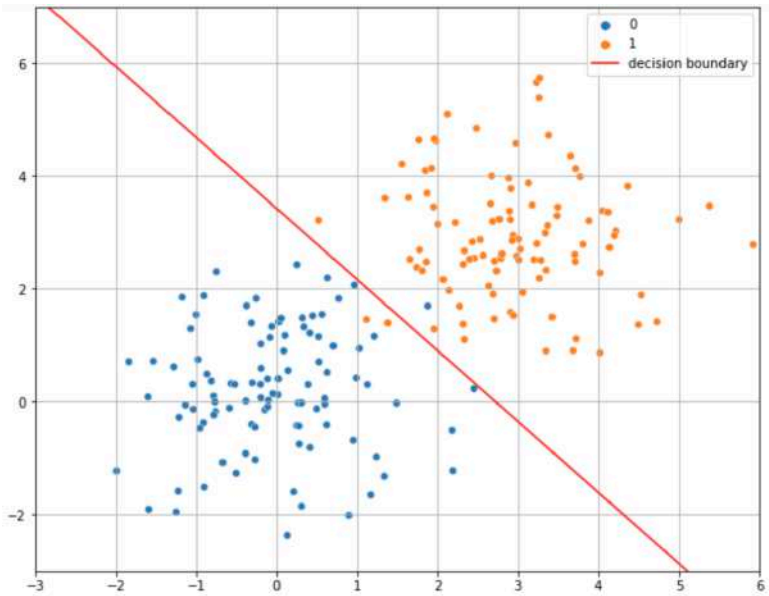
Il concetto chiave è il "confine di decisione". Il classificatore deve decidere se assegnare un dato alla classe 1 o alla classe 0.

- Se la probabilità P(y=1|x)$\geq$ $0.5,\ allora\ prediciamo\ 1$;
- Altrimenti 0.

Il confine geometrico si trova esattamente dove il classificatore è incerto, ovvero dove la probabilità è 0.5.

Questo si mostra nella seguente maniera:

P(y=1 | x) = 0.5 $\Leftrightarrow e^{-(\beta_0+\beta_1 x_1+\beta_2 x_2)} = 1 \Leftrightarrow 0 = \beta_0 + \beta_1 x_1 + \beta_2 x_2$ che è l'equazione di una retta.



Si trova quindi questa retta, chiamata **confine di decisione** che separa elementi di due classi. Nel caso in cui si considerano n variabili, allora troveremo le equazioni degli iperpiani.

Regressore Softmax

Un'alternativa alla regressione logistica multinomiale è la regressione Softmax, che evita di utilizzare una classe baseline e quindi è simmetrica.

In un **regressore softmax**, le probabilità sono modellate come segue:

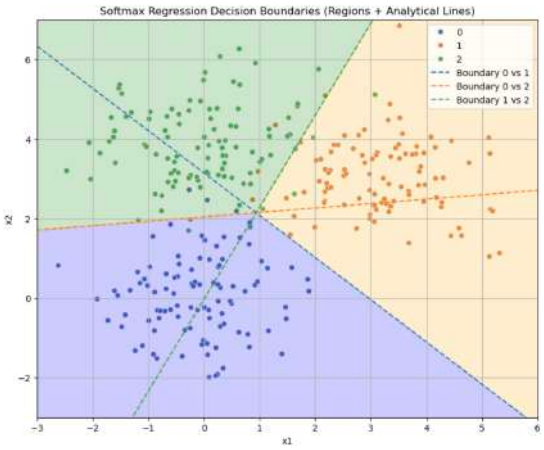
$$\boldsymbol{P(Y = k \mid X = x) = \frac{e^{\beta_k^T x}}{\sum_{l=1}^{K} e^{\beta_l^T x}}, \forall k = 1, \ldots, K}$$

Quindi si stimano k i coefficienti e non k-1. **L'ottimizzazione del modello avviene fissando una funzione di costo** e ottimizzandola con metodi iterativi(machine learning).

Un'osservazione di classe x viene assegnata alla classe con probabilità più alta. Il confine di decisione tra due classi è formato dai punti in cui la probabilità delle due classi è uguale.

L'equazione è un iperpiano nello spazio delle caratteristiche(feature), pertanto il regressore softmax suddivide lo spazio in regioni separate da confini lineari.

Inoltre, ogni vettore β_k definisce una direzione nello spazio delle caratteristiche che favorisce la classe k.



Un'alternativa alle precedenti è il '**one-vs-rest**' o '**one-vs-all**', che fornisce una tecnica per trasformare qualsiasi modello di classificazione binaria in un modello multi-classe.

Dato un **problema di classificazione con k classi**, l'approccio One-vs-Rest costruisce **k classificatori binari**, uno per ciascuna classe. Per farlo dobbiamo avere un classificatore binario come la regressione logistica, che fornisca una probabilità di trovare le classi.

Per un compito di classificazione con k classi, l'approccio OvR addestra K classificatori binari a distinguere una classe da tutto il resto.

Quindi avremmo:

- Classificatore1(h_1): addestrato a riconoscere la classe 1 vs(classe2+classe3...+classe n)
- Classificatore2(h_2): addestrato a riconoscere la classe 1 vs(classe1+classe3...+classe n)
- Classificatore3(h_3): addestrato a riconoscere la classe 1 vs(classe1+classe2...+classe n)

Per stimare una nuova osservazione x, si passa tra tutti i classificatori, e si sceglie quella che produce un punteggio di confidenza più alto. (APPROCCIO DIVIDE ET IMPERA)

Il codice carica il dataset Iris, separa le variabili dalle classi e suddivide i dati in training e test mantenendo la stessa proporzione di classi grazie all'opzione **stratify**.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report


from sklearn.datasets import load_iris
from sklearn.multiclass import OneVsRestClassifier


iris = load_iris()
X_i = iris.data
y_i = iris.target

target_names_i = iris.target_names

X_train_i, X_test_i, y_train_i, y_test_i = train_test_split(X_i,y_i,
test_size=0.3,random_state=42, stratify=y_i)
```

Esegue una Grid Search sul **modello Softmax** (Logistic Regression multinomiale) cercando il valore ottimale di **C** tramite validazione incrociata a 5 fold(cv = 5) e riportando il parametro migliore e l'accuratezza media ottenuta.

```
#SOFTMAX
pipe_multi = Pipeline(
    [
        ('scaler',StandardScaler()),
        ('model',LogisticRegression(solver='lbfgs',max_iter=200))
    ]
)

param_grid_multi ={
    'model__C': np.logspace(-3,3,7)
}

print("Running Grid Search for Multinomial(Softmax)...")

search_multi = GridSearchCV(pipe_multi,param_grid_multi,cv=5, scoring='accuracy')
search_multi.fit(X_train_i,y_train_i)
print(f"Best 'C' for Multinomial: {search_multi.best_params_['model__C']}")
print(f"Best CV Accuracy: {search_multi.best_score_:.4f}")
```

Esegue una Grid Search sulla strategia **One-vs-Rest**, ottimizzando il parametro **C** del classificatore logistico base attraverso una validazione incrociata a 5 fold e mostrando il valore migliore e l'accuratezza media ottenuta.

```
#OVR
pipe_ovr = Pipeline([
    ('scaler', StandardScaler()),
    ('model',OneVsRestClassifier(LogisticRegression(solver='liblinear')))
])

param_grid_ovr = {
    'model__estimator__C': np.logspace(-3,3,7)
}

print("Running Grid Search for One-vs-Rest(OvR)...")
search_ovr = GridSearchCV(pipe_ovr,param_grid_ovr,cv=5, scoring='accuracy')
search_ovr.fit(X_train_i,y_train_i)

print(f"Best 'C' for OvR: {search_ovr.best_params_['model__estimator__C']}")
print(f"Best CV Accuracy: {search_ovr.best_score_:.4f}")
```

Confronta le prestazioni dei due modelli (**Softmax vs OvR**) sul test set, calcolando l'accuracy e stampando i classification report completi per valutare precision, recall e F1-score delle due strategie.

```
#CONFRONTO
y_predi_multi = search_multi.predict(X_test_i)
acc_multi = accuracy_score(y_test_i,y_predi_multi)

y_predi_ovr = search_ovr.predict(X_test_i)
acc_ovr = accuracy_score(y_test_i,y_predi_ovr)


results = {
    "Multinomial(Softmax)": acc_multi,
    "One-vs-Rest": acc_ovr
}

results_df = pd.DataFrame.from_dict(results, orient='index', columns=['Test
Accuracy'])

print("\n--- Multi-Class Model Comparison ---")
print(results_df.sort_values(by='Test Accuracy', ascending=False))

print("\n--- Report for Multinomial (Softmax) Model ---")
print(classification_report(y_test_i, y_predi_multi, target_names=target_names_i))

print("\n--- Report for One-vs-Rest (OvR) Model ---")
print(classification_report(y_test_i, y_predi_ovr, target_names=target_names_i))
```

	Test Accuracy		
Multinomial(Softmax)	0.911111	One-vs-Rest	0.866667

Classificatori generativi e Naive Bayes

I modelli per la classificazione analizzati precedentemente sono discriminanti, ovvero mirano ad apprendere il confine decisionale tra le classi.

I modelli generativi diversamente pongono l'attenzione nell'apprensione dello storico o il profilo di ciascuna classe in modo indipendente.

Essi mirano a costruire un modello probabilistico di ciascuna classe. Nel dettaglio analizzano la distribuzione della probabilità congiunta P(X,Y) -> P(X|Y)P(Y).

Si divide in due pezzi separati:

- P(X|Y) -> Come si comporta X quando è in questa classe;
- P(Y=k) -> Quanto è comune questa classe in generale.

I classificatori sono costruiti sul principio del massimo a posteriori (MAP). Si vuole trovare la classe k più probabile dopo aver osservato i dati x, ossia:

$$\boldsymbol{h(x) \ = \ arg\ max\ P(Y = \ k \mid X \ = \ x)}$$

Utilizzando il teorema di Bayes, possiamo avere che:

$$\boldsymbol{h(x) \ = \ arg\ max\ \frac{P(X = \ x \mid Y \ = \ k)P(Y = \ k)}{P(X \ = \ X)}}$$

Dato che P(x) è costante, dato che è indipendente dalla classe k, notiamo che:

$$\boldsymbol{P(Y \ = \ k|x) \propto \ P(X|Y = k)P(Y = k)}$$

Ciò porta alla **regola finale di classificazione MAP**:

$$\boldsymbol{h(x) \ = \ arg\ max\ P(X \mid Y = \ k)P(Y = \ k)}$$

Dove:

- **P(Y) è la probabilità a priori** di una data classe, essa si può stimare:
  - Attraverso le osservazioni del dataset;
  - Attraverso lo studio di esempi di ciascuna classe nel mondo reale;
  - Attraverso delle assunzioni;

  in ogni caso si stima una probabilità a priori e quindi stiamo introducendo il grado di convinzione.
- **P(X|Y) è il termine di verosimiglianza** ed è più complesso da calcolare. Il modo in cui lo facciamo dipende dalle nostre feature, per questo motivo ci rimbattiamo nella maledizione della dimensionalità.

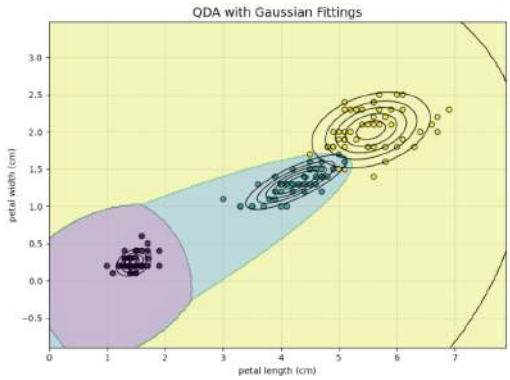
Ad esempio, se noi dovessimo classificare un'e-mail spam attraverso uno spam detector, esso si costruirebbe a partire dalle parole contenute nelle mail. Si costruirebbe un sottoinsieme del dizionario e se avessimo circa 10000 parole nel training set avremmo 2^{10000}combinazioni possibili. Impossibile computazionalmente e statisticamente.

Se le feature sono continue, non si può costruire una tabella. L'approccio 'ideale' è modellare P(X=Y=k) come una distribuzione gaussiana multivariata -> **N($\boldsymbol{\mu_k}, \sum_{\boldsymbol{k}}$).**

L'approccio citato prima viene chiamato **analisi discriminante quadratica QDA**, ed è il modello gaussiano più flessibile.

Ipotizzando che ogni classe k ha il suo vettore medio $\boldsymbol{\mu_k}$ e la sua matrice di covarianza completa $\sum_{\boldsymbol{k}}$, si avrà una dima dei parametri pari a $\frac{\boldsymbol{d^2}}{\boldsymbol{2}}$ con d caratteristiche di $\sum_k$.

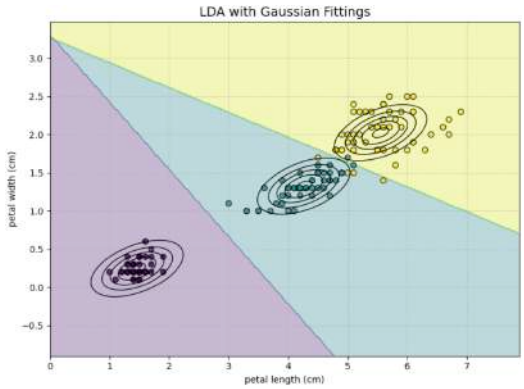
Ciò vuol dire che per ogni classe k si dovranno stimare molti parametri. Ad esempio, per 100 caratteristiche la stima dovrà essere di $\frac{100^2}{2} = \frac{10000}{2} = 5000$ parametri, un incubo computazionalmente.



Si stima un confine decisionale potente e curvo tra le classi, e ogni classe ha la propria distribuzione gaussiana.

Una prima semplificazione è l'**analisi discriminante lineare LDA**.

Si semplifica supponendo che tutte le classi abbiano la stessa matrice di covarianza(stessa distribuzione), così stimando una sola matrice $\sum$ (cambiano solo le medie, che le centrano in punti diversi). Questo permette semplificazioni matematiche e il confine della decisione è lineare.



Il confine decisionale è sempre lineare e l'approccio è più robusto e meno sensibile a overfitting.

Rimane il problema, nelle due analisi attraverso distribuzioni, della maledizione della dimensionalità, sebbene meno grave nell'LDA rispetto alla QDA, dato che si stima solo una matrice di covarianza. Ad esempio, si hanno d = 100 feature e k =10 classi k, allora:

- per l'LDA il numero di parametri da stimare è $\frac{100^2}{2} = \frac{10000}{2} = 5000.$
- per QDA il numero di parametri da stimare è $10 * \frac{100^2}{2} = \frac{100000}{2} = 50000.$

Naive Bayes

Per non incorrere nella maledizione della dimensionalità, si può ricorrere ad una semplificazione finale, ossia il **Bayes Ingenuo**.

Sappiamo he P(X|Y) = P($X_1, \ldots, X_n|Y$) = P($X_1|Y$)*P($X_2|X_1, Y$)* P($X_3|X_2, X_1, Y$) * P($X_n|X_{n-1}, \ldots, X_1, Y$) dato che X = ($X_1, X_2, \ldots, X_n$).

Il classificatore Naive Bayes parte dal presupposto ingenuo, ossia:

- **tutte le caratteristiche X_i sono condizionalmente indipendenti data la classe Y = k.**

Il modello, quindi, fa finta che ogni caratteristica sia soltanto dipendente da Y e non da tutto il resto. Questa assunzione in realtà e falsa, ma è una condizione di partenza per semplificare la ricerca della soluzione. Funziona in questo modo:

- realtà -> solitamente in un'e-mail contenente la parola "volo" , saranno contenute quasi sicuramente le parole "bagaglio", "partenza", "arrivo" o "check-in";
- ipotesi ingenua -> se in un'e-mail è contenuta la parola "volo", le probabilità che saranno presenti le parole "bagaglio", "partenza", "arrivo" o "check-in" non aumenteranno.

In generale non è vero, ma questo permette di moltiplicare le probabilità singole, dato che sono eventi indipendenti:

$$P(volo|viaggio) \times P(bagaglio|viaggio) \times P(partenza|viaggio) \times P(arrivo|viaggio)$$

Quindi la formula si semplificherà in:

$$\boldsymbol{P(X_1, \ldots, X_n|\, Y) \;=\; P(X_1|Y)P(X_2|Y)\ldots P(X_n|Y) \Leftrightarrow X_1 \perp X_2 \perp \ldots \perp X_n|Y}$$

La regola di classificazione MAP si potrà scrivere come:

$$\textbf{f(x) = } \boldsymbol{arg\ max\ \ P(x_1|Y = k)P(x_2|Y = k)\ldots P(x_n|Y = k)P(Y = k)}$$

Adesso, possiamo modellare questi termini in modi diversi. Gli approcci comuni consistono nell'utilizzare una distribuzione gaussiana o una distribuzione multinomiale.

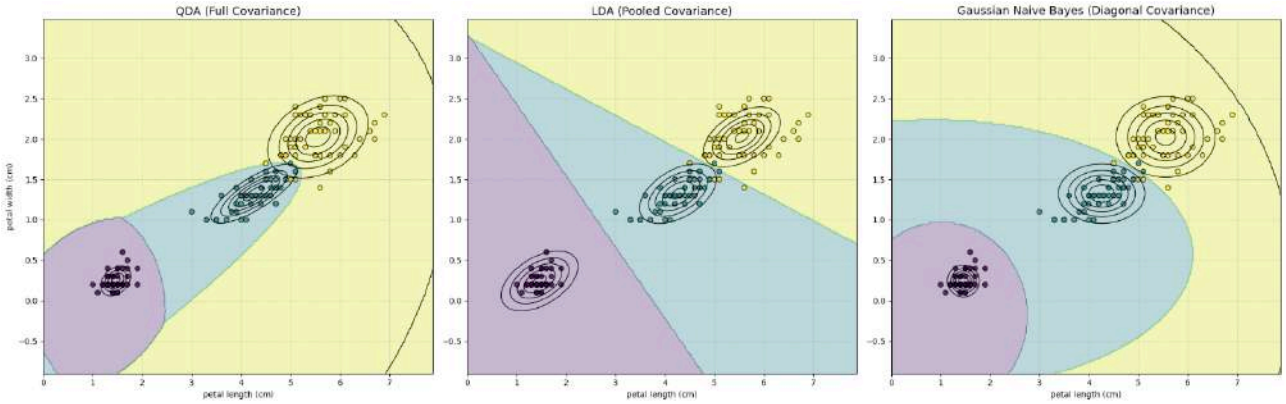
Bayes Ingenuo Gaussiano(Gaussian Naive Bayes)

Il modello gaussiano di Bayes presuppone che le caratteristiche siano indipendenti tra loro, data l'etichetta di classe. Ciò porta ad una matrice di covarianza diagonale per ciascuna classe $\sum_k$ e quindi se si hanno d caratteristiche si dovranno trovare d parametri(quelli lungo la diagonale).

$$\sum_k = \begin{matrix} \boldsymbol{\sigma_1^2} & \boldsymbol{0} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{\sigma_2^2} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{\sigma_3^2} \end{matrix}$$ per 3 feature si sono trovati 3 parametri.

Questo fa si che ogni feature contribuisca in modo indipendente alla verosimiglianza. Nonostante ciò, le matrici prodotte per ciascuna classe sono diverse quindi per n classi dovranno essere prodotte n matrici.

Inoltre, il modello non è capace di modellare le interazioni tra le caratteristiche all'interno di una classe, e per questo motivo che l'ipotesi ingenua è fortemente limitata.



- QDA è il più flessibile, studiando la distribuzione per ogni classe e i confini che ci sono tra le caratteristiche;
- LDA utilizza per tutte le classi la stessa varianza, riducendo il numero di parametri da calcolare e utilizza confini lineari(un buon compromesso);
- GNB il più semplice, studia un'ellisse per classe, ma non analizza le correlazioni che ci sono tra le caratteristiche.

Bayes Ingenuo Multinomiale(Multinomial Naive Bayes)

Quado le caratteristiche di input sono discrete, dobbiamo usare una versione di Naive Bayes dalla Gaussian Naive Bayes(tratta valori discreti).

Un valido sostituto è la Naive Bayes con una distribuzione multinomiale, dato che modella la probabilità di ottenere $(x_1, \ldots, x_d)$ occorrenze per ciascun dei d risultati.

Utilizzando la formula della distribuzione multinomiale, la verosimiglianza si può scrivere come:

$$\boldsymbol{P(x|Y = k) = \frac{(\sum_{i=1}^{d} x_i)}{x_1! \ldots x_d!} \cdot p_{k_1}^{x_1} \cdot p_{k_2}^{x_2} \cdot \ldots \cdot p_{k_d}^{x_d}}$$

Nella classificazione MAP, possiamo scrivere:

$$h(x) = argmaxP(Y = k) \cdot \frac{(\sum_{i=1}^{d} x_i)}{x_1! \ldots x_d!} \cdot \prod_{i=1}^{d} (p_{k_i})^{x_i}$$

Il termine fattoriale grande$\frac{(\sum_{i=1}^{d} x_i)}{x_1! \ldots x_d!}$ viene rimosso dalla formula, perché dipende esclusivamente dal vettore degli input x e non da k(costante positiva moltiplicativa). Quindi siccome si applica a tutte le classi ed è uguale, possiamo rimuoverlo e ottenere:

$$\boldsymbol{h(x) = arg\ max\ P(Y = k) \cdot \prod_{i=1}^{d} (p_{k_i})^{x_i}}$$

dove:

- $p_{k_i} = \frac{numero\ ricorrenze\ i\ in\ classe\ k}{totale\ ricorrenze\ in\ classe\ k}$ la probabilità di avere i;
- p(Y = k) =$\frac{\sum_j [y^{(j)} = k] x_i}{N}$ probabilità di avere classe k;

Se usiamo però le formule sopra, in classificazioni ad esempio riguardanti un'e-mail, potremmo avere un errore del tipo:

- probabilità di 0 per un nuovo termine, es. 'città', non presente in fase di allenamento: questo potrebbe portare ad avere 0 al numeratore e quindi probabilità = 0. Per risolvere questo problema si finge di aver visto almeno una volta ogni parola del vocabolario, questo si traduce in questo modo:
    - $\boldsymbol{p_{k_i}} = \frac{(\sum_j [\boldsymbol{y}^{(j)} = \boldsymbol{k}] \boldsymbol{x}_i^{(j)}) + \boldsymbol{1}}{(\sum_{l=1}^{d} \sum_j [\boldsymbol{y}^{(j)} = \boldsymbol{k}] \boldsymbol{x}_l^{(j)}) + |\boldsymbol{V}|}$ dove |V| è la dimensione del vocabolario, e si mette per mantenere la somma delle probabilità uguale a 1.
- Underflow aritmetico: dato che le probabilità sono molto piccole, si utilizzano le probabilità logaritmiche e il risultato sarà:
    - $\boldsymbol{h(x) = arg\ max\ \log P(Y = k) \cdot \sum_{i=1}^{d} x_i} \cdot$

RAPPRESENTAZIONE DATI

Nei capitoli precedenti si è posto l'interesse maggiormente sull'interpretazione dei valori di ciascuna variabile. Tuttavia, è anche utile vederli dal punto di vista dell'interpretazione geometrica, vedendo le osservazioni come dei **punti dati d-dimensionali**.

Non si assegna un significato esplicito alle variabili, e quindi ciascuna variabile viene indicata come una **caratteristica dei dati**(non ha un significato semantico o "umano", ma solamente numerico).

Possiamo quindi parlare di estrazione delle caratteristiche, ossia $x \in \mathfrak{N}^d$ si trasforma in $y \in \mathfrak{N}^m$. Esiste una funzione:

$$\boldsymbol{f: \mathfrak{N}^d \rightarrow \mathfrak{N}^m}$$

detta funzione di estrazione di feature(funzione di rappresentazione).

$\mathfrak{N}^d$ e $\mathfrak{N}^m$ sono due "spazi di feature" o "spazi di rappresentazione", ossia due spazi vettoriali a cui appartengono i punti dati.

Le norme sono funzioni che misurano la lunghezza di un vettore e dicono quanto è grande un vettore nello spazio. Quindi dato uno spazio vettoriale S, una norma è una funzione che restituisce un numero reale non negativo associato alla grandezza del vettore.

$$\boldsymbol{p: S \rightarrow \mathcal{R}_0^+}$$

Una famiglia di norme utilizzata in analisi dei dati è la famiglia delle norme L_p:

$$|\boldsymbol{x}|_p = \left(\sum_{i=1}^{d} |x_i|^p\right)^{1/p}$$

Attraverso questa funzione calcoliamo quanto un vettore è lontano dall'origine.

Abbiamo varie forme delle norme L_p, ossia:

- **Norma L_1:** $||\boldsymbol{x}|| = \sum_{i=1}^{d} |x_i|$ prende soltanto la somma dei valori assoluti -> la formula estesa sarebbe $||\boldsymbol{x}|| = \left(\sum_{i=1}^{d} |x_i|^1\right)^{1/1}$.
- **Norma L_2:** $||\boldsymbol{x}|| = \sqrt{\sum_{i=1}^{d} x_i^2}$ prende la radice quadrata della somma quadratica dei valori assoluti -> penalizza valori maggiori.
- **Norma L_∞:** $||\boldsymbol{x}|| = \boldsymbol{max}(x_i)$ prende il valore maggiore, dato che calcolando il limite a +∞ tutto converge verso il massimo.

x = (1,2,5)

- $L_1 = 1 + 2 + 5 = 8$
- $L_2 = \sqrt{1 + 4 + 25} = \sqrt{30} \approx 5.48$
- $L_\infty = \max(1, 2, 5) = 5$

Come possiamo osservare dall'esercizio, aumentando il valore di p, il risultato si avvicinerà sempre di più al valore massimo.

Dato uno spazio S, una funzione:

$$\boldsymbol{m}: \boldsymbol{S} \times \boldsymbol{S} \ \rightarrow \Re$$

è una **metrica**. Le proprietà che devono essere soddisfatte sono:

- La **distanza di un punto da sé stesso è zero**: m(x,x) = 0;
- La **distanza tra due punti è sempre positiva**;
- La distanza tra due punti m =(x,y) è uguale alla distanza m = (y,x) -> **simmetria**;
- **Disuguaglianza triangolare**, ossia **m(x,y)** $\leq$ **m(x,z)+(z,y)** $\forall\ \boldsymbol{x}, \boldsymbol{y}, \boldsymbol{z} \in \boldsymbol{S}$;

Le **metriche L_p** sono delle metriche ricavate dalle norme L_p e la famiglia delle metriche è la seguente:

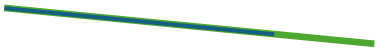
$$\boldsymbol{L_p(x,y)} \ = \ ||\boldsymbol{x}-\boldsymbol{y}||_{\boldsymbol{p}} \ = \ (\sum_{\boldsymbol{i=1}}^{\boldsymbol{d}} |\boldsymbol{x_i}-\boldsymbol{y_i}|^{\boldsymbol{p}})^{1/p}$$

A partire da questa famiglia di metriche abbiamo le varie distanze, ossia:

- **Distanza di Manhattan** ← Distanza L_1;
- **Distanza euclidea** ← Distanza L_2;

Il problema della distanza euclidea è che misura quanto sono lontani due punti nello spazio. Anche se i due vettori puntano nella stessa direzione si conta solo la grandezza dei due vettori.

$Confronto\ tra\ grandezze$:

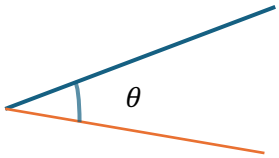


Ad esempio, se si hanno due commenti:

- Amo la pizza;
- Amo la pizza Amo la pizza Amo la pizza;

Il secondo documento è la versione più lunga del primo avendo una semantica identica.

Confronto tra direzioni:



Se vogliamo vedere la differenza semantica(nel caso del testo) si confronta l'angolo tra i vettori e non le grandezze. Questo confronto è possibile attraverso la **similarità del coseno**, e la formula è la seguente:

$$\boldsymbol{S_C} = \boldsymbol{Cos(\theta)} \ = \ \frac{\boldsymbol{x} \cdot \boldsymbol{y}}{|\boldsymbol{x}| \cdot |\boldsymbol{y}|}$$

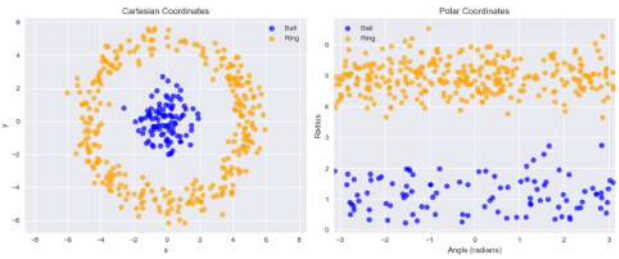
La similarità del coseno è un numero normalizzato tra -1 e 1, dove:

- 1 massimo allineamento;
- -1 vettori opposti;
- 0 indipendenza tra i due vettori.

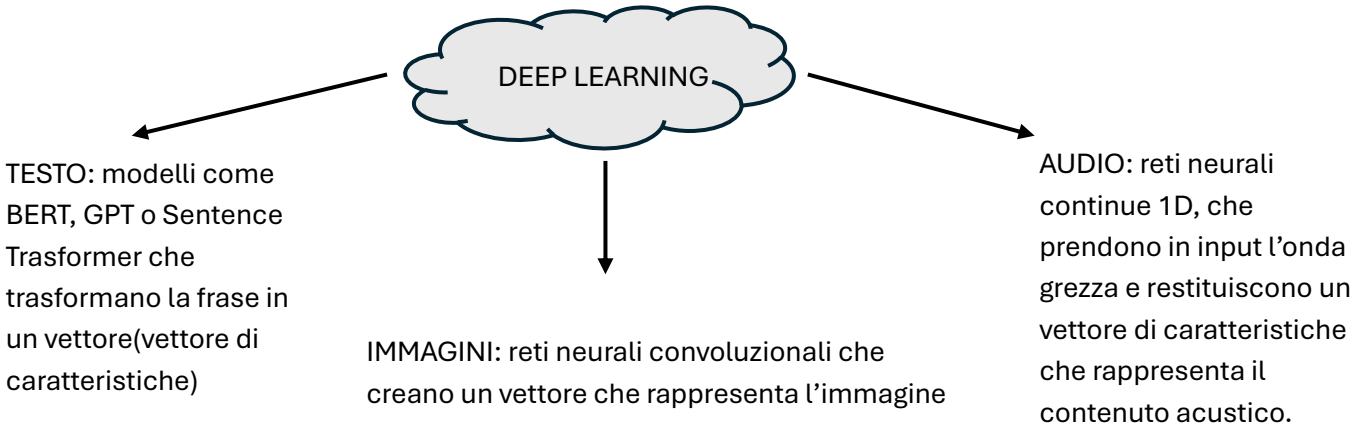
La dis**tanza del coseno** si calcola attraverso la seguente formula:

$$\boldsymbol{D_C} \ = \ \boldsymbol{1} \ - \ \boldsymbol{S_C}$$

Cambiare la rappresentazione dei dati permette di cambiare il modo di vedere un problema(non il problema stesso). Ciò vuol dire che un problema apparentemente difficile in uno spazio diventa molto facile da risolvere in un altro. Ad esempio, il cambio da **coordinate cartesiane** a **coordinate polari** semplifica il problema proposto nell'esempio.



Per dati complessi non si può manualmente scoprire la funzione caratteristica perfetta, per questo motivo entra in gioco l'**apprendimento automatico**(DEEP LEARNING). Ossia, che si apprende la funzione di rappresentazione f dai dati stessi.



La **disuguaglianza nell'elaborazione dei dati**(DPI -> **data process inequality**) afferma che l'applicazione di qualsiasi trasformazione dei dati, non può mai aumentare il contenuto informativo, ma può solo perdere o preservare le informazioni.

Questo principio ha profonde implicazioni per l'ingegneria delle funzionalità, dato che:

- se si perde un'informazione cruciale, al livello sottostante, l'informazione è persa in modo permanente;
- una progettazione scadente delle funzionalità crea un collo di bottiglia delle informazioni;
- si devono preservare le informazioni più rilevanti per il compito;

La **qualità dell'analisi**(clustering incluso) è **determinata molto più dalla rappresentazione dei dati e dalla metrica scelta** che dall'algoritmo in sé.

Ad esempio, possiamo applicare il KNN su immagini rappresentate da:

- **Raw Pixels ->** ogni immagine è vista come un vettore di intensità dei pixels e le feature non hanno un significato semantico. Si confrontano i valori dei pixel e quindi piccole variazioni significano grandi distanze. Due immagini identiche(a meno di colori ad esempio) possono risultare lontane nello spazio.
- **CNN features ->** trasforma pixel in **feature astratte** -> catturano: **bordi e texture**, **parti di oggetti** e **concetti semantici** -> le distanze nello spazio delle feature misurano la similarità semantica e non la distanza tra pixel. Immagini diverse a livello pixel ma simili a livello concettuale risultano essere vicine.

L'utilizzo di un modello piuttosto che di un altro porterebbe:

- **primo caso(raw pixels)** -> il clustering troverebbe K = 4 gruppi, dato che secondo il raw pixels le 4 immagini rappresentano cose diverse;
- **secondo caso(CNN features)** -> il clustering troverebbe K = 1 gruppi, dato che secondo il CNN feature le immagini rappresentano cose molto simili(bordi e concetti semantici uguali).

CLUSTERING

Con dati molto complessi potrebbe essere difficile comprendere la struttura sottostante dei dati. Un approccio per farlo è attraverso il **raggruppamento(clustering)** in cluster contenenti punti con caratteristiche simili. Il raggruppamento è molto utile e sta alla base di operazioni come:

- **segmentazione della clientela** -> raggruppamento clienti in base a loro preferenze o comportamenti d'acquisto;
- **segmentazione dell'immagine** -> suddivisione dell'immagine in segmenti o regioni significative in base alla somiglianza -> facilita il riconoscimento;
- **elaborazione vocale e audio** -> classificazione audio o identificazione degli oratori o analisi del parlato sono esempi di questo tipo di applicazione;
- **commercio elettronico** -> raggruppamento di prodotti o utenti in base a modelli di acquisto per ottimizzare la gestione dell'inventario e migliorare l'esperienza utente.

Un algoritmo di clustering mira a trovare automaticamente gruppi e ad assegnare ciascun punto dati al gruppo più probabile. Quindi, partendo da X l'obiettivo è quello di dividere X in K gruppi chiamati cluster.

Il che significa che l'algoritmo non assegnerà semantico ai cluster, ma si occuperà di inserire punti dati simili nello stesso cluster.

Se **$S = \{S_1, S_2, \ldots, S_n\}$** è l'**insieme dei gruppi** che creeremo e **$x^{(i)}$** una singola osservazione delle N osservazioni abbiamo che:

$$\forall \boldsymbol{x}^{(i)} \in \boldsymbol{X}\ \exists!\ \boldsymbol{S_j} \in \boldsymbol{S} :\ \boldsymbol{x}^{(i)} \in \boldsymbol{S_j}$$

Dove $\exists!$ significa esiste uno e uno solo.

I cluster contengono al proprio interno solo punti simili tra loro. Punti diversi saranno contenuti in altri cluster.

K-Means tenta di creare K cluster il più possibile compatti e K è un parametro specificato dall'utente(o ottenibile mediante algoritmi[elbow, silhouette score]).

Si inizia definendo i centri del cluster(**centroidi**), ossia K vettori diversi $\mu_K \in \Re^n$. Si definiscono poi N×K variabili binarie così come segue:

$$\boldsymbol{r_{ij}} = \begin{cases} \boldsymbol{1\ se\ x^{(i)} \in S_i} \\ \boldsymbol{0\ otherwise} \end{cases}$$

dove i è il numero dell'osservazione e j è il cluster.

Si definisce una funzione di costo, detta anche funzione di distorsione, che rende compatta più compatti i cluster:

$$\boldsymbol{J} = \sum_{j=1}^{K} \sum_{i=1}^{N} \boldsymbol{r_{ij}} ||\boldsymbol{x_i} - \boldsymbol{\mu_j}||^2$$

La **funzione J** è la **somma totale delle distanze quadratiche dei punti dai centroidi dei cluster**(WCSS varianza interna ai cluster).

L'**obiettivo** del K-Means è quello di trovare l'**insieme dei centroidi $\{u_j\}$** e delle **assegnazioni dei punti$\{r_{ij}\}$** ai cluster che **minimizza l'inerzia totale J**.

Quindi:

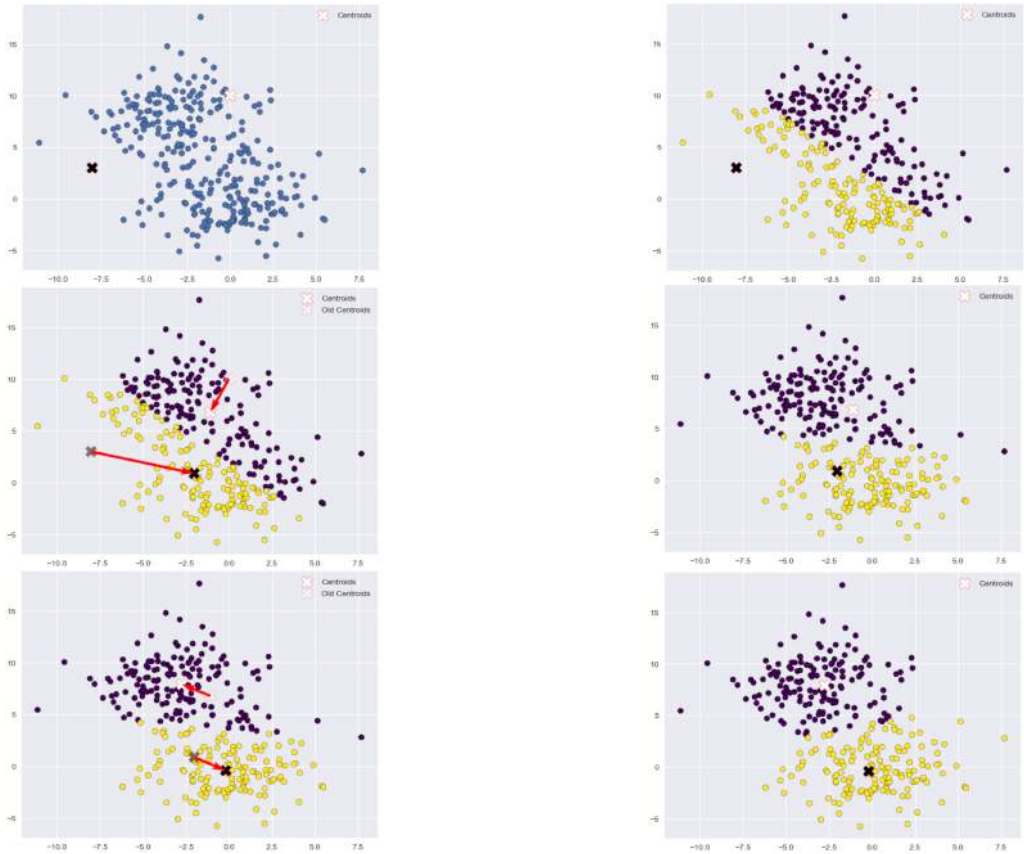
$$\{\hat{r}_{ij}\},\{\hat{\mu}_j\} = arg\ min\ J = arg\ min \sum_{j=1}^{K} \sum_{i=1}^{N} r_{ij}||x_i - \mu_j||^2$$

È necessario che ogni volta che si aggiunge un elemento ad un cluster si aggiornino i centroidi attraverso la media aritmetica ad esempio(ci sono tanti modi per aggiornare il centroide).

Quindi abbiamo che:

$$\mu_j = \frac{1}{|S_j|} \sum_{x \in S_j} x$$

L'algoritmo converge quando l'aggiornamento non modifica alcun centroide. C'è la remota possibilità che non terminino questi algoritmi, per questo motivo si introduce un criterio di terminazione, ossia un numero massimo di interazioni.



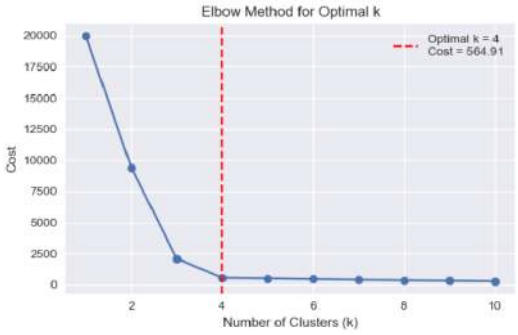
La procedura di ottimizzazione termina quando i centroidi non si sono spostati.

Nell'algoritmo di clustering K-Means, K è un iper-parametro, cioè un valore che non è determinato automaticamente dalla procedura di ottimizzazione e influenza il modello finale che otteniamo.

Per determinare i valori degli iper-parametri si potrebbero fare delle ipotesi e adattare i valori ipotizzati, infine si sceglie il valore migliore. Come citato in altre materie, per determinare il valore di K si utilizzano due tecniche principali.

Le tecniche principali per determinare K sono:

- **metodo del gomito**: consiste nel calcolare il costo e verificare quando diminuisce in funzione di K. Se k è molto grande ovviamente ha un costo minore(**WCSS**) dato che ogni punto è molto vicino al proprio centroide. Si sceglie un K, la cui funzione di costo rallenti la velocità di diminuzione, e generalmente il punto è chiamato **punto del gomito**.



- **metodo silhouette**: consiste nel valutare quanto bene i dati siano raggruppati nei cluster. Se il modello è buono ci si aspetta che ogni punto si adatti bene al proprio cluster e non si adatti bene a nessun altro cluster. Per questo motivo si calcola **la distanza media del punto i con qualsiasi altro punto all'interno del cluster**:

$$a(i) = \frac{1}{|C_I| - 1} \sum_{j \in C_I \, con \, j \neq i} d(i,j)$$

dove d(i,j) è la distanza euclidea tra i punti dati i e j.
Poi si calcola la **distanza media tra i punti di tutti i cluster restanti con il punto i** e si prende il valore minimo:

$$b(i) = min \frac{1}{|C_J|} \sum_{j \in C_j} d(i,j)$$

Infine, si confrontano i due valori attraverso la seguente regola:

$$silhouette(i) = \frac{b(i) - a(i)}{max\{a(i), b(i)\}}$$

Se il valore è:
  - **vicino a 1**: punto nel posto perfetto;
  - **vicino a 0**: zona di confine;
  - **vicino a -1**: punto nel gruppo sbagliato.

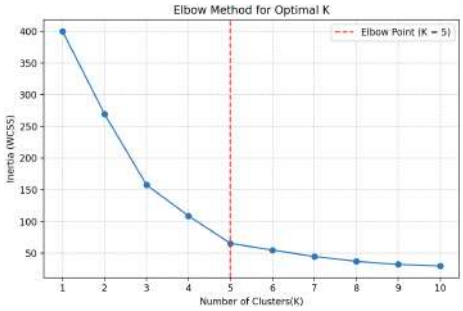
Calcolando il silhouette per ogni punto e sommando a tutto il resto si ottiene un valore chiamato **silhouette score** che indica quanto il nostro clustering raggruppa bene. Si prende il valore di K che massimizza il silhouette score.

Esistono vari algoritmi di clustering oltre al K-Means, che offrono prospettive diverse e affrontano sfide specifiche. Ci sono svariati tipi di clustering:

- **clustering gerarchico** → punto di partenza dove ogni punto dato corrisponde ad un cluster e procedendo si fondono i cluster tra loro(in base alla vicinanza tra di loro);
- **DBSCAN(Density-Based Spatial Clustering of Applications with Noise)** → zone dense di punti saranno viste come cluster del dataset;
- **clustering spettrale** → interpreta i dati come un grafo pesato e individua i cluster analizzando la struttura degli archi del grafo(in prossimità di archi più deboli si taglia il grafo).

In python, il clustering(KMeans) si applica attraverso la funzione ‘KMeans’ contenuta all’interno della libreria ‘sklearn’. Come spiegato precedentemente il valore di k si può trovare in due modi:

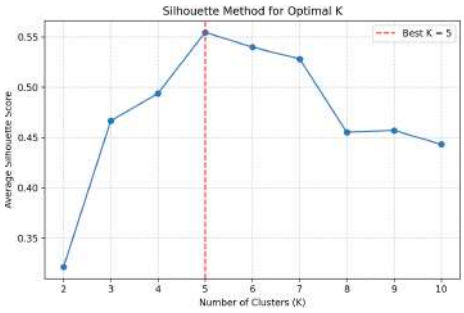
METODO ELBOW




```
k_values = range(1,11)
inertias = []

for k in k_values:
    kmeans = KMeans(n_clusters=k, random_state=42,
n_init=10)
    kmeans.fit(X_scaled)
    inertias.append(kmeans.inertia_)
```

Infine, si prende il valore di rottura, ovvero dove cambia la velocità con cui sale o scende il valore dell’inerzia.

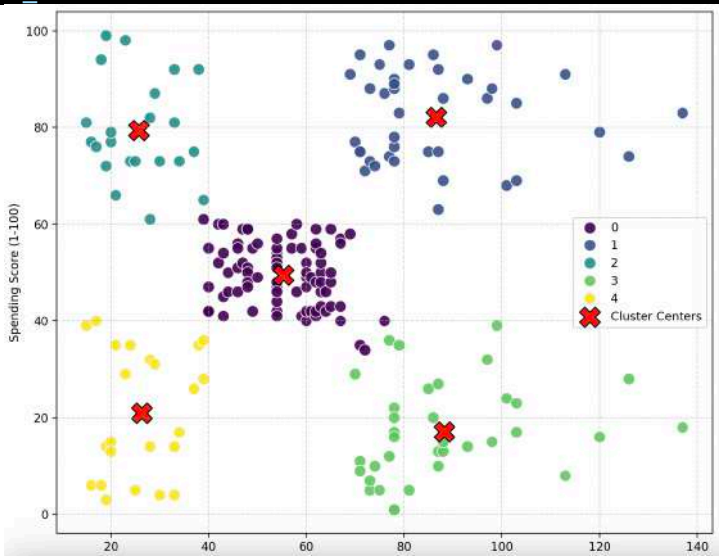
METODO SILHOUETTE




```
k_values_sil = range(2,11)
silhouette_scores = []

for k in k_values_sil:
    kmeans = KMeans(n_clusters=k, random_state=42,
n_init=10)
    labels = kmeans.fit_predict(X_scaled)

    score = silhouette_score(X_scaled,labels)
    silhouette_scores.append(score)
best_k_sil =
k_values_sil[np.argmax(silhouette_scores)]
```

Si calcola il valore di score per ogni k e poi si prende il numero di cluster che produce un punteggio di score più alto

```
final_kmeans = KMeans(n_clusters=best_k_sil,random_state=42, n_init=10)
final_labels = final_kmeans.fit_predict(X_scaled)

final_centers_scaled = final_kmeans.cluster_centers_

df['Cluster'] = final_labels
```



STIMA DELLA DENSITA'

Gli algoritmi di clustering sono utili a estrarre la struttura sottostante ai dati, in particolare il k-means permette di identificare parti dello spazio delle caratteristiche con elevata densità.

Un approccio alternativo sarebbe quello di studiare la probabilità di X, ossia dove sono i dati. Infatti, potremmo scoprire naturalmente dei cluster nei dati, ossia zone con valori di probabilità elevati.

Una densità di probabilità è una funzione continua, e quindi non è semplice stimarla come una funzione di massa di probabilità(solo per interi).

Esistono due metodi per la stima della densità:

- **metodi non parametrici**: provano a stimare la densità direttamente dai dati *senza ipotesi* sulla distribuzione della sorgente. Si devono gestire pochi iper-parametri ma sebbene sia possibile calcolare il valore, non è possibile conoscere la forma analitica di f.
- **metodi parametrici**: metodi che mirano ad adattare una distribuzione nota ai dati, ad esempio l'*adattamento di una gaussiana* ai dati. Il vantaggio è legato alla forma analitica di f restituita dai metodi, ma si devono fare delle ipotesi sulla natura dei dati.

I metodi non parametrici che stimano la densità direttamente dai dati seguono due approcci:

- **Fixed Windows**(Istogrammi): metodo che discretizza lo spazio delle caratteristiche in una griglia di contenitori e conta i punti che rientrano all'interno di ciascun contenitore;
- **Mobile Windows**(stima della densità del Kernel -> KDE): centra una "finestra" su ogni punto dati e ne somma i contributi di tutti i kernel. Il risultato dipende dal kernel scelto e dal parametro di banda(bandwidth). Approccio noto come **metodo delle finestre di Parzen**.

Il metodo più semplice è quello delle finestre fisse. Un semplice conteggio è fuorviante, dato che un contenitore 4 volte più grande potrebbe contenere 4 volte più punti ma avere la stessa densità di un altro contenitore.

La funzione deve soddisfare la regola:

il volume totale sotto la funzione deve essere integrato a 1 -> $\int \boldsymbol{f}(x)dx = 1$

La formula per calcolare la densità di un contenitore R_i è la seguente:

$$\boldsymbol{f(x)} = \frac{\boldsymbol{P(R_i)}}{\boldsymbol{V(R_i)}}$$

con:

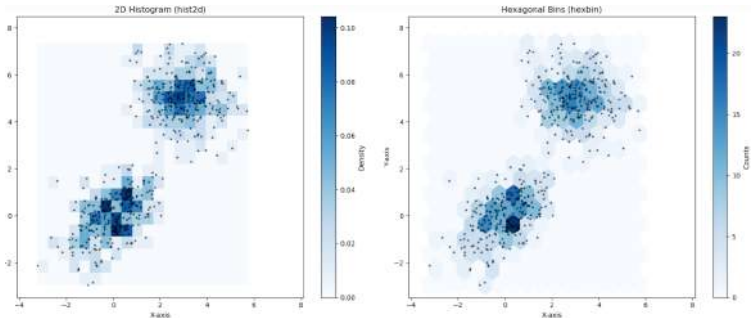
$\boldsymbol{P(R_i)}$: probabilità del contenitore R_i;

$\boldsymbol{V(R_i)}$: volume del contenitore;

Ciò serve a garantire che un contenitore piccolo e affollato abbia densità più alta di uno grande e sparso. Gli iper-parametri di quest'approccio sono:

- dimensione del contenitore;
- origine della griglia;

Inoltre, anche la forma del contenitore permette di stimare in modo preciso o meno la densità.



Gli esagoni piastrellano in modo più naturale lo spazio e il grafico appare molto più fluido rispetto all'utilizzo di contenitori quadrati.
Il problema principale è che la dimensione del bin è un parametro difficile da scegliere.
Anche se l'abbiamo visto in 2D, si può generalizzare a N dimensioni.

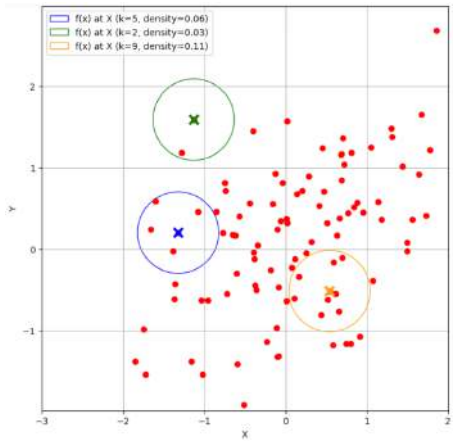
Un approccio non parametrico più flessibile e potente sta nell'utilizzare una finestra mobile.

Questo metodo, anche noto come **Kernel Density Estimation(KDE)**, e prevede che invece di posizionare una finestra fissa e vedere quanti punti ci sono all'interno di questa finestra, posiziona una finestra per ogni punto dato.

L'**approccio 'NAIVE'** prevede che si centri un cerchio di raggio fisso h su un punto x e che si calcoli la densità secondo i punti che ricadono nel **cerchio N(x,h)**.

La funzione sarà quindi:

$$f(x) = \frac{|N(x,h)|/N}{V(h)}$$



Si può anche definire come:

$$f(x) = \frac{1}{|X|}\sum_{i=1}^{|X|} K_h(x_i - x)$$

Norma euclidea

dove:

$$K_h(x_i - x) = \begin{cases} \frac{1}{V(h)} \; if \; \left\|\frac{x_i - x}{h}\right\|_2 \leq 1 \\ 0 \; otherwise \end{cases}$$

K_h è una funzione del kernel che dipende dal parametro h, ossia il parametro della larghezza di banda. Le parentesi indicano che si calcola la distanza attraverso la norma euclidea e possiamo notare che se l'elemento è all'interno del cerchio allora la distanza normalizzata(attraverso la divisione) varia tra -1 e 1. Se l'elemento è fuori dal cerchio, avrà una distanza maggiore di 1.

Il problema principale è dato dall'elevata sensibile alla posizione in cui si calcola la densità, infatti, se spostassimo il cerchio di poco potremmo avere densità diverse.

Inoltre, il kernel K prende "decisioni difficili" dato che un punto può essere:

- 100% fuori e ottenere un voto nullo;
- 100% dentro e ottenere un voto pieno;

Quindi, spostando anche se di poco il centro x, un punto dati potrebbe uscire dal cerchio.

La soluzione è usare un kernel liscio che assegna un peso graduale, in modo tale che al crescere della distanza il peso decresce.

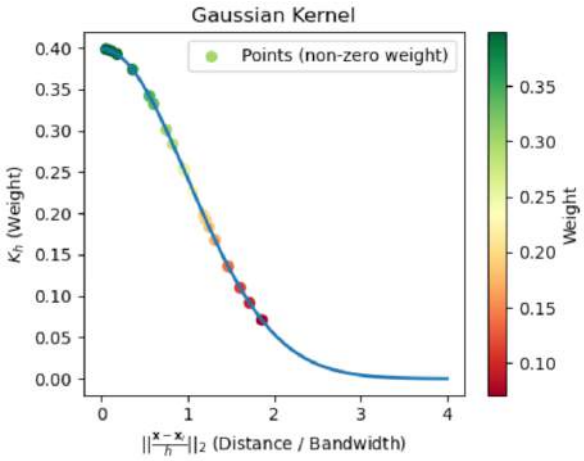
Un possibile kernel liscio è il kernel gaussiano, la cui formula è la seguente:

$$K_h(x_i - x) \propto exp(-\frac{||x_i - x||^2}{2h^2})$$

Esponenziale per ottenere la tipica forma a campana

Attraverso questa funzione i punti più vicini contribuiscono di più rispetto a punti lontani. La formula generale per la stima della densità del kernel è la somma dei kernel:

$$f(x) = \frac{1}{N}\sum_{i=1}^{N} K_h(x_i - x)$$



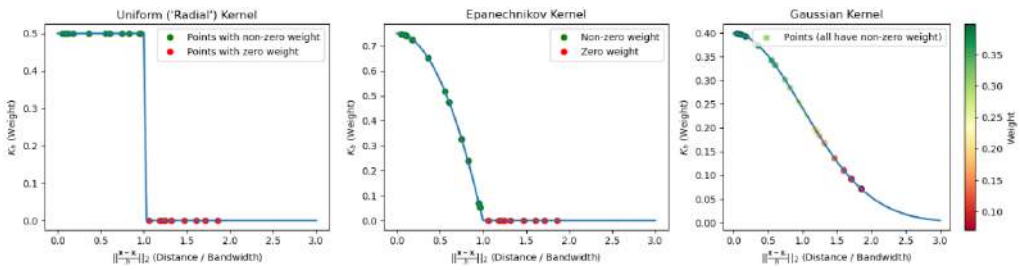
Il kernel Gaussiamo risolve il problema della sensibilità, ma è molto costoso da calcolare su dataset molto ampi. Un'alternativa è impostare a zero tutti gli elementi sufficienti distanti. Questo è ciò che fa il kernel Epanechnikov, definito come:

$$K_h(x_i - x) = \frac{3}{4h^2}(1 - \frac{\|x_i - x\|^2}{h^2})\mathbb{I}(\|x_i - x\| \leq h)$$

Dove:

$$\mathbb{I}(x) = \begin{cases} 1 \; se \; x \; is \; true \\ 0 \; otherwise \end{cases}$$

Quindi prende in considerazione tutti i punti dati contenuti all'interno del cerchio e esclude quelli fuori. A differenza dei kernel circolari da un peso maggiore ai punti dati vicini al centro e un peso minore ai punti dati più lontani.



La larghezza di banda è l'iper-parametro più importante nella stima della densità, dato che controlla il compromesso tra bias e varianza. Un raggio piccolo introduce alta varianza ma basso bias e si adatta eccessivamente ai dati di training, viceversa un valore h molto alto.

I metodi parametrici mirano a stimare la densità adattando un modello parametrico ai dati(es. Gaussiano).

Questo come citato prima ha dei vantaggi come:

- Formulazione analitica su cui si può ragionare(*f* è nota);
- Modello compatto, che una volta adattato, può essere anche utilizzato per prevedere i valori di densità;
- Vincoli imposti che possono aiutare nel caso in cui si hanno pochi dati.

Gli svantaggi sono:

- Tempo impiegato per trovare valori appropriati che consentono di spiegare i dati;
- Se il modello fa l'ipotesi sbagliata(gaussiana non si adatta) allora il modello non sarà molto accurato.

Il metodo parametrico più comune è quello di supporre che i dati seguano una distribuzione gaussiana multivariata.

La probability density function per una Gaussiana d dimensionale è:

$$\boldsymbol{N(x;\mu,\Sigma)} = \frac{\mathbf{1}}{\sqrt{(\boldsymbol{2\pi})^{d}\boldsymbol{det}(\boldsymbol{\Sigma})}}\boldsymbol{e}^{-\frac{\mathbf{1}}{\mathbf{2}}(x-\mu)^{T}\Sigma^{-1}(x-\mu)}$$

Dove:

- $\boldsymbol{\mu}$ **vettore medio**;
- $\boldsymbol{\Sigma}$ **matrice di covarianza**;

Si devono trovare μ e Σ che si adattano meglio ai dati. Quindi, si prende un set di dati X = $\{x_1, x_2, \ldots, x_n\}$ e si assume che la distribuzione sia Gaussiana.

Attraverso l'utilizzo del principio di Massima Verosimiglianza(ML), si cercano i parametri che massimizzano i valori di probabilità che la distribuzione gaussiana calcolerebbe per i dati forniti.

Si prova implicitamente l'intera famiglia di gaussiane e si sceglie quella che assegna la maggiore probabilità complessiva ai dati osservati, che viene quindi usata come modello per descriverli.

La formula della verosimiglianza è:

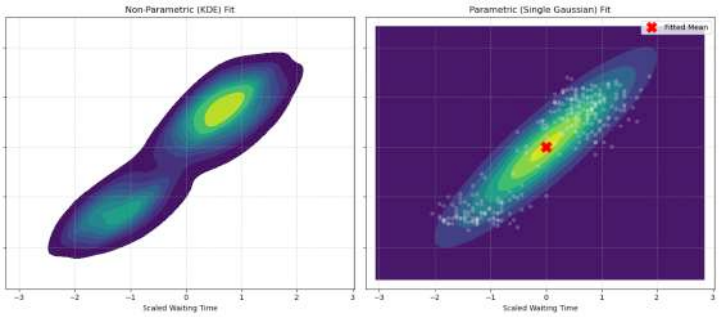
$$\boldsymbol{P(X|\mu,\Sigma)} = \prod_{\boldsymbol{i}} \mathbf{N}(\mathbf{x}_{\boldsymbol{i}}|\boldsymbol{\mu},\boldsymbol{\Sigma})$$

Si deve trovare la gaussiana specifica che massimizza la produttoria delle densità di probabilità di singoli punti. Le stime di massima verosimiglianza per i parametri gaussiani sono:

- La stima per la media è la media campionaria:
  - $\boldsymbol{\mu_{ML}} = \frac{\mathbf{1}}{\boldsymbol{N}}\sum_{\boldsymbol{i=1}}^{\boldsymbol{N}} \boldsymbol{x_i}$
- La stima per la covarianza è la matrice di covarianza del campione:
  - $\boldsymbol{\Sigma_{ML}} = \frac{\mathbf{1}}{\boldsymbol{N}}\sum_{\boldsymbol{i=1}}^{\boldsymbol{N}}(\boldsymbol{x_i} - \boldsymbol{\mu_{ML}})(\boldsymbol{x_i} - \boldsymbol{\mu_{ML}})^{\boldsymbol{T}}$

Sostanzialmente l'adattamento di una gaussiana è dire il calcolo della media e della varianza.

Il modello gaussiano singolo è molto potente, ma si basa sul presupposto che i dati provengono da un singolo cluster. Ciò vuol dire che se ci sono, ad esempio, due punti del dataset(distanti) molto popolati e attorno una concentrazione dei punti gaussiana cosa troviamo con una sola gaussiana.



In presenza di un **dataset bimodale**, cioè caratterizzato da due cluster distinti, l'utilizzo di una singola distribuzione gaussiana può portare a stimare un centro che si colloca in una regione con pochi o nessun dato, non rappresentativa della reale struttura del dataset. Possiamo vedere come il modello parametrico riesce a trovare correttamente i due cluster, al contrario del modello parametrico gaussiano.

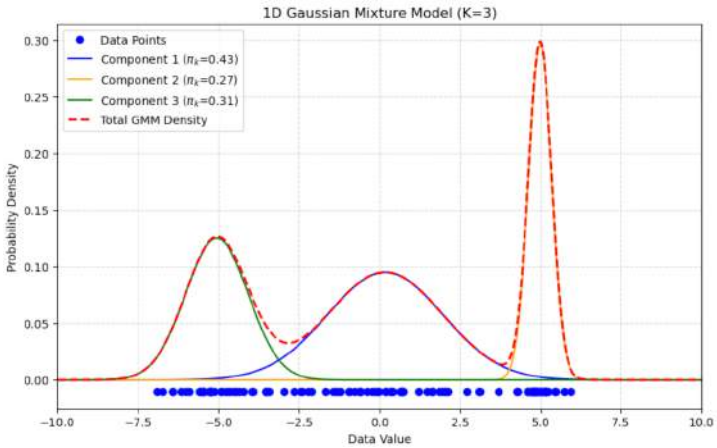
Un modello parametrico, usato su **dati multimodali**, più flessibile è il **Gaussian Mixture Model** (GMM -> modello di miscela gaussiana). Invece di cercare una campana che copra tutto, immaginiamo che i dati siano generati da una media ponderata di K diverse gaussiane.

La formula della GMM è la seguente:

$$\mathbf{P(x)} = \sum_{k=1}^{k} \boldsymbol{\pi_k} \boldsymbol{N}(\boldsymbol{x}|\boldsymbol{\mu_k}, \boldsymbol{\Sigma_k})$$

Dove:

- $\boldsymbol{\pi_k}$ peso di ogni componente di ogni componente -> la somma deve fare sempre 1;
- $\boldsymbol{\mu_k}$ ogni componente ha il suo centro(media locale);
- $\boldsymbol{\Sigma_k}$ ogni componente ha la sua forma e larghezza;



Il GMM può anche essere visto come una versione soft di K-Means dato che calcola la probabilità di assegnare un elemento ad un cluster.

Questo avviene introducendo una **variabile latente Z**, ossia una variabile che **non è osservabile** ma si assume che faccia **parte del processo generativo dei dati**.

In pratica Z = k indica il caso in cui X = x appartiene al cluster k. Quindi, per ogni punto dati x_i esiste una variabile nascosta z_i che indica quale delle K gaussiane ha generato quel punto. L'assegnazione soft consiste nel trovare:

$$p(z_i = k \mid x_i)$$

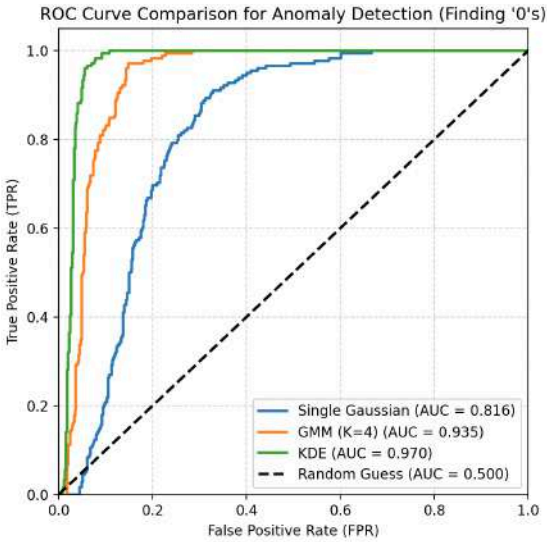
Per calcolare la probabilità a posteriori si fa come segue:

$$\Upsilon_k = P(Z = k|\, x) = \frac{\pi_k N(x \mid \mu_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j N(x|\mu_j, \Sigma_j)}$$

Per adattare il modello, si utilizza la stima di massima verosimiglia(MLE) proprio come si è fatto per la singola gaussiana.

Si vogliono trovare i parametri θ che massimizzano la verosimiglianza logaritmica dei dati osservati:

$$logP(X|\theta) = \sum_{i=1}^{N} log[\sum_{k=1}^{K} \pi_k N(x_i|\mu_k, \Sigma_k)]$$




```
model_single_gauss = GaussianMixture(n_components=1, random_state=42)
model_single_gauss.fit(X_train_scaled)

model_gmm = GaussianMixture(n_components=4,random_state=42)
model_gmm.fit(X_train_scaled)

model_kde = KernelDensity(kernel='gaussian')
model_kde.fit(X_train_scaled)
```

I tre modelli generano:

1) 1 gaussiana;
2) 4 gaussiane;
3) tante gaussiane quanti sono i dati;

RIDUZIONE DELLA DIMENSIONALITA'

ANALISI DELLE COMPONENTI PRINCIPALI(PCA)

Spesso i data set contengono migliaia di dati e migliaia o milioni di feature. Per questo motivo si hanno i seguenti problemi:

- maledizione della dimensionalità -> nello spazio ad alta dimensionalità tutto è distante e modelli basati sulla distanza falliscono;
- multicollinearità -> caratteristiche sono molto ridondanti;

Non si possono semplicemente utilizzare solo poche caratteristiche, invece si devono riassumere i dati. Questa è la riduzione della dimensionalità, che consiste nel trasformare i dati da d dimensioni a k funzionalità intelligenti, perdendo la minor quantità di informazioni possibile.

Il metodo utilizzato è la **PCA** o **Principal Component Analysis**.

Abbiamo visto in precedenti casi che quando le caratteristiche sono correlate, se ne devono scartare alcune per collinearità. Le variabili da scartare possono essere scelte attraverso:

- coefficienti alti di correlazione tra due variabili;
- p-value elevato;
- regressione Ridge e Lasso filtrano le feature impostando coefficienti bassi o nulli.

Invece di selezionare un sottoinsieme di funzionalità su cui lavorare, la tecnica di riduzione si occupa di riassumere l'insieme originale, perdendo solo una piccola quantità di informazioni.

Ad esempio, invece di rimuovere una dimensione che è collineare ad un'altra potremmo semplicemente effettuare una proiezione lineare dei dati in una singola dimensione.

La formula potrebbe essere la seguente:

$$z = \mu_1 x_1 + \mu_2 x_2 = \mu^T x$$

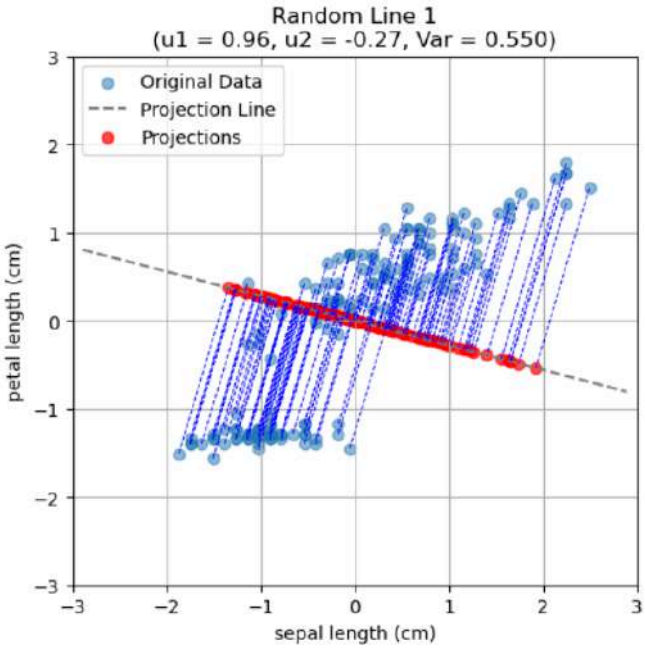
Come si può osservare si prende una migliore approssimazione in 1d del piano.

Si perde informazione spaziale dato che dalle coordinate (x,y) si passa a (z) -> perdendo:

- direzione;
- distanza tra i punti;
- forma distribuzione;

Vorremmo però proiettare i punti in modo che mantengano il più possibile la loro distintività.

Si può quantificare la distintività dei punti attraverso varianza dei dati proiettati.



Sia $\{x_n\}$ essere un insieme di N osservazioni e x_nè un vettore di dimensionalità D. L'analisi delle componenti principali è una proiezione dei dati su M dimensioni(con M<D) tali da massimizzare la varianza dei dati proiettati.

Prima di cercare le varie componenti cerchiamo la prima(M=1), quindi, si cerca un vettore $u_1$la cui dispersione dei dati sia la massima possibile. Il vettore sarà unitario, ovverosia $\boldsymbol{u_1^T u_1} = \mathbf{1}$, perché siamo interessati solo alla direzione della proiezione.

La media dei dati proiettati è:

$$u_1^T \bar{x}$$

La varianza dei dati proiettati sarà:

$$\frac{1}{N}\sum_{n=1}^{N}(u_1^t x - u_1^t \bar{x})\,(u_1^t x - u_1^t \bar{x})^T$$

dato che il termine comune è u_1^t lo possiamo portare fuori dalle parentesi:

$$\frac{1}{N}\sum_{n=1}^{N} u_1^T (x - \bar{x})\, u_1^{T^T} (x - \bar{x})^T$$

Infine, la proprietà matriciale ci dice che $a^{T^T} = a$, quindi:

$$\boldsymbol{\frac{1}{N}\sum_{n=1}^{N} u_1^T (x - \bar{x})\, u_1 (x - \bar{x})^T = u_1^T S\, u_1}$$

Per trovare u_1 si deve massimizzare la varianza dei dati proiettati, ovvero:

$$\boldsymbol{arg\ max\{u_1^T S\, u_1\} subject\ to\ u_1^T u_1 = 1}\ \text{(ma con l'obbligo che } u_1^T u_1 = 1)$$

Si procede in modo iterativo per trovare le altre componenti e ottenere una base valida in altri sistemi di riferimento. La componente successiva u_2, deve massimizzare la varianza tra tutti i vettori ortogonali a u_1.

La matrice W è definita come segue:

$$\boldsymbol{W} = \begin{pmatrix} \boldsymbol{u_1} \\ \dots \\ \boldsymbol{u_M} \end{pmatrix} = \begin{pmatrix} \boldsymbol{u_{11}} & \boldsymbol{u_{12}} & \dots & \boldsymbol{u_{1D}} \\ \dots & \dots & \dots & \dots \\ \boldsymbol{u_{M1}} & \boldsymbol{u_{M2}} & \dots & \boldsymbol{u_{MD}} \end{pmatrix}$$

La matrice X è definita nel seguente modo:

$$\boldsymbol{X} = \begin{pmatrix} \boldsymbol{x_1} \\ \dots \\ \boldsymbol{x_N} \end{pmatrix} = \begin{pmatrix} \boldsymbol{x_{11}} & \boldsymbol{x_{12}} & \dots & \boldsymbol{x_{1D}} \\ \dots & \dots & \dots & \dots \\ \boldsymbol{x_{N1}} & \boldsymbol{x_{N2}} & \dots & \boldsymbol{x_{ND}} \end{pmatrix}$$

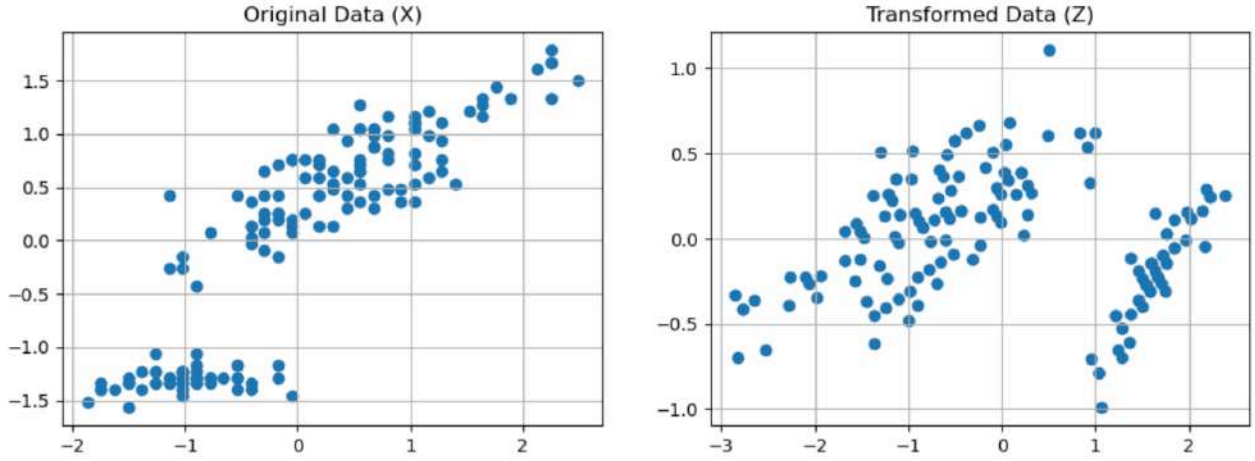
Possiamo quindi proiettare la matrice dei dati X in componenti principali con la seguente formula:

$$\mathbf{Z} = \boldsymbol{XW^T}$$

dove Z è una matrice di dimensioni N×M, chiamata **variabile latente** in quanto contiene **codifiche delle proprietà dei dati** e non vengono osservate. Come nella trasformata di Fourier(studiata ad Interazione e multimedia) la maggiore concentrazione di informazioni sta tra i primi elementi della matrice (contiene la varianza più alta).

Solitamente il computer calcola tutti gli auto-vettori della matrice di covarianza S e vengono messi in una matrice W_{all}. Si ordinano in modo decrescente, (in alto l'auto-vettore con l'auto-valore più grande e così via) e alla fine avviene il **troncamento** scegliendo solo gli M auto-vettori con autovalori maggiori.

Possiamo anche mantenere tutti gli elementi della matrice W_{all}, con W = D. Questo processo può essere visto come una semplice **rotazione dei dati**, in modo che il primo componente diventi il nuovo asse x. W sarà detta **matrice di rotazione**.



Il Whitening(sbiancamento) serve a normalizzare tutte le componenti principali in modo che abbiano tutte la stessa varianza, ossia che ogni componente conti allo stesso modo.

Questo serve per verificare se c'è un'anomalia in una componente, che normalmente varia poco, ma che in presenza di PCA normalizzata non "verrà nascosta" quest'anomalia(anche con conponenti aventi grande varianza).

Per effettuare il Whitening dividiamo la matrice della PCA per la deviazione standard di ogni elemento, ossia:

$$PCA + WHITENING = \begin{pmatrix} \frac{z_{11}}{\sqrt{\lambda_1}} & \cdots & \cdots \\ \cdots & \cdots & \cdots \\ \cdots & \cdots & \frac{z_{ij}}{\sqrt{\lambda_i}} \end{pmatrix}$$

Possiamo quantificare la frazione di varianza spiegata da ogni componente n, nel seguente modo:

$$\frac{\boldsymbol{var(z_n)}}{\boldsymbol{V}}$$

Con **V = var(z_1)+ var(z_2)+...+ var(z_n)**

	z1	z2	z3	z4
z1	4.23	0,00	0,00	0,00
z2	0,00	0,24	-0,00	0,00
z3	0,00	-0,00	0,08	-0,00
z4	0,00	0,00	-0,00	0,02

Se sommassimo V = 4,23+0,24+0,08+0,02 = 4,57 potremmo poi calcolare:

- % var(z1) = 92,5%
- % var(z2) = 5,2%
- ...

Questo approccio può guidarci sulla scelta nel numero giusto di dimensioni da mantenere.

Quando si trasformano i dati con l'analisi delle componenti principali, le nuove variabili hanno un'interpretazione meno diretta. Infatti, una variabile è combinazione lineare tra altre variabili.

Ciascuna componente principale è associata a un vettore di pesi che indica quanto ogni variabile originale contribuisce a quella componente(il valore è chiamato caricamento). Un valore molto alto nel modulo(valore assoluto del vettore) indica che una variabile è molto influente per la componente.